

SAPIENZA
UNIVERSITÀ DI ROMA

Multi-agent Decision Support System for Space Mission Operations

Dipartimento di ingegneria informatica, automatica e gestionale "Antonio Ruberti
Artificial Intelligence and Robotics

Antonio Lissa Lattanzio
ID number 2154208

Advisor
Prof. Danilo Comminiello

Co-Advisor
Daniele Baldassini

Academic Year 2025/2026

Multi-agent Decision Support System for Space Mission Operations
Master's thesis. Sapienza University of Rome

© 2026 Antonio Lissa Lattanzio. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: antoniolissalattanzio@gmail.com

«Ut nihil non iisdem verbis redderetur auditum»

Jorge Luis Borges, Funes el memorioso
(citazione da Plinio il Vecchio, Naturalis Historia, VII, 24)

Abstract

Modern space mission operations rely on multiple heterogeneous systems providing information related to mission planning, spacecraft monitoring, network status, anomaly management, and operational knowledge. The distribution of this information across different sources places a significant cognitive burden on mission operators, who must correlate heterogeneous evidence to maintain situational awareness and support timely operational decisions.

This thesis presents FUNES (Foresight Understanding for Nominal and Emergency Scenarios), an AI-based Decision Support System designed to assist mission managers in satellite ground-segment operations. FUNES adopts a multi-agent architecture based on Large Language Models, Retrieval-Augmented Generation, and tool-based information retrieval. Rather than providing a general-purpose LLM with the complete operational context, the proposed architecture explicitly plans information acquisition, selects the relevant operational sources, refines the retrieved evidence, and synthesizes the final response through specialized agents coordinated by a graph-based orchestration layer.

Our proposed architecture was evaluated through a Proof of Concept against a baseline consisting of a general-purpose Llama 3.3 70B Instruct model operating directly on the complete operational context. The results show that FUNES achieved an overall score of 89.9% on our benchmark, compared with 63.0% for the baseline, corresponding to an improvement of 26.9 percentage points. The results indicate that explicit information selection and orchestration can improve the effectiveness of LLM-based decision support, particularly when relevant information is distributed across heterogeneous operational sources. However, the evaluation also highlights limitations in complex operational reasoning and final answer synthesis.

Contents

1 Introduction **1**

2 Evolution of Natural Language Processing and Decision Support Systems **5**
2.1 Early Symbolic Approaches 5
2.2 Early Neural Approaches 7
2.3 From Static Embeddings to Sequential Modeling 9
2.4 The Transformer Revolution 12
2.5 Scaling Laws and Emergent Abilities 15
2.6 Foundation Models and General-Purpose AI 15
2.7 Alignment and Instruction Tuning 16
2.8 The Reliability Gap: Hallucinations and Uncertainty 17
2.9 Computational Burden and Compression Strategies for LLMs 18
2.10 Reasoning Frameworks for Decision Support 19
2.11 Long-Context Modeling and Context Management 20
2.12 Agentic Large Language Models 21
2.13 Modern LLM-based Decision Support Systems 23

3 System Engineering & Requirement Analysis **29**
3.1 Domain and Mission Context 29
3.2 The Mission Manager's Role and Challenges 30
3.3 FUNES System Overview 30
3.4 System and Software Requirement Specification according to ECSS . 31
3.5 System Engineering Approach 32
3.6 System Requirements Definition 33
3.7 Software Requirement Specification 34
3.7.1 Functional Decomposition and Allocation 34
3.7.2 Software Architecture 37
3.7.3 Software Requirements Derivation 41
3.7.4 Software Design 42
3.7.5 System Data Flows and Operational Use Cases 45
3.7.6 LLM Model Selection 48

4 Proof of Concept Development **51**
4.1 PoC Overview and Architecture 51
4.2 Storage Manager 52

4.2.1 Architecture and Design Principles . . . 52
4.2.2 Rag System . . . 52
4.2.3 Additional Data Interfaces . . . 61
4.2.4 Chat Storage . . . 61
4.2.5 Storage Manager Limitations . . . 63
4.3 AI Manager Module . . . 63
4.3.1 Overview . . . 63
4.3.2 Software Architecture and Implementation . . . 65
4.3.3 Multi-Agent Architecture . . . 66
4.3.4 Graph-Based Orchestration with LangGraph . . . 69
4.3.5 LLM Agent Implementation . . . 72

5 Proof of Concept Evaluation 79
5.1 Evaluation Objectives . . . 79
5.2 Evaluation Scenario . . . 80
5.2.1 Mission Planning . . . 82
5.2.2 Ground Segment and Network Status . . . 82
5.2.3 Operational Events and Tickets . . . 82
5.2.4 Mission Knowledge Base . . . 83
5.2.5 Scenario Design Rationale . . . 83
5.3 Evaluation Tasks and Experimental Setup . . . 84
5.3.1 Evaluation Tasks . . . 84
5.3.2 Experimental Setup . . . 85
5.4 Evaluation Methodology . . . 85
5.5 Results . . . 91
5.5.1 Overall Results . . . 91
5.5.2 Results by question complexity . . . 91
5.6 Discussion . . . 94
5.6.1 Overall Comparison . . . 94
5.6.2 Effect of task complexity . . . 95
5.6.3 Where FUNES Helps and Where It Does Not . . . 98
5.6.4 Architectural Implications . . . 99
5.7 Limitations . . . 101

6 Future Work 103
6.1 Architectural Improvements . . . 103
6.2 LLM and Reasoning Strategies . . . 104
6.3 Benchmark and Evaluation Extensions . . . 105
6.4 Operational Validation . . . 106

7 Conclusion 109

List of Figures

2.1 The original Transformer encoder-decoder architecture proposed by Vaswani et al. [60]. The figure illustrates the composition of encoder and decoder stacks, the multi-head attention modules, feed-forward layers, residual connections, and positional encodings. Source: reproduced from [60]. 14

2.2 Unified framework proposed by [61] for LLM-based autonomous agents. Source: Wang et al.[61]. 23

3.1 Requirement refinement and traceability within the ECSS-based development process. 32

3.2 The funes system context diagram shows the main external systems that interact with the FUNES system. The data flows continuously from the external systems to FUNES and then to the user through the decision support interface, showed in Fig. 3.3. 40

3.3 The Funes high level software architecture is composed of 5 Configuration Items. The data flows from external subsystems to the Data Processing Manager (DPM) module which will turn them into an internal standard data representation after performing cleaning and validation. The data is then fed to the Storage Manager (SM) for beign stored and to the AI Analytics for AI processing. The user utilizes the system through the Decision Support Interface (DSI). The Config & model management module is responsible for the system configuration and AI control. 41

3.4 End-to-end data flow for near real-time anomaly detection and recommendation, from continuous data acquisition and normalization to AI-based analysis, operator notification, and final decision. 46

3.5 Conversational data flow for AI-assisted operator investigation, showing query submission, historical data retrieval, RAG-based analysis, and interactive response generation. 47

3.6 Operator-initiated investigation data flow, showing manual filtering, historical data retrieval, on-demand telemetry analysis, and audit logging of operator actions. 47

4.1 Document chunking workflow implemented in the Storage Manager. The document is preprocessed for metadata extraction like: "chunk_id", "content_hash", "source", "title", "author", "last_modified", "page_number", "chunk_index_in_page", "chunk_size" and "format". Then the repetitive parrterns like page headers and footers are removed. The text is then cleaned and transformed into markdown. Finally the chunking process will turn the text into vectors that are going to be associated with the metadata and the original text. 55

4.2 **Architecture of the Hybrid RAG System.** The system integrates a *Storage Pipeline* (top) that cleans, dynamically chunks, and deduplicates raw documents using SHA-1 hashing, indexing unique content simultaneously into a dense vector database (`ChromaDB`) and a sparse lexical in-memory index (`BM25Okapi`, serialized via `pickle`); and a *Retrieval Pipeline* (bottom) that executes parallel dense-semantic and sparse-keyword searches for a given query, merges the candidate results using Reciprocal Rank Fusion (RRF) based on a unified `chunk_id`, and re-ranks them using a Cross-Encoder model (`BAAI/bge-reranker-base`) to output the top-k most contextually relevant passages for the downstream LLM. 59

4.3 Multi-agent reasoning workflow of the AI Manager, showing the iterative interaction between the Planner, tool-based evidence retrieval, Synthesizer, and Critic agents. 65

4.4 This figure summarizes the workflow of the different agents of the system. The mission manager query is passed to the planner agent that decides what tools and agents have to be called to answer. The results are then passed to the Synthesizer that will provide a final answer. The synthesizer's response and the tools results are then passed to the critic that will determine if other tools are necessary for a more complete response. If so the critic will generate a feedback that is given to the planner for a plan refinement and another tool calling, only the new tools are going to be called if the inputs for the others are unchanged. 76

5.1 Overview of the evaluation scenario: the are 3 ground segment stations GS-01, GS-02, and GS-03, each with 4 antennas, and 4 satellites SAT-01, SAT-02, SAT-03, and SAT-04. The GS-01 is connected to both GS-02 and GS-03 through inter-station network links. The link between GS-01 and GS-02 is degraded, while the link between GS-01 and GS-03 is operational but exhibits increased latency. The GS-01/ANT-02 is unavailable, while GS-01/ANT-03 is degraded. The remaining antennas of GS-01 are operational, while GS-02 and GS-03 are nominal at station level. The ANT positioning is close to the respective ground stations. 81

List of Tables

2.1 Bag-of-Words representation of two syntactically different but lexically identical sentences. 6

3.1 Capability-Function Traceability Summary 37

4.1 Retrieval performance obtained during the Storage Manager validation. 60

4.2 Position of the correct document within the retrieved Top-5 results. 60

4.3 LLM models employed by the specialized agents of the AI Manager Module. The model allocation follows the functional complexity of the agents. The core orchestration agents use the larger Llama 3.3 70B model, as they are responsible for planning, evidence synthesis, and investigation validation. In contrast, the tool-support agents use the smaller Llama 3.1 8B model, since their tasks are more constrained and domain-specific, such as query reformulation, parameter extraction, and relevance filtering. 76

5.1 Evaluation question set. The evaluation is composed of 10 easy questions, 6 medium questions, and 4 hard questions. 88

5.2 Information sources and task type associated with each evaluation question. **Legenda:** KB = Knowledge Base, MP = Mission Planning, NM = Network Monitoring, TM = Ticket Management. 89

5.3 Applicability of evaluation criteria to the evaluation questions. . . . 90

5.4 Benchmark overall performance results for Llama 3.3 vs FUNES grouped by task difficulty. The scores are computed by aggregating the points obtained across all evaluation criteria applicable to the questions belonging to each difficulty category, their details are in Tables 5.5, 5.6, and 5.7. 91

5.5 Benchmark scores for easy questions, each metric has a maximum score of 2 points. The total score is computed by aggregating the points obtained across all evaluation criteria applicable to the question. **Legenda**: AC = Answer Correctness, AI = Anomaly Identification, EC = Evidence Correlation, NF = Noise Filtering, DSS = DSS Role Adherence, ORQ = Operational Recommendation Quality. 92

5.6 Benchmark scores for medium questions, each metric has a maximum score of 2 points. The total score is computed by aggregating the points obtained across all evaluation criteria applicable to the question. **Legenda**: AC = Answer Correctness, AI = Anomaly Identification, EC = Evidence Correlation, NF = Noise Filtering, DSS = DSS Role Adherence, ORQ = Operational Recommendation Quality. 93
5.7 Benchmark scores for hard questions, each metric has a maximum score of 2 points. The total score is computed by aggregating the points obtained across all evaluation criteria applicable to the question. **Legenda**: AC = Answer Correctness, AI = Anomaly Identification, EC = Evidence Correlation, NF = Noise Filtering, DSS = DSS Role Adherence, ORQ = Operational Recommendation Quality. 94

Chapter 1

Introduction

Space mission operations require the continuous coordination and interpretation of large amounts of heterogeneous information. Within a modern ground segment, operational data are distributed across multiple systems responsible for mission planning, spacecraft monitoring, telemetry, network status, anomaly and incident management, and procedural knowledge. Although these systems are individually specialized, mission operations emerge from their continuous interaction. Consequently, maintaining an accurate and coherent view of the overall mission state requires operators to correlate information across multiple sources, often under time constraints and in the presence of incomplete or conflicting evidence. This places a substantial cognitive burden on mission operators, particularly during unexpected events and other situations requiring rapid operational assessment.

The Mission Manager is particularly exposed to this information complexity. The role requires maintaining situational awareness, evaluating operational events, coordinating responses to anomalies, and supporting decisions during both nominal and contingency operations. These activities involve integrating information originating from different operational domains, including telemetry, mission planning, flight dynamics, communication systems, procedures, and operator reports. The increasing volume and heterogeneity of such information motivate the adoption of intelligent Decision Support Systems (DSS) capable of aggregating relevant evidence, reducing information fragmentation, and assisting operators in interpreting the current operational context.

Recent advances in Large Language Models (LLMs) have created new opportunities for AI-based decision support. Their ability to process natural language, interact with external tools, and integrate information from multiple sources makes them promising components for complex operational environments. However, directly providing a general-purpose LLM with a large operational context does not necessarily ensure that the model will identify and use the most relevant information effectively. In particular, LLM-based systems remain affected by hallucinations, irrelevant context, and limitations in evidence aggregation, while the complexity of mission operations requires responses to remain grounded in the available operational information. The literature reviewed in this work also shows that existing LLM-based decision-support systems for space operations are largely domain-specific and prototype-oriented, often addressing individual scenarios rather than

providing a general multi-agent architecture capable of coordinating heterogeneous operational information and reasoning processes.

This thesis addresses this problem through the development of FUNES (Foresight Understanding for Nominal and Emergency Scenarios), an AI-based Decision Support System designed for space mission operations. FUNES acts as an intermediate layer between existing mission operational systems and human operators, collecting heterogeneous operational data and providing context-aware decision support. Rather than relying on a single general-purpose LLM to perform the entire investigation process, the proposed architecture decomposes the task into specialized agents responsible for planning information acquisition, querying operational sources, filtering retrieved evidence, synthesizing the final assessment, and evaluating whether additional investigation is required. These agents are coordinated through a graph-based orchestration mechanism and operate according to an evidence-driven approach. The main objective of the work is therefore not to replace the reasoning capability of the underlying language model, but to improve how heterogeneous operational information is selected, organized, and provided to it. The proposed architecture introduces explicit mechanisms for information planning, source-specific retrieval, evidence refinement, iterative investigation, and response synthesis. In this sense, FUNES is intended to act as a control and orchestration layer around the LLM, reducing irrelevant contextual information while preserving the evidence required to support an operational assessment.

To investigate the effectiveness of this approach, a Proof of Concept was developed and evaluated against a general-purpose Llama 3.3 70B Instruct model operating directly on the complete operational context. The evaluation consists of twenty operational questions organized into three levels of complexity: single-source information retrieval, multi-source information correlation, and decision-support tasks requiring anomaly identification, impact assessment, and operational recommendations. The results show that FUNES achieved an overall score of 89.9% on our benchmark, compared with 63.0% for the baseline, corresponding to an improvement of 26.9 percentage points. The proposed system achieved higher aggregate scores across all three task-complexity categories, with the largest improvement observed for retrieval-oriented tasks. At the same time, the evaluation shows that improved information retrieval and evidence correlation do not automatically guarantee superior operational reasoning. In some complex tasks, FUNES retrieved the relevant evidence but did not fully translate it into the specific operational information requested by the user. These results highlight the importance of the final synthesis stage and indicate that the proposed architecture should be understood primarily as a mechanism for controlling and structuring the information available to the underlying LLM, rather than as a replacement for its reasoning capabilities.

The remainder of this thesis is organized as follows. Chapter 2 reviews the evolution of Natural Language Processing and Decision Support Systems, with particular emphasis on Transformers, Large Language Models, hallucination and reliability issues, Retrieval-Augmented Generation, reasoning frameworks, and agentic architectures. Chapter 3 presents the system engineering and requirement analysis of FUNES, including the mission context, system architecture, operational use cases, and model selection. Chapter 4 describes the development of the Proof of Concept. Chapter 5 presents the experimental evaluation, results, discussion, and

limitations of the proposed system. Chapter 6 discusses future research directions, while Chapter 7 concludes the thesis.

Chapter 2

Evolution of Natural Language Processing and Decision Support Systems

The field of Natural Language Processing (NLP) has experienced a remarkable evolution over the past decades. Early approaches relied on manually engineered representations and statistical models, where language was treated as a collection of symbolic features rather than as a structured and contextual system. Advances in machine learning progressively shifted the focus toward distributed representations and neural architectures capable of learning linguistic patterns directly from data. This progression ultimately culminated in the Transformer architecture, which forms the foundation of modern Large Language Models (LLMs). Understanding this historical evolution is essential for appreciating the capabilities and limitations of current AI-based Decision Support Systems (DSS).

The development of NLP can be broadly divided into three major phases. The first phase is characterized by sparse statistical representations such as Bag-of-Words and n-gram language models. The second phase introduced recurrent neural architectures capable of modeling sequential dependencies. Finally, the third phase replaced recurrence with attention mechanisms, enabling scalable contextual reasoning over long sequences.

2.1 Early Symbolic Approaches

Early NLP systems were primarily based on symbolic representations and sparse statistical features. Rather than learning semantic representations from data, documents were encoded through manually designed features derived from word occurrences. One of the most influential representations was the *Bag-of-Words* (BoW) model [40, 49]. In this representation, each document is transformed into a vector containing the frequency of every word in a predefined vocabulary, while completely disregarding grammar, syntax, and word order.

For example, consider the following two sentences:

"The doctor treats the patient."

"The patient treats the doctor."

Although the semantic meaning of the two sentences is clearly different, the Bag-of-Words representation assigns them identical vectors because they contain exactly the same set of words. The only retained information is the frequency of each term, whereas all structural information is discarded. Consequently, language is reduced to an unordered multiset of independent lexical features.

This can be explicitly observed in their Bag-of-Words representations, as shown in Table 2.1.

Term	Sentence 1	Sentence 2
the	2	2
doctor	1	1
patient	1	1
treats	1	1

Table 2.1. Bag-of-Words representation of two syntactically different but lexically identical sentences.

Despite this limitation, Bag-of-Words is effective for several tasks, including document classification, spam detection, topic categorization, and information retrieval. Its simplicity, low computational cost, and compatibility with classical machine learning algorithms such as Naïve Bayes and Support Vector Machines [40] contributed to its widespread adoption.

However, the independence assumption underlying BoW prevents the model from capturing compositional meaning or contextual relationships between words. Expressions such as "artificial intelligence", "credit card", or "New York" are decomposed into independent terms, ignoring the fact that their meaning emerges from the combination of multiple words rather than from each individual token.

The statistical foundations of these early methods were strongly influenced by the *distributional hypothesis*, originally proposed by Harris [26], which states that words occurring in similar linguistic contexts tend to exhibit similar semantic properties. This principle later became one of the cornerstones of statistical NLP and distributional semantics.

Motivated by the need to incorporate sequential information, researchers introduced *n-gram language models* [21]. Instead of treating words as independent observations, n-gram models estimate the probability of a word by conditioning only on the previous $n-1$ tokens according to the Markov assumption:

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-(n-1)}, \ldots, w_{t-1})$$

Depending on the value of n, different language models can be obtained. A unigram model ($n = 1$) assumes complete independence between words and is therefore equivalent to the Bag-of-Words assumption. A bigram model predicts each word using only the immediately preceding token, whereas a trigram model considers the previous two words.

For instance, given the sentence

"The patient needs medical assistance."

the extracted n-grams are:

- Unigrams: {The, patient, needs, medical, assistance}
- Bigrams: {The patient, patient needs, needs medical, medical assistance}
- Trigrams: {The patient needs, patient needs medical, needs medical assistance}

Unlike Bag-of-Words, these representations preserve a limited notion of local word order, allowing statistical models to learn common expressions, collocations, and short syntactic structures.

Nevertheless, n-gram models remain fundamentally constrained by their fixed context window. Long-range dependencies cannot be modeled because only the previous $n-1$ tokens are considered. For example, in the sentence

> "The physician who examined the patient yesterday **prescribes** the medication."

the relationship between the subject “physician” and the verb “prescribes” spans several intermediate words. A small bigram or trigram model cannot capture such dependencies effectively.

A second limitation concerns scalability. As the value of n increases, the number of possible word combinations grows exponentially. Consequently, even very large corpora contain only a small fraction of all possible n-grams, leading to severe data sparsity problems.

These shortcomings motivated the transition toward neural language models capable of learning continuous distributed representations instead of relying exclusively on explicit frequency counts.

2.2 Early Neural Approaches

To overcome the limitations of n-gram models, early neural approaches introduced the idea of learning distributed representations over sequences. Simple Recurrent Networks (SRNs) by Elman [17] established a foundational architecture for processing sequential data and are considered a precursor to modern Recurrent Neural Networks (RNNs).

The training of neural networks, including recurrent architectures, is based on backpropagation [52]. In the recurrent setting, this is extended to Backpropagation Through Time (BPTT) [24], which enables gradient-based optimization over temporal dependencies by unrolling the network across time.

Unlike feedforward models, RNNs process input tokens sequentially while maintaining a hidden state that acts as a memory of past information. A standard formulation of an RNN is given by:

$$h_t = f(W_h h_{t-1} + W_x x_t + b_h)$$
$$y_t = W_y h_t + b_y$$

This formulation allows, in principle, the modeling of arbitrarily long dependencies through the recurrent hidden state. However, training RNNs in practice is challenging due to the vanishing and exploding gradient problems, which limit their ability to preserve information over long sequences.

To understand this issue, consider a simplified recurrent formulation:

$$h_t = \phi(W h_{t-1})$$

Under BPTT, the gradient of the loss with respect to a hidden state can be expressed as a product of Jacobians across time:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

Taking the norm yields:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \approx \prod_{k=t}^{T-1} \left\| \frac{\partial h_{k+1}}{\partial h_k} \right\|$$

This shows that the gradient magnitude is determined by repeated multiplication of Jacobian matrices across time steps. Depending on their spectral properties, this product can either grow or decay exponentially, leading respectively to exploding or vanishing gradients.

This fundamental limitation significantly hinders the ability of vanilla RNNs to learn long-term dependencies, and motivates the development of more advanced recurrent architectures designed to improve gradient flow over long sequences, like Long Short-Term Memory (LSTM) networks [28].

LSTMs introduce an explicit memory cell c_t designed to preserve long-term information, together with a set of gating mechanisms that regulate information flow. The core idea is to decouple long-term memory from short-term hidden representations.

The LSTM update equations are defined as:

$$\begin{aligned}
I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\
F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\
O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \\
\tilde{C}_t &= \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \\
C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t \\
H_t &= O_t \odot \tanh(C_t)
\end{aligned}$$

The key property of this formulation is the additive update of the cell state c_t, which contrasts with the purely multiplicative transformations of standard RNNs. This structure allows gradients to flow effectively through time, since the derivative of c_t with respect to c_{t-1} is modulated by the forget gate f_t, which can learn to preserve information when needed.

As a result, LSTMs significantly mitigate the vanishing gradient problem and enable learning of long-range dependencies that are difficult to capture with vanilla

RNNs. This makes them particularly effective in sequence modeling tasks such as machine translation, speech recognition, and sequence labeling.

A further simplification of this architecture led to Gated Recurrent Units (GRUs), which combine the forget and input gates into a single update mechanism. GRUs maintain similar performance to LSTMs in many tasks while reducing computational complexity and the number of parameters.

The GRU formulation is defined as:

$$
\begin{aligned}
Z_t &= \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z) \\
R_t &= \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r) \\
\tilde{H}_t &= \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h) \\
H_t &= (1 - Z_t) \odot H_{t-1} + Z_t \odot \tilde{H}_t
\end{aligned}
$$

Despite their success, RNN-based architectures still process sequences sequentially, limiting parallelization during training and inference. This inherent inefficiency, combined with residual difficulties in modeling extremely long dependencies, eventually motivated the development of attention-based models and Transformers, which replaced recurrence with global context modeling.

This progression from BoW and n-gram models to RNNs and gated variants marks a fundamental shift in NLP: from static, order-agnostic representations toward dynamic, stateful systems capable of modeling language as a temporal process. This evolution forms the conceptual bridge toward modern contextual embedding methods and Transformer-based architectures.

2.3 From Static Embeddings to Sequential Modeling

While recurrent architectures addressed the problem of modeling sequential dependencies, they still relied on input representations that did not explicitly capture semantic similarity between words. This motivated the development of distributed word representations, commonly referred to as word embeddings. Building upon the distributional hypothesis, neural language models sought to learn continuous vector representations in which semantically related words occupy nearby regions of the embedding space. This led to the first fundamental advancement in how machines represent language: the introduction of *Word Embeddings*.

Instead of treating words as isolated discrete entities (as in BoW or classical n-grams), Bengio et al. [4] proposed mapping each word i in a vocabulary V to a continuous, real-valued vector $C(i) \in \mathbb{R}^m$ (where $m \ll |V|$), stored in a shared embedding matrix C. A feedforward neural network then processes the concatenation of these vectors within a fixed context window to predict the probability of the next token w_t via a *softmax* layer. The architecture allows semantically similar words are forced to settle in nearby regions of this continuous vector space, allowing the model to automatically generalize from a training sentence (e.g., *"the cat is walking"*) to unseen variations (e.g., *"a dog was running"*).

Although theoretically revolutionary, this early model suffered from a massive computational bottleneck caused by computing the softmax normalization over the

entire vocabulary $|V|$ at each step. Nevertheless, it laid the conceptual foundations for modern representation learning.

The most influential work in this area is Word2Vec, proposed by Mikolov et al. [42]. This model demonstrated that words could be mapped into a high-dimensional vector space where semantic relationships are preserved as geometric distances.

The framework produces these embeddings through two distinct model architectures that leverage local context windows. The Continuous Bag-of-Words (CBOW) model learns representations by predicting a central target word w_t given its surrounding context words within a sliding window of maximum distance C. The vectors of the context words are projected and averaged into a single position, discarding structural word order. The training objective of CBOW is to maximize the average log probability:

$$\mathcal{L}_{\text{CBOW}} = \frac{1}{T}\sum_{t=1}^{T} \log P(w_t \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}) \tag{2.1}$$

Conversely, the Continuous Skip-gram model reverses this logic by using the current central word w_t as input to predict the continuous vector representations of its surrounding context words. The Skip-gram objective function maximizes the following conditional log probability:

$$\mathcal{L}_{\text{Skip-gram}} = \frac{1}{T}\sum_{t=1}^{T} \sum_{-C \le j \le C, j \neq 0} \log P(w_{t+j} \mid w_t) \tag{2.2}$$

The resulting vector space demonstrated linear regularities, meaning that relationships between words were preserved as stable geometric translations. This allowed the models to solve complex semantic and syntactic analogies using simple algebraic operations, such as computing the vector $X = v_{\text{King}} - v_{\text{Man}} + v_{\text{Woman}}$ and finding that its closest cosine distance neighbor was exactly v_{Queen}. Evaluated across extensive benchmark datasets, CBOW achieved state-of-the-art performance on syntactic tasks such as tense and plural inflections, while Skip-gram significantly outperformed all previous architectures on complex semantic questions.

While Word2Vec enabled early NLP tasks to capture basic conceptual analogies, these embeddings remained static: a single vector was assigned to a word regardless of its context (e.g., "bank" in a financial vs. geographical sense). Moreover, Word2Vec does not model word order or syntactic structure, since embeddings are learned independently of the surrounding sentence. As a consequence, it cannot capture long-range dependencies or contextual nuances that are essential in many decision-support scenarios. The representation is therefore powerful at the lexical level but fundamentally limited for tasks requiring a deeper understanding of sequence dynamics.

Shortly after, Pennington et al. [47] introduced GloVe (Global Vectors for Word Representation), which addresses the structural limitations of local window-based methods by leveraging the global statistical information of the corpus. While Word2Vec relies on local context windows, GloVe explicitly constructs a word-word co-occurrence matrix X, where entries X_{ij} record how frequently word j appears in the context of word i. The core insight of GloVe is that the ratio of co-occurrence

probabilities P_{ik}/P_{jk}, rather than the probabilities themselves, carries more meaningful semantic information. To capture this in a linear vector space, the authors propose a weighted least-squares model that seeks to satisfy the following objective:

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(w_i^T \tilde{w} j + b_i + \tilde{b} j - \log Xij \right)^2 \tag{2.3}$$

In this formulation, w_i and $\tilde{w}_j$ represent the target and context word vectors, while b_i and $\tilde{b}_j$ are bias terms that account for individual word frequencies. The function $f(Xij)$ acts as a weighting mechanism, ensuring that rare co-occurrences are not overweighted, while preventing common word pairs (e.g., "the", "and") from dominating the training objective. GloVe demonstrated significant improvements in word analogy tasks and named entity recognition, consistently outperforming its predecessors by producing a vector space with a more robust and meaningful substructure. However, like its predecessors, GloVe remains a static embedding model, providing a single vector representation for each word regardless of the polysemy inherent in natural language.

While traditional word embedding models, such as Word2Vec, have proven successful in mapping semantic relationships into continuous vector spaces, they rely on treating words as atomic and isolated entities. This approach ignores the internal morphological structure of language, which represents a significant limitation for morphologically rich languages and results in the challenge of out-of-vocabulary terms, where rare words fail to acquire reliable representations. To overcome these constraints, the FastText framework [8] was introduced as a powerful extension of the skip-gram architecture that explicitly incorporates subword information. The core innovation of this approach lies in the decomposition of each word into a set of character n-grams. By adding special boundary symbols to distinguish prefixes and suffixes, the model represents every word as the sum of the vector representations of its constituent character n-grams. This design provides several critical advantages that distinguish FastText from previous static embedding methods. First, it ensures high robustness to rare words and previously unseen tokens, as the model can generate meaningful vector representations for any word by composing the vectors of its known n-grams. Furthermore, FastText captures morphological similarities between words, such as different inflected forms and compound words. This feature is particularly useful for morphologically rich languages, where it often performs better than traditional word-level models.

To address the issue of polysemy and contextual nuance, the field moved toward Deep Contextualized Word Representations. A landmark in this transition was ELMo (Embeddings from Language Models), proposed by Peters et al. [48]. Unlike static embedding models, ELMo represents each word as a function of its surrounding context by exploiting a deep bidirectional Language Model (biLM). Rather than relying solely on word-level lookup tables, ELMo constructs input representations from character-level convolutions (CNNs), allowing the model to capture rich morphological information and generate meaningful embeddings even for out-of-vocabulary words.

The biLM consists of multiple stacked bidirectional LSTM layers trained with a self-supervised language modeling objective. The forward language model predicts

the next token given the preceding context, while the backward language model predicts the previous token given the following context. By processing the sequence in both directions, the model captures both short and long-range dependencies. Lower layers tend to encode syntactic and morphological information, whereas deeper layers capture increasingly abstract semantic features.

Instead of using only the final hidden layer, ELMo computes the representation of the k-th token as a task-specific weighted combination of all internal biLM representations:

$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^{L} s_j^{\text{task}} \, h_{k,j}^{LM}$$

where $h_{k,j}^{LM}$ denotes the representation of the k-th token produced by the j-th layer of the bidirectional language model, s_j^{task} are task-specific normalized weights learned during downstream training, and γ^{task} is a trainable scaling parameter.

Although ELMo represented a major breakthrough by introducing contextual embeddings, it still relied on recurrent architectures to model sequences. Consequently, contextual representations had to be computed sequentially, limiting parallelization and making long-range dependency modeling computationally expensive. These limitations ultimately motivated the development of the Transformer architecture, which replaced recurrence with self-attention.

2.4 The Transformer Revolution

The definitive breakthrough occurred with the publication of *"Attention Is All You Need"* [60]. Vaswani et al. proposed a novel architecture entirely based on the **Self-Attention mechanism**, dispensing with recurrence and convolutions. The core innovation lies in the model's ability to weigh the significance of different parts of the input data independently of their distance in the sequence. Mathematically, the Scaled Dot-Product Attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.4}$$

where Q, K, and V represent the Query, Key, and Value matrices, and d_k is the dimension of the keys. The scaling factor $\sqrt{d_k}$ is important for stabilizing the optimization process. Without this normalization term, the dot products between high-dimensional query and key vectors could grow excessively large, causing the softmax function to produce extremely peaked probability distributions and resulting in vanishing gradients during training.

Conceptually, the attention mechanism can be interpreted as a weighted aggregation process. Each query vector determines how much importance should be assigned to all available key-value pairs, enabling the model to selectively focus on the most relevant contextual information for a given token. This selective processing capability constitutes one of the fundamental reasons behind the remarkable effectiveness of Transformer-based architectures.

This mechanism allows the model to build a global understanding of the input, enabling the emergence of "contextual intelligence" where word representations change dynamically based on the surrounding text. In the context of NLP, this marked the transition from local pattern matching toward richer contextual representations capable of modeling long-range semantic and syntactic dependencies. These representations later proved essential for the emergence of increasingly sophisticated reasoning-like behaviors in large-scale language models. A key extension introduced in the original Transformer architecture is the multi-head self-attention mechanism. Instead of computing a single attention map, the model projects the input into multiple sets of query, key, and value spaces and applies the attention function in parallel across these sets.

Formally, multi-head attention is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)W^O \tag{2.5}$$

where

$$head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{2.6}$$

and W_i^Q, W_i^K, W_i^V, and W^O are learnable projection matrices. This formulation enables each attention head to specialize in capturing different forms of dependencies within the data, thereby enriching the overall representation learned by the model.

The outputs from these attention heads are then concatenated and linearly transformed, allowing the model to jointly attend to diverse types of relationships in the input sequence, such as syntactic and semantic patterns simultaneously. By combining multiple attention heads, the network can capture richer representations than a single attention head would allow, without increasing the overall model depth.

Each Transformer block combines multi-head self-attention with a position-wise feed-forward network (FFN). Residual connections and layer normalization are applied around each sub-layer, facilitating stable optimization and enabling the successful training of very deep architectures.

Although self-attention enables every token to attend to all other tokens in the sequence, the mechanism itself is inherently permutation invariant and therefore provides no information about token order. To preserve the sequential nature of language, the original Transformer introduces **Positional Encodings**, which are added to the input embeddings before they are processed by the encoder or decoder.

Rather than learning positional representations, Vaswani et al. proposed deterministic sinusoidal functions capable of encoding both absolute and relative positions within a sequence:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{2.7}$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{2.8}$$

where pos denotes the token position and d_{model} is the embedding dimension. This formulation allows the model to infer relative distances between tokens while

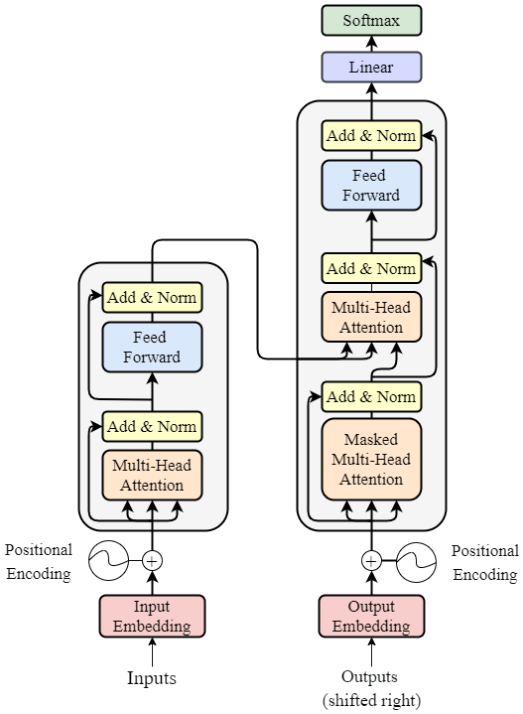


Figure 2.1. The original Transformer encoder-decoder architecture proposed by Vaswani et al. [60]. The figure illustrates the composition of encoder and decoder stacks, the multi-head attention modules, feed-forward layers, residual connections, and positional encodings. Source: reproduced from [60].

also enabling generalization to sequence lengths not encountered during training. Although modern large language models often replace sinusoidal encodings with learned positional embeddings or rotary positional embeddings (RoPE), the original formulation remains a foundational contribution that demonstrated how sequential information could be incorporated without relying on recurrent architectures.

Beyond the attention mechanism itself, the original Transformer introduced a modular encoder-decoder architecture, illustrated in Figure 2.1. The encoder consists of a stack of identical layers, each composed of a multi-head self-attention module followed by a position-wise feed-forward network. Residual connections and layer normalization surround each sub-layer, improving optimization stability and enabling deeper architectures.

The decoder follows a similar structure but incorporates two important differences. First, it employs masked self-attention to prevent each output token from attending to future positions during training. Second, an additional cross-attention layer allows the decoder to attend to the contextual representations generated by the encoder, enabling effective sequence-to-sequence learning for tasks such as machine translation.

Another important practical advantage of Transformer architectures is their massive parallelization potential. Unlike recurrent networks, which process tokens sequentially, each attention layer in a Transformer can compute interactions between all positions in the input sequence simultaneously. As a result, Transformers can

be trained more efficiently on large corpora, enabling the scale-up that underpins modern large language models.

Despite its advantages, the standard self-attention mechanism exhibits quadratic time and memory complexity with respect to the input sequence length, as every token attends to every other token. This limitation has motivated extensive research into efficient attention mechanisms like flash attention [14], which uses tiling to reduce the number of memory reads/writes, or linformer [62], that approximate the standard attention mechanism with low rank matrices.

Collectively, these innovations established the Transformer as the dominant neural architecture for NLP and, subsequently, for a wide range of machine learning applications. More importantly, its scalability and training efficiency made it possible to train models on unprecedented amounts of data and parameters, laying the foundation for the emergence of modern Large Language Models.

2.5 Scaling Laws and Emergent Abilities

The evolution culminated in the transition from task-specific models to *Large Language Models* (LLMs). This shift was driven by the discovery of scaling laws [34], which suggested that model performance, and consequently capabilities on NLP tasks, improve with increases in parameter count, dataset size, and compute, which can indirectly improve the performance of systems that leverage NLP components, including DSS applications.

Furthermore, the introduction of the Pre-training and Fine-tuning paradigm, popularized by BERT [15] and the GPT series [50], enabled these systems to internalize vast amounts of world knowledge. This trajectory led to the discovery of *emergent abilities*: capabilities not explicitly present in smaller models, such as in-context learning and multi-step logical reasoning [65]. These behaviors are referred to as emergent because they are not explicitly programmed nor consistently observed in smaller models, but rather appear once the model exceeds certain scales of parameters and training data. Although the precise mechanisms underlying emergence remain an active area of research, empirical evidence suggests that increasing scale enables qualitatively new capabilities beyond simple improvements in benchmark accuracy. For modern Decision Support Systems, this represents a transition from "engines that process text" to systems that can generate knowledge-informed recommendations, while human oversight remains necessary.

2.6 Foundation Models and General-Purpose AI

The scalability of the Transformer architecture, coupled with the empirical discovery of scaling laws, has catalyzed a paradigm shift in artificial intelligence: the emergence of Foundation Models. As defined by Bommasani et al. [9], a foundation model is any model trained on a vast amount of data, typically using self-supervision at scale, that can be adapted (e.g., fine-tuned) to a wide range of downstream tasks. This represents a fundamental departure from the traditional AI development lifecycle, where models were engineered for specific objectives. Instead, foundation models serve as a versatile base, encoding broad representations of language, logic,

and world knowledge that can be repurposed across diverse domains, including the sophisticated requirements of Decision Support Systems.

A pivotal demonstration of this potential was provided by Brown et al. [10] with the introduction of GPT-3. This work highlighted that when language models are scaled to billions of parameters, they begin to exhibit *in-context learning* capabilities. Unlike previous approaches that required gradient-based fine-tuning to perform a new task, GPT-3 demonstrated the ability to adapt to specific instructions simply by observing a few examples provided within the input prompt (few-shot learning). This shift fundamentally changed the interaction between users and systems, moving from "training" to "prompting" as the primary mechanism for directing model behavior.

However, the rapid expansion of model size brought to light significant challenges regarding computational efficiency. The industry trend initially focused on increasing parameter counts indiscriminately; this was challenged by Hoffmann et al. [29] in their study on *Chinchilla*. The authors demonstrated that the performance of a language model is not solely dependent on its parameter count, but rather on an optimal balance between model size and the volume of training data. Their analysis showed that, for a fixed computational budget, allocating additional resources to training data rather than solely increasing parameter count yielded substantially better performance. This finding established new scaling laws, suggesting that most contemporary models were significantly under-trained. This insight provided a roadmap for developing "compute-optimal" models, enabling superior performance with more efficient resource allocation.

2.7 Alignment and Instruction Tuning

Although large language models acquired broad linguistic knowledge through large-scale pre-training, they were not inherently optimized to follow user instructions or assist with practical tasks like Decision Support Systems, this powerful capability was enabled by the development of *Alignment* techniques. While base LLMs possess extensive statistical information derived from their training corpora, they often fail to follow specific user intents or produce helpful, safe, and concise reasoning. To bridge this gap, Instruction Tuning was introduced, most notably through the work on InstructGPT by Ouyang et al. [44]. This process involves fine-tuning the model on a dataset of human-written prompt-response pairs, instruction tuning changes the training distribution by exposing the model to datasets composed of instructions paired with high-quality responses, thereby teaching it to better interpret and follow user requests.

A critical component of this alignment is Reinforcement Learning from Human Feedback (RLHF). This methodology, popularized by Christiano et al. [12] and later refined for LLMs [58], allows models to optimize their outputs based on human preferences. The RLHF pipeline typically consists of three stages. First, a supervised fine-tuning phase is performed, where the initial language model is trained on high-quality demonstrations generated by human annotators. Second, a reward model is trained from human preference comparisons, where evaluators rank alternative model outputs according to criteria such as helpfulness, relevance, and overall qual-

ity. Finally, the language model is optimized against the learned reward model using reinforcement learning, typically through Proximal Policy Optimization (PPO) [55]. By incorporating human preferences into the optimization process, RLHF guides the model toward outputs that better satisfy alignment objectives, although limitations such as hallucinations, biases, and sensitivity to reward model imperfections remain. While RLHF has been the industry standard for alignment, its reliance on a separately trained reward model and the computational complexity of the Proximal Policy Optimization (PPO) algorithm have led to the development of more stable alternatives. A prominent breakthrough is Direct Preference Optimization (DPO), introduced by Rafailov et al. [51]. DPO simplifies the alignment process by mathematically deriving a direct mapping between the reward function and the optimal policy, effectively bypassing the need for a separate reward model and reinforcement learning training loops. By optimizing the language model directly on preference data via a simple classification loss, DPO offers a computationally efficient and more stable approach to alignment, which has rapidly become a preferred method for fine-tuning modern Large Language Models.

2.8 The Reliability Gap: Hallucinations and Uncertainty

One of the primary obstacle to the integration of LLMs in critical Decision Support Systems (DSS) is the phenomenon of hallucinations, defined as the generation of factually incorrect or ungrounded statements despite high linguistic fluency [31]. This stems from the underlying training objective of next-token log-likelihood, which prioritizes plausible linguistic continuation over factual verification. In a DSS context, **extrinsic hallucinations**, where the output cannot be verified from provided sources or real-world knowledge, are particularly hazardous, as the model may provide a confident but false justification for a recommendation [3].

Beyond explicit errors, a critical risk is the lack of self-awareness regarding uncertainty. State-of-the-art models are often poorly calibrated, meaning their internal probability estimates do not align with their actual correctness [32].

Recent surveys [1] identify a broad taxonomy of hallucination detection methods, designed to assess the factuality and faithfulness of LLM outputs before they are used in high-stakes Decision Support Systems.

- **Retrieval-based methods** detect hallucinations by comparing generated statements against external knowledge sources or retrieved evidence. These techniques, often implemented through RAG pipelines [35], are particularly effective for identifying *extrinsic hallucinations* by checking whether claims are supported by authoritative documents.

- **Uncertainty-based approaches** estimate unreliability through internal model signals such as token-level probabilities, entropy, attention stability, or structured uncertainty propagation. This line of work directly addresses the calibration issues highlighted in [32], offering a principled way to detect when a model is uncertain despite fluent output.

- **Embedding-based methods** measure semantic alignment between inputs, outputs, and reference sources within a shared embedding space. Low semantic similarity indicates drift from the provided context or task grounding.

- **Learning-based detectors**, including supervised, weakly supervised, and agent-based systems, train dedicated models to classify hallucinated versus factual outputs. These techniques capture complex error patterns that cannot be identified through simple probability thresholds or similarity metrics.

- **Self-consistency-based techniques** generate multiple answers or paraphrased queries to assess stability across model outputs. Inconsistencies in meaning or reasoning serve as indicators of potential hallucinations, making these methods valuable when external verification is unavailable.

Overall, this taxonomy underscores that mitigating hallucinations involves not only filtering incorrect content but also understanding where and why LLM reasoning becomes unreliable, which is essential for safe integration into critical DSS.

Recent work suggests that some failure modes traditionally framed as hallucinations may come not only from generative uncertainty but also from structural challenges in long-context evidence aggregation [6]. Studies on retrieval in extended contexts show that the size and distribution of relevant information influence an LLM's ability to locate and integrate the correct evidence. When small but crucial "gold" contexts are embedded within much longer distractor passages, models exhibit reduced aggregation accuracy and increased positional sensitivity. These effects are particularly relevant for retrieval-augmented and agentic systems, where evidence sources often vary considerably in length. In such settings, short but important documents may be underutilized, potentially leading the model to produce unsupported outputs even when the necessary information is included in the input.

2.9 Computational Burden and Compression Strategies for LLMs

Even though LLMs are extremely useful they also are computationally expensive, the development of new models has been charcterized by an increase of the parameters size that grown to hundreds of billions of parameters in the last years, with hardware that are getting more and more powerful and optimized to keep up with the evolution of this technology [36]. The main methods to reduce the cost are applicable during the fine-tuning stage or the inference task:

- Parameter-efficient fine-tuning (PEFT): refers to the process of adjusting the parameters of a pre-trained large model to adapt it to a specific task or domain while minimizing the number of additional parameters introduced or computational resources required [25]. One of their main methods is LoRA [30] which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, reducing the number of trainable parameters.

- Parameters quantization: focuses on reducing the size of the parameters to usually 4 or 8 bits like Smoothquant [66], representative post-training quantization methods include [22], which quantizes weights block-wise and iteratively corrects quantization errors using Hessian information, and Activation-aware Weight Quantization (AWQ), which focuses on reducing the error by preserving the most important weights [38].

- Quantization Aware Fine Tuning: QA-Lora represents a hybrid approach between PEFT and quantization, where the model is fine-tuned while accounting for quantization constraints [67].

- Pruning: focus on lightening the model size by removing the connections or neurons that are less important [23, 57]

- Knowledge Distillation: is a technique where a smaller model (the student) is trained to replicate the behavior of a larger model (the teacher), effectively transferring knowledge from the teacher to the student [27, 53].

Compression techniques inevitably involve trade-offs between efficiency and performance, as aggressive quantization, pruning, or parameter-efficient fine-tuning may degrade model accuracy or require additional implementation effort, and task-specific performance variability must be considered [22, 23, 30, 38].

2.10 Reasoning Frameworks for Decision Support

Beyond architectural and alignment improvements, the operational use of LLMs in decision-making has been revolutionized by specialized prompting and reasoning frameworks. The introduction of **Chain-of-Thought (CoT)** prompting [64] demonstrated that forcing a model to generate intermediate reasoning steps significantly enhances performance on complex logical tasks.

While CoT provides a "slow thinking" approach essential for auditing a DSS, relying on a single greedy-decoded trajectory can lead to error propagation. To mitigate this, **Self-Consistency** [63] introduces a decoding strategy that samples multiple reasoning paths and selects the most frequent marginal result, leveraging the intuition that complex problems often admit multiple paths to a unique correct answer.

Further advancing non-linear reasoning, the **Tree of Thoughts (ToT)** framework [68] generalizes the CoT paradigm by representing intermediate reasoning steps as discrete "thoughts" organized in a tree structure. Unlike CoT, where the model commits to a single sequential chain of deductions, ToT enables the exploration of multiple candidate reasoning trajectories through a search process. At each step, the LLM can generate alternative intermediate states, evaluate their quality through self-assessment or external feedback mechanisms, and prune unpromising branches. In the context of Decision Support Systems (DSS), ToT is especially valuable for scenarios characterized by uncertainty, long planning horizons, or conflicting objectives, where premature commitment to a single reasoning path may lead to suboptimal recommendations.

Building upon this idea, the **Graph of Thoughts (GoT)** framework [5] relaxes the hierarchical constraints of tree-based exploration by modeling reasoning processes as directed graphs rather than strictly branching structures. While ToT assumes that each new thought derives from a single previous state, GoT allows arbitrary dependencies between intermediate reasoning units, enabling operations such as aggregation, merging, refinement, and transformation of partial solutions. For DSS applications, GoT provides a more flexible framework for representing interconnected evidence, alternative hypotheses, and multi-stage decision processes, where information may need to be combined, revisited, or propagated across different reasoning components.

The **ReAct** framework [69] represents a fundamental transition from isolated reasoning toward agentic decision-making by interleaving reasoning and action. Instead of generating a final answer based solely on the information encoded in the model parameters, ReAct structures the interaction as an iterative loop composed of three main phases: reasoning, action, and observation. The model first generates a reasoning trace to determine the next objective or strategy, then executes an external action, such as querying a database, retrieving documents, invoking an API, or interacting with a software environment, and finally incorporates the resulting observations into subsequent reasoning steps.

This mechanism allows LLMs to dynamically acquire information, verify intermediate assumptions, and adapt their behavior according to external feedback. Compared with purely reasoning-oriented approaches such as CoT or ToT, ReAct introduces an explicit connection between cognitive processes and environmental interaction, reducing the limitations caused by static knowledge and closed-world assumptions. For Decision Support Systems, this capability is particularly relevant because decisions often depend on continuously changing data sources, domain-specific tools, and feedback from operational environments.

When integrated with Retrieval-Augmented Generation (RAG) [35], ReAct-based systems can combine structured reasoning with external knowledge retrieval, enabling LLMs to identify information gaps, retrieve relevant evidence, evaluate available alternatives, and progressively refine recommendations. This combination shifts LLMs from static predictive models toward interactive reasoning agents capable of supporting complex decision processes in dynamic and uncertain environments.

2.11 Long-Context Modeling and Context Management

While early Transformer-based models were limited to relatively short context windows, modern LLMs have progressively expanded their capacity to process longer sequences. However, the standard Transformer architecture suffers from a quadratic computational and memory complexity, $O(n^2)$, relative to sequence length n. This constraint turns the context window into a significant bottleneck: as sequence length increases, the memory footprint of the Key-Value (KV) cache grows linearly, often exhausting GPU VRAM during inference, while the self-attention mechanism becomes the primary computational latency driver. To overcome these limitations, recent research has focused on architecture and positional encoding advancements.

Initial approaches relied on linear attention approximations, but many state-of-the-art models primarily adopt **Rotary Positional Embeddings (RoPE)** [59]combined with scaling techniques. Works such as Position Interpolation [11] and **YaRN** [46] have demonstrated that context windows can be extended far beyond pre-training limits by mathematically re-scaling the rotational frequency of embeddings, allowing models to 'perceive' longer sequences without extensive re-training.

Another problem is the increased risk of hallucinations and reasoning errors when the model is forced to process long contexts, as the relevant information may be diluted among irrelevant content. This is particularly problematic in Decision Support Systems, where critical evidence may be embedded within lengthy documents or multi-source data streams. The work of Liu et al. [39] demonstrates that current language models, including those explicitly designed to handle long contexts, struggle to robustly retrieve and use information located in the middle of their input window. Instead of treating all parts of the context equally, these models exhibit a U-shaped performance curve where performance is highest when relevant information is placed at the very beginning (primacy bias) or at the very end (recency bias) of the input. Performance drops significantly when the model is forced to access information located in the middle of the context. In some cases, performance is even worse than if the model had been given no context at all.

2.12 Agentic Large Language Models

The evolution from Large Language Models to Agentic Large Language Models represents a shift from passive text generation toward autonomous task execution. While traditional LLMs operate primarily as reactive systems that map user prompts to generated responses, agentic architectures embed language models within broader computational systems that provide planning, memory, action executions, and, in some cases, profiling modules [61].

The **profiling module** defines the role, objectives, capabilities, and behavioral constraints of the agent.

The **memory module** can store data in several formats, each offering unique trade-offs between semantic richness, retrieval efficiency, and ease of manipulation: Natural Language, Embeddings, Databases, Structured Lists, or hybrid. The selection of data is typically governed by three criteria: recency, relevance, and importance. Memory is commonly divided into short-term memory, which maintains the context of the current interaction, and long-term memory, which stores persistent knowledge, previous experiences, or retrieved information across multiple tasks [61]. Effective memory management is essential for long-horizon reasoning and continual adaptation. A notable implementation of this concept is provided by Generative Agents [45], where memories are organized as a continuously growing memory stream. Rather than retrieving past experiences indiscriminately, the agent ranks memories according to their recency, semantic relevance, and estimated importance. Furthermore, higher-level reflections are periodically synthesized from accumulated experiences, enabling the agent to form persistent knowledge that can guide future planning and decision-making.

The **planning module** is responsible for managing long-horizon tasks, orchestrating complex workflows, and ensuring coherent action sequences across multiple steps. The planning can be performed without feedback, where the agent does not get a feedback that influences its behaviour or with feedback for tasks where initial plans frequently fail due to unforeseen environmental dynamics or overly complex preconditions. The feedback can be provided by the environment, by other agents, or by a human-in-the-loop. Planning is commonly supported by reasoning strategies such as Chain-of-Thought (CoT), Tree of Thoughts (ToT), or Graph of Thoughts (GoT) as described in 2.10.

The **action module** serves as the final downstream component of an autonomous agent architecture, translating high-level decisions into tangible outcomes through direct environmental interaction.

An agent's ability to execute these actions relies on a synergy between the LLM's inherent reasoning and a suite of external resources:

- Tool Integration: External tools bridge critical gaps in an LLM's capabilities, such as mitigating hallucinations or providing domain-specific expertise.
- APIs: Agents utilize APIs to interface with web services, calculators, compilers, and specialized software libraries, enabling them to perform precise, executable tasks.
- Databases and Knowledge Bases: allow agents to query structured data, ensuring that actions are grounded in verifiable, external information.
- External Models: Agents can offload complex or multimodal processing to specialized supplementary models.

Tool integration was further formalized by Toolformer [54], which demonstrated that language models can learn when and how to invoke external tools during generation through self-supervised training. This capability extends the model beyond its parametric knowledge by enabling access to calculators, search engines, translation systems, or APIs whenever additional information or computation is required.

One of the most influential frameworks for planning and interaction is ReAct (Reason + Act) [69], which interleaves reasoning steps with actions performed in the external environment. Rather than generating an entire solution in a single pass, the agent alternates between internal reasoning ("Thought"), executing an action (e.g., querying a search engine or database), and incorporating the resulting observations into subsequent reasoning. This iterative loop:

$$\text{Thought} \rightarrow \text{Action} \rightarrow \text{Observation}$$

enables adaptive decision-making and significantly improves performance on knowledge-intensive and interactive tasks.

Beyond planning and acting, recent agentic systems increasingly incorporate explicit self-reflection mechanisms. Reflexion [56] works on the principle that LLMs can improve by analyzing their own past performance through natural language. First, the Actor generates a trajectory, which is essentially a sequence of thoughts and actions intended to solve a given task. Once the action is taken, the Evaluator

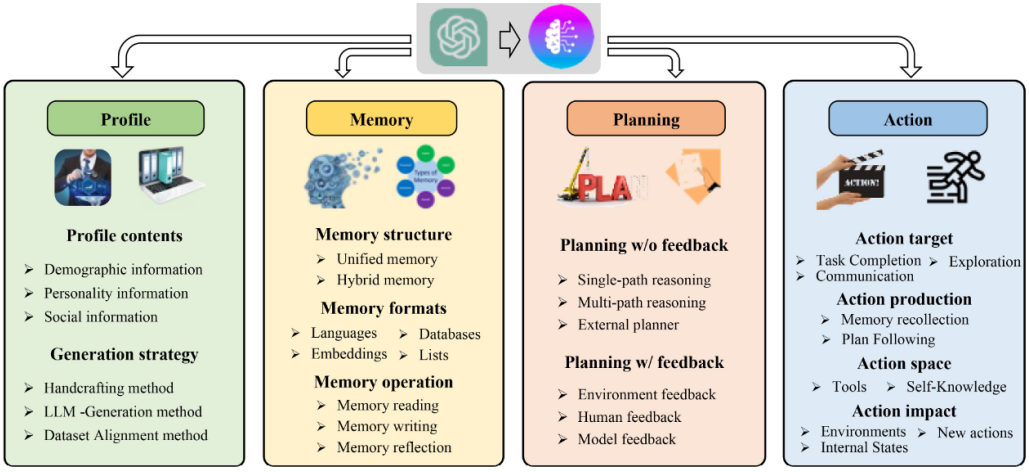


Figure 2.2. Unified framework proposed by [61] for LLM-based autonomous agents. Source: Wang et al.[61].

assesses the resulting output by using mechanisms such as heuristics, unit tests, or another LLM, ultimately providing a score or specific feedback based on performance. In instances where the task fails, the system triggers the Self-Reflection module, where an LLM analyzes the trajectory and the feedback to write a verbal "lesson learned." This reflection is then stored in the agent's Memory, where it is retrieved and utilized by the Actor in the next trial, effectively guiding the agent toward the correct behavior through its accumulated experience.

Together, these components transform LLMs from passive generators into autonomous systems capable of reasoning, interacting with external environments, retaining experience, and improving their behavior over time. This transition provides the foundation for more advanced architectures, including multi-agent systems and agent-based Decision Support Systems, where multiple specialized components collaborate to solve complex tasks.

2.13 Modern LLM-based Decision Support Systems

The potential for LLMs to transform decision-making is underscored by Eigner and Händler [16], who provide a comprehensive analysis of the determinants that govern successful human-AI collaboration. While traditional decision-making follows the classic intelligence-design-choice phases, the integration of LLMs introduces a far more complex set of variables. Their research highlights that effective support is not merely a technical challenge but a socio-technical one, involving a delicate interplay between technological aspects (such as prompt engineering and transparency), psychological factors (such as user mental models, trust, and decision-making styles), and decision-specific determinants (such as task difficulty and accountability). By mapping these interdependencies into a consolidated dependency framework, Eigner and Händler emphasize that improving decision quality requires moving beyond simple model performance to focus on the alignment between AI capabilities and human cognitive needs. The state-of-the-art in this domain has recently moved toward

adaptive, multi-agent architectures that operationalize these reasoning frameworks within high-stakes environments. A primary example is the work of Öğdü et al. [43], who developed a multi-layered, adaptive Clinical Decision Support System (CDSS). Unlike traditional static models, this architecture integrates biomedical RAG with real-time web intelligence through a collaborative framework of autonomous agents. The system introduces several key innovations:

- **Timeliness and Evidence Freshness:** By incorporating a dedicated web-intelligence agent, the system mitigates the "knowledge cutoff" problem of static LLMs, ensuring recommendations are grounded in the latest clinical guidelines.
- **Confidence-Aware Optimization:** The system utilizes an adaptive feedback loop that triggers re-querying and hypothesis refinement whenever internal confidence scores fall below a defined threshold ($\tau = 0.65$).
- **Domain-Specific Grounding:** To ensure precision in medical contexts, the system employs a specialized BioBERT-based vector database for processing complex clinical jargon.

Validated on the MedQA and PubMedQA datasets, achieving accuracies of 94% and 88% respectively, this approach demonstrates how the integration of multi-source data fusion and adaptive optimization loops can overcome the reliability barriers of standard LLM-based reasoning.

Building upon this transition toward adaptive architectures, Alkayyal and Malberg introduce StrategicAI [2], a Decision Support System (DSS) tailored for strategic business contexts. StrategicAI operationalizes the logic tree methodology to manage the high uncertainty and cognitive load inherent in strategic planning. The system functions through a three-phase multi-agent process:

- **Recursive Problem Decomposition:** Rather than providing a single output, the system uses a recursive, multi-agent architecture to decompose complex strategic problems into granular issue trees, mapping potential causes and solutions hierarchically.
- **Evidence-Based Retrieval Augmentation:** The system integrates a retrieval-augmented LLM (RAG) that draws from both internal organizational documentation and external market data. This ensures that the generated tree nodes are not merely speculative, but grounded in evidence-based business intelligence.
- **Human-in-the-loop Orchestration:** Addressing the socio-technical challenges identified by Eigner and Händler, StrategicAI maintains a collaborative workflow where the AI structures and validates the logic, while the human expert retains control over the strategic direction.

By automating the structural decomposition of business problems and mitigating cognitive biases through evidence-backed reasoning, StrategicAI demonstrates

the utility of applying adaptive, agentic workflows to high-stakes management decisions where traditional static models often fail to account for complex, non-linear variables.

In the domain of urban governance, Kalyuzhnaya et al. [33] propose a multi-agent LLM framework designed to bridge the gap between strategic urban planning and operational execution. Moving beyond standalone model limitations, the authors introduce a modular architecture that integrates LLMs with existing municipal information systems to address the inherent fragmentation of urban data. The system operates through a hierarchical orchestration framework:

- **Hierarchical Orchestration Layer:** The architecture is centered on an "agent-conductor," which serves as the system's intelligent core. This agent is responsible for query intent classification and routing, ensuring that incoming requests are directed to the appropriate specialized processing pipelines.
- **Specialized Processing Agents:** The system utilizes three primary functional agents to handle complex urban queries:
  - *Tool Calling Agent:* Manages real-time data retrieval via APIs, providing access to transportation metrics, urban service availability, and geospatial layers.
  - *RAG Operations Agent:* Executes retrieval-augmented generation against a dedicated vector database, grounding the LLM's reasoning in validated regulatory documents and municipal development frameworks.
  - *Answer Summarization & Validation Agents:* Synthesizes retrieved data and validates the generated responses for consistency, reducing the risk of hallucinations often associated with non-grounded LLM implementations.
- **Integration with Digital Urban Platforms:** Unlike isolated decision models, the system maintains a bidirectional link with city information systems, such as the Digital Urban Platform of St. Petersburg. This allows the agents to process heterogeneous data, enabling the system to provide context-aware strategic recommendations.

Unlike conventional business intelligence systems, StrategicAI does not only retrieve and summarize information but assists decision-makers in structuring ambiguous problems before evaluating possible alternatives.

While the urban governance framework focuses on synthesizing administrative data for high-level decision support, the technical principles of modular agentic architectures have also been validated in high-precision scientific domains. Coscientist [7] bridges the gap between high-level reasoning and physical execution in chemical laboratories. While the urban governance model prioritizes RAG operations for regulatory compliance, Coscientist emphasizes a closed-loop "tool-use" paradigm:

- **Action-Oriented Modularity:** The system decomposes complex research tasks into discrete commands (e.g., web search, code execution, hardware API interaction).

- **Grounding via Dynamic Documentation:** Beyond static RAG, Coscientist employs vector-based retrieval on technical documentation to dynamically learn and interact with APIs it was not explicitly trained to handle.
- **Autonomous Self-Correction:** A significant contribution of this work to the field of decision support is the system's capacity for diagnostic iteration. When an generated protocol fails (e.g., an invalid hardware call), the agent autonomously queries its documentation module to rectify the error. This "closed-loop" reasoning is essential for moving toward autonomous systems capable of operating in high-stakes environments where human oversight may be intermittent.

By comparing these two domains, urban governance and experimental chemistry, it becomes clear that the utility of LLM-based decision support lies not in the model's inherent knowledge, but in its ability to operate as an intelligent orchestrator within a modular, tool-rich ecosystem. Unlike conventional business intelligence systems, both StrategicAI and the Coscientist framework shift the paradigm from retrieval to intervention, assisting decision-makers in structuring ambiguous problems and executing evidence-based alternatives in real-time.

To address the operational challenges of future deep-space exploration, the work of Kaisheng et al. [37] proposes an Earth-independent, agentic AI framework for Extravehicular Activity (EVA) mission planning. The system functions as a localized "virtual flight controller," integrating a fine-tuned Large Language Model (LLM) with a robust Retrieval-Augmented Generation (RAG) architecture and external physics-based simulation models.

The core innovation of the proposed system lies in its ability to synthesize heterogeneous data sources into actionable and traceable decision support. By utilizing an agentic orchestration layer, the system performs iterative, chain-of-thought reasoning that mimics the analytical rigor of ground-based Mission Control.

The decision support system provides critical utility in three primary domains:

- **Dynamic Replanning:** Enables astronauts to conduct "what-if" simulations, allowing for the rapid recalculation of mission timelines and resource margins in response to unexpected discoveries or environmental changes.
- **Off-Nominal Contingency Management:** Provides immediate, context-aware troubleshooting guidance during system failures, ensuring safety margins are continuously monitored even when connectivity to Earth is unavailable.
- **Cognitive Load Reduction:** Acts as an intelligent interface that abstracts complex sensor data and procedural constraints into concise, explainable recommendations, thereby allowing the crew to maintain high levels of operational awareness during high-stakes maneuvers.

By grounding LLM-generated outputs in verifiable physics models and authoritative mission documents, the architecture ensures high transparency and auditability. This framework serves as a foundational step toward increasing crew independence and operational safety for long-duration missions on the lunar surface and beyond.

Addressing the integration of Large Language Models into space-faring autonomy, Maranto [41] proposes LLMSAT: a hierarchical architecture that positions an LLM as a high-level mission operations planner, decoupled from low-level spacecraft control. To mitigate the inherent risks of non-deterministic LLM behavior, this architecture employs a 'correctness-by-construction' paradigm, where the agent interacts exclusively with validated flight software services. Safety is enforced through a tripartite validation framework: function-level argument checking, plan-level simulation-based validation, and reflexive spacecraft-level safety overrides. The implementation leverages Retrieval-Augmented Generation (RAG) to manage context-window constraints and facilitate experiential learning, demonstrating a shift toward agentic mission management while emphasizing the need for robust, ground-validated command interfaces in space-computing environments.

Despite these promising developments, current LLM-based DSS for mission operations remain largely domain-specific and prototype-oriented. Existing systems typically focus on individual operational scenarios, such as EVA planning or spacecraft autonomy, rather than providing a general-purpose, multi-agent decision support architecture capable of integrating heterogeneous operational data, procedural knowledge, human oversight, and coordinated reasoning. This gap motivates the research presented in this thesis, which investigates a multi-agent LLM-based Decision Support System tailored for space mission operations.

Chapter 3

System Engineering & Requirement Analysis

3.1 Domain and Mission Context

The ground segment of a space mission comprises all the facilities, software systems, communication infrastructure, and operational personnel responsible for supporting spacecraft operations throughout the mission lifecycle. It acts as the interface between the spacecraft and mission operators on Earth, enabling mission planning, spacecraft monitoring, command transmission, and anomaly management.

Within the Fucino Space Centre, the core ground segment is organized into several specialized subsystems, each responsible for a specific operational function. The main components include:

- **Satellite Control Center (SCC)**: responsible for spacecraft monitoring and command execution;
- **Mission Planning Center (MPC)**: responsible for planning operational activities and scheduling mission tasks;
- **Telemetry, Tracking and Control (TT&C)**: responsible for telemetry acquisition, spacecraft tracking, and command uplink;
- **Flight Dynamics**: responsible for orbit determination, maneuver planning, and trajectory analysis;
- **Mission Monitoring and Control (MMC)**: responsible for supervising mission execution and operational status;
- **Planning and Control Center (CPCM)**: responsible for coordinating mission planning and control activities and supporting the execution of operational procedures;
- **Communication Infrastructure (COMI)**: responsible for enabling data exchange between the different ground segment subsystems.

Although each subsystem performs a well-defined function, mission operations emerge from their continuous interaction. Operational data are generated by multiple heterogeneous sources and distributed across independent software systems. Consequently, obtaining a complete and coherent picture of the mission state requires correlating information originating from several subsystems operating simultaneously.

This distributed operational environment makes the ground segment inherently information-intensive and places significant cognitive demands on mission operators responsible for supervising mission execution.

3.2 The Mission Manager's Role and Challenges

Among the different operational roles, the Mission Manager is responsible for supervising the overall execution of the mission and coordinating the activities of the operational teams. The Mission Manager maintains situational awareness of the mission, evaluates operational events, coordinates responses to anomalies, and supports decision-making during both nominal and contingency operations.

To perform these tasks, the Mission Manager must continuously integrate information coming from multiple sources, including spacecraft telemetry, mission planning systems, flight dynamics analyses, communication systems, operational procedures, and reports generated by different operators. These information sources are often heterogeneous, asynchronous, and, in some situations, may provide incomplete or even conflicting evidence regarding the current mission state.

The increasing complexity of modern space missions has significantly amplified the amount of information that operators must process in real time. During critical operational phases or unexpected events, rapidly identifying relevant information and understanding its implications becomes particularly challenging. Maintaining situational awareness under these conditions requires considerable expertise and imposes a substantial cognitive workload on the Mission Manager.

These challenges motivate the adoption of intelligent Decision Support Systems capable of aggregating information from heterogeneous sources, assisting operators in assessing the current operational context, and supporting timely and informed decision-making.

The system we are going to develop is called FUNES: Foresight Understanding for Nominal and Emergency Scenarios. The objective of FUNES is to support mission managers during operational decision-making by reducing information fragmentation, improving situational awareness, and decreasing the cognitive workload associated with complex mission scenarios.

3.3 FUNES System Overview

FUNES (Foresight Understanding for Nominal and Emergency Scenarios) is an AI-based Decision Support Tool designed to support mission operations within an Earth Observation ground segment environment. The system acts as an intermediate layer between existing mission operational systems and human operators, collecting heterogeneous operational data, analysing mission conditions, and providing

decision-support outputs.

FUNES does not replace mission operators or execute autonomous spacecraft operations. Instead, it supports human decision-making by improving situational awareness, reducing information fragmentation, and providing recommendations based on available mission knowledge and operational data.

The main functions provided by FUNES include:

- acquisition and processing of heterogeneous mission data;
- detection of anomalies and operational degradations;
- correlation of information from different mission domains;
- generation of operational recommendations;
- provision of explainable AI outputs to support operator decisions.

The system operates within the existing ground segment infrastructure and interfaces with external mission systems, including telemetry sources, mission planning systems, flight dynamics systems, monitoring tools, and operator interfaces.

3.4 System and Software Requirement Specification according to ECSS

Following the European Coordination for Space Standardization (ECSS) requirement engineering approach, incorporating standard guidelines for system engineering [18], software development [19], and software product assurance [20]. The FUNES development process produced two main specification documents:

- **System Requirements Document (SRD):** defining the technical requirements of the FUNES system as a complete entity within the mission ground segment environment;
- **Software Requirements Specification (SRS):** defining the software-level requirements derived from the system specification and allocated to the software components.

The System Requirements Document represents the first level of technical specification. It describes the expected behaviour of the system independently from the implementation details and defines the capabilities required to satisfy operational needs. The SRD includes:

- **Functional requirements**, describing the capabilities provided by the system;
- **Performance requirements**, defining measurable system constraints;
- **Interface requirements**, describing interactions with external and internal entities;

- **Operational requirements**, defining system behaviour during different operational modes;

- **Safety, dependability, data management, and AI-specific requirements**, addressing system safety, dependability, data management, and the specific constraints associated with the use of AI technologies.

According to ECSS principles, each requirement is written using a structured and verifiable formulation based on the "shall" statement. Requirements are required to be necessary, unambiguous, atomic, implementation-independent, verifiable, and traceable.

The Software Requirements Specification is derived from the system specification by allocating system functions to software components. It defines the expected behaviour of the software architecture, including software functionalities, internal interfaces, data processing requirements, AI module interactions, and verification criteria.

The relationship between the two documents follows a hierarchical requirement flow where system-level requirements are progressively refined into software-level requirements while maintaining bidirectional traceability through a Requirement Traceability Matrix (RTM), the process is summarized in Figure 3.1.

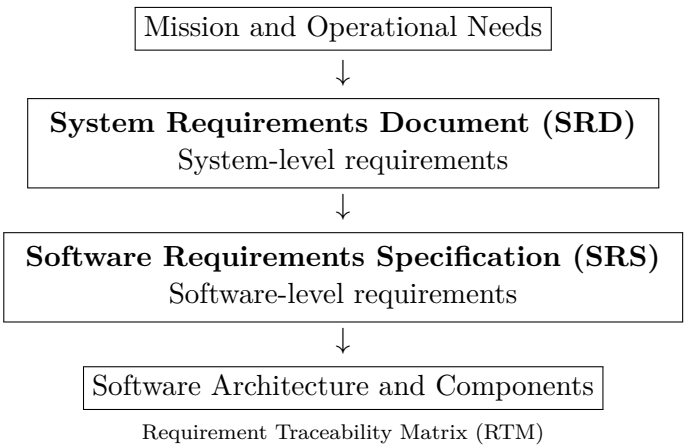


Figure 3.1. Requirement refinement and traceability within the ECSS-based development process.

3.5 System Engineering Approach

The development of the FUNES system followed a system engineering approach based on the principles defined by the ECSS. The objective of the system engineering process was to transform operational needs identified within the mission control environment into a structured and traceable set of system and software requirements.

The adopted approach followed a top-down requirement derivation process, starting from the analysis of the operational scenario and stakeholder expectations, and progressively refining them into functional, performance, interface, operational, and AI-specific requirements. The requirement definition process was performed according to the principles of ECSS-E-ST-10C [18], ensuring that each requirement was necessary, unambiguous, verifiable, traceable, atomic, and implementation-independent.

The identification of stakeholder needs represents the first step of the system engineering process. For the FUNES system, the primary stakeholders were identified within the mission operations environment, including Mission Managers, spacecraft operators, ground segment operators, and system maintenance personnel.

The analysis of the operational scenario highlighted several challenges associated with current mission operations. Mission operators must continuously monitor heterogeneous information sources, including spacecraft telemetry, mission planning information, ground segment status, flight dynamics data, network data, and operational reports.

During nominal and non-nominal operations, the integration and interpretation of such information require significant human effort, especially when rapid decisions are required. Therefore, the main operational need identified for FUNES was the capability to improve mission situational awareness by aggregating heterogeneous operational information and providing decision-support capabilities while preserving human authority over final operational decisions.

3.6 System Requirements Definition

The System Requirements Document (SRD) defines the technical requirements governing the functional behavior and operational constraints of the FUNES system.

Following the ECSS-E-ST-10C standard, a total of around 160 requirements were formulated to guarantee end-to-end traceability between operational needs and verification activities. The main requirements are:

- **Functional Requirements (FUN - 86 requirements):** Core system capabilities, including data acquisition, anomaly detection, recommendation generation, explainability, and human-in-the-loop interaction.
- **Performance Requirements (PER - 6 requirements):** Quantitative constraints on accuracy, latency, reliability, scalability, and resource consumption.
- **Interface Requirements (INT - 12 requirements):** External integrations with telemetry sources, mission planning systems, databases, and user interfaces.
- **Operational Requirements (OPR - 12 requirements):** System behavior during nominal, validation, training, and maintenance operations.
- **Non-functional Requirements (NFR - 13 requirements):** Cross-cutting quality attributes addressing system robustness, interface reliability, and operational constraints.

- **AI Requirements (AI - 42 requirements):** Requirements related to the Artificial Intelligence (AI) capabilities of the FUNES system, including model selection, data usage, explainability, bias mitigation, and lifecycle management.

3.7 Software Requirement Specification

The Software Requirements Specification (SRS) was derived from the System Requirements Document following the ECSS top-down requirement engineering process. While the SRD defines the expected behaviour of the system from an operational perspective, the SRS allocates these requirements to software components and specifies the software architecture required to implement them.

3.7.1 Functional Decomposition and Allocation

Starting from the system-level requirements defined in the SRD, the FUNES system was decomposed into a set of functional capabilities representing the main operational objectives of the Decision Support System.

Following the ECSS top-down requirement engineering approach, each capability was further refined into a set of lower-level system functions. These functions describe the internal behaviours required to implement the corresponding capability and provide the basis for the allocation of requirements to software Configuration Items.

The main capability-to-component allocation is summarized in Table 3.1.

C-01: Mission Data Ingestion Capability The system shall be capable of acquiring, normalizing, validating, and correlating heterogeneous mission data from multiple external sources in near real-time. This capability is decomposed in the following set of functions:

- **F-01 Data acquisition from external systems:** Establishes and manages secure communication channels with external sources to ingest mission data streams.
- **F-02 Data format normalization and harmonization:** Converts heterogeneous incoming data structures into a unified, standardized internal data model.
- **F-03 Data validation and quality assessment:** Inspects ingested data against predefined syntactic and semantic rules to filter out corrupted or non-compliant payloads.
- **F-04 Timestamping and cross-source correlation:** Appends synchronized system timestamps and correlates data packages sharing spatial, temporal, or logical relationships.
- **F-05 Secure ingestion and integrity verification:** Cryptographically verifies data origin and integrity (e.g., via checksums/signatures) to prevent tampering during ingestion.

C-02: AI-based Analysis and Decision Support Capability The system shall be capable of performing AI-driven analysis of mission data to detect anomalies, assess system performance, and generate operational recommendations with associated confidence and explainability information. This capability is decomposed in the following set of functions:

- **F-06 Anomaly detection inference:** Evaluates data streams to identify deviations from nominal events.
- **F-07 Trend and pattern analysis:** Processes historical and real-time data to identify long-term behaviors, recurring operational patterns, and performance degradation.
- **F-08 Operational recommendation generation:** Translates analytical insights and detected anomalies into actionable mitigation strategies or operational decisions for the operator.
- **F-09 Confidence scoring of analytical outputs:** Computes a probabilistic metric representing the certainty level of each detected anomaly and generated recommendation.
- **F-10 Explainability artifact generation:** Produces human-readable justifications and feature-importance metrics to explain the rationale behind AI-driven outputs.

C-03: Mission Status Visualization and Operator Interaction Capability The system shall be capable of providing mission status visualization, including alerts, recommendations, and operational reports, and supporting operator interaction and feedback collection. This capability is decomposed in the following set of functions:

- **F-11 Conversational Interface:** Handles the conversation with the LLM-based system.
- **F-12 Mission status dashboard generation:** Renders the primary user interface, displaying real-time system metrics, operational KPIs, and overall mission health.
- **F-13 Alert and anomaly visualization:** Highlights detected anomalies and system alerts using visual hierarchies, sound cues, and priority levels to capture operator attention.
- **F-14 Recommendation presentation and prioritization:** Displays generated operational recommendations, sorted by confidence score, priority, and critical urgency.
- **F-15 Operational reporting and historical view rendering:** Compiles standard operational reports and allows the operator to query, filter, and view historical telemetry and past events.

- **F-16 Operator interaction and feedback handling:** Captures explicit operator inputs, acknowledgments of alerts, and qualitative feedback on recommendations, routing them to relevant subsystems.

C-04: Configuration and Model Lifecycle Management Capability The system shall be capable of managing system configuration, AI model lifecycle, versioning, deployment, and retraining workflows under controlled operational governance. This capability is decomposed in the following set of functions:

- **F-17 System configuration management:** Controls, tracks, and validates system-wide operational parameters, thresholds, and environmental settings.
- **F-18 AI model versioning and deployment control:** Manages the repository of trained AI models, overseeing version control, seamless deployment to production, and rollback capabilities.
- **F-19 Dataset versioning and metadata management:** Tracks and catalogues training, validation, and testing datasets, preserving metadata regarding data lineage and provenance.
- **F-20 Adaptation trigger definition and orchestration:** Monitors model performance metrics and system-defined thresholds to identify conditions requiring model retraining or adaptation and orchestrates the corresponding workflow triggers.
- **F-21 System rollback and restore point management:** Generates system state snapshots and coordinates rollbacks to previous stable configurations in the event of deployment failures.

C-05: Persistent Storage and Historical Retrieval Capability The system shall be capable of persistently storing operational data, analytical artifacts, configuration snapshots, and historical records, and providing controlled retrieval interfaces to authorized components. This capability is decomposed in the following set of functions:

- **F-22 Persistent data storage:** Manages the secure, long-term physical or logical storage of all incoming data, normalized records, and system states.
- **F-23 Historical data retrieval:** Exposes high-performance querying and indexing APIs to allow system components to fetch historical records.
- **F-24 Audit log persistence:** Records an unalterable sequential log of all user actions, system state transitions, and security-relevant events for compliance.
- **F-25 Data integrity and consistency enforcement:** Executes periodic checks, replication routines, and database constraints to guarantee data consistency and prevent corruption.

- **F-26 Data retention policy enforcement:** Automatically archives or purges older data records according to predefined regulatory or operational storage lifecycle policies.

Table 3.1. Capability-Function Traceability Summary

Capability ID	Capability	Functions
C-01	Mission Data Ingestion Capability	F-01, F-02, F-03, F-04, F-05
C-02	AI-based Analysis and Decision Support Capability	F-06, F-07, F-08, F-09, F-10
C-03	Mission Status Visualization and Operator Interaction Capability	F-11, F-12, F-13, F-14, F-15, F-16
C-04	Configuration and Model Lifecycle Management Capability	F-17, F-18, F-19, F-20, F-21
C-05	Persistent Storage and Historical Retrieval Capability	F-22, F-23, F-24, F-25, F-26

3.7.2 Software Architecture

The software architecture of the FUNES system was derived through a progressive refinement process starting from the operational needs identified during the system engineering phase and the requirements defined in the System Requirements Document (SRD).

Following the ECSS top-down requirement engineering approach, system-level capabilities were progressively decomposed into system functions and allocated to dedicated software components. This process allowed the transition from an operational view of the system, focused on mission manager needs and decision-support capabilities, to a software-oriented view defining the internal organization of the FUNES platform.

The main architectural driver was the need to support heterogeneous mission data processing, AI-based analysis, operator interaction, configuration control, and historical data management while maintaining modularity, scalability, and verification independence.

Starting from the system requirements, the main functional capabilities were decomposed and allocated to five independent **Configuration Items (CIs)**, each representing a logically separated software subsystem responsible for a specific functional domain (Fig. 3.3).

Configuration Items Breakdown

Below is the detailed functional decomposition and allocation for each Configuration Item defined in the System Requirements Specification (SRS).

CI-1 – Data Processing Manager (DPM) The Data Processing Manager is responsible for data ingestion, format normalization, quality validation, and spatio-

temporal correlation of heterogeneous mission information coming from multiple space and ground segment sources.

- **Inputs:** Spacecraft telemetry, operational status, Mission Planning System (MPS) schedules, Ground Station & antenna status, Payload Data Ground Segment (PDGS) outputs, Flight Dynamics System (FDS) orbit/attitude data, infrastructure/network telemetry, and ticketing incident data.
- **Outputs:** Normalized data records formatted according to the Unified Internal Data Model
- **Key Responsibilities:**
  - Multi-source data acquisition via standardized Interface Control Documents (ICDs).
  - Timestamp assignment, cross-source correlation, and cryptographic integrity check.
  - Isolation of faulty or corrupted data streams to ensure system fault tolerance.
  - Persistence of normalized records to CI-5.
- **Allocated Functions:** `F-01, F-02, F-03, F-04, F-05`.

CI-2 – AI Analytics Manager (AIM) The AI Analytics Manager executes AI-driven analytical processing using an ensemble of neural models, deterministic processing rules, and tool-assisted workflows to evaluate anomalies and generate actionable operational guidance.

- **Inputs:** Normalized and correlated data records from CI-1; historical context data from CI-5.
- **Outputs:** Structured analytical artifacts containing: `anomaly_flag`, `confidence_score`, `affected_system`, `recommended_action`, `model_version`, and `explainability_reference`.
- **Key Responsibilities:**
  - Stream and batch execution of AI model inference (anomaly detection, trend analysis, root cause analysis).
  - Recommendation generation, prioritization, and explainability artifact generation (XAI).
  - AI model performance monitoring, bias detection, and operator feedback integration.
- **Allocated Functions:** `F-06, F-07, F-08, F-09, F-10`.

  Note: All analytical outputs generated by CI-2 are explicitly designated as advisory. Operator evaluation is strictly required prior to operational application.

CI-3 – Decision Support Interface (DSI / User Interface) The Decision Support Interface provides operator-facing dashboards and interaction capabilities for real-time mission monitoring, situation awareness, and decision support.

- **Inputs:** Real-time alerts, rankings, recommendations, and XAI artifacts from CI-2; historical reports from CI-5; configuration status from CI-4.
- **Outputs:** Operator queries, recommendation feedback (commit, reject, modify, rate), operational reports, and configuration requests.
- **Key Responsibilities:**
  - Interactive dashboard rendering for alerts and recommendations.
  - Authentication, Role-Based Access Control (RBAC), and Multi-Factor Authentication (MFA).
  - Displaying system state transitions, safe-state entry, and retraining notifications.
- **Allocated Functions:** `F-12, F-13, F-14, F-15, F-16`.

CI-4 – Configuration & Model Management (CMM) This component governs the global configuration parameters of the FUNES system, AI model lifecycles, prompt/dataset versioning, and deployment operations.

- **Inputs:** Configuration requests from operators (via CI-3); model performance degraded-metrics and retraining triggers from CI-2.
- **Outputs:** System configuration updates, model versioning commands, and rollback procedures.
- **Key Responsibilities:**
  - Centralized configuration parameters and operational mode state machine management.
  - AI model repository control, prompt engineering versioning, and fine-tuning dataset management.
  - Orchestrating model retraining workflows and creating pre-update restore points.
- **Allocated Functions:** `F-17, F-18, F-19, F-20, F-21`.

CI-5 – Storage Manager (SM) The Storage Manager provides centralized, secure, and high-performance persistent storage services for all system components.

- **Inputs:** Raw and normalized records, analytical artifacts, audit logs, and configuration snapshots from CI-1, CI-2, CI-3, and CI-4.
- **Outputs:** Historical datasets, retrieved records, system configuration snapshots, and audit logs.

- **Key Responsibilities:**
  - Providing continuous storage and retrieval APIs for all CIs.
  - Enforcing data retention, consistency, and integrity policies across logical data stores.
- **Allocated Functions:** `F-22, F-23, F-24, F-25, F-26`.

External System Interfaces

To operate effectively within the ground segment ecosystem, FUNES establishes continuous interfaces with several external systems:

- **Spacecraft Telemetry Systems:** Telemetry stream ingestion (HKTM).
- **Mission Planning System (MPS):** Timelines, scheduled operational events, and plan execution states.
- **Ground Segment & Antenna Monitoring:** Ground station health status and RF link metrics.
- **Flight Dynamics System (FDS):** Orbital ephemerides, attitude state vectors, and maneuver plans.
- **Ticketing & Incident Management:** Operational anomaly records and maintenance logs.

The whole system contex is showed in Figure 3.2 and the high level architecture is showed in Figure 3.3.

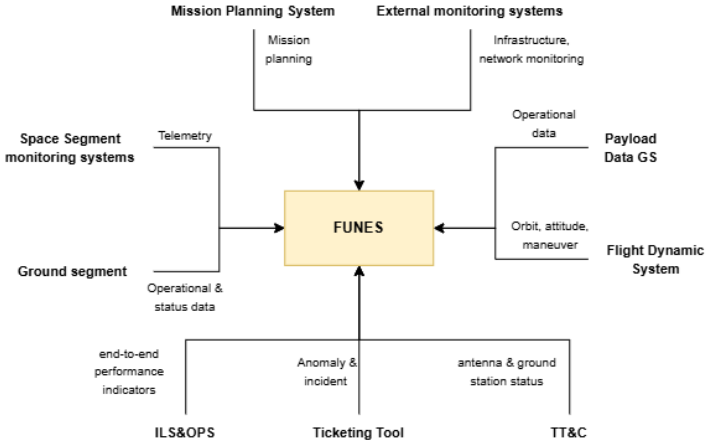


Figure 3.2. The funes system context diagram shows the main external systems that interact with the FUNES system. The data flows continuously from the external systems to FUNES and then to the user through the decision support interface, showed in Fig. 3.3.

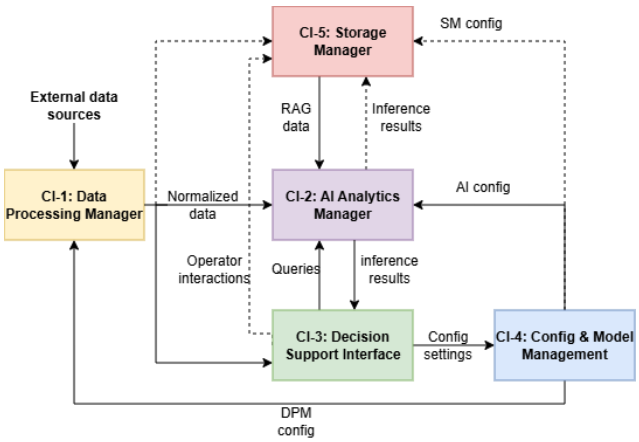


Figure 3.3. The Funes high level software architecture is composed of 5 Configuration Items. The data flows from external subsystems to the Data Processing Manager (DPM) module which will turn them into an internal standard data representation after performing cleaning and validation. The data is then fed to the Storage Manager (SM) for beign stored and to the AI Analytics for AI processing. The user utilizes the system through the Decision Support Interface (DSI). The Config & model management module is responsible for the system configuration and AI control.

3.7.3 Software Requirements Derivation

Following the functional decomposition presented in the previous section, the Software Requirements Specification (SRS) was obtained through the progressive refinement of the system requirements defined in the System Requirements Document (SRD).

In accordance with the ECSS top-down requirement engineering methodology, each system requirement was first allocated to one or more functional capabilities and subsequently decomposed into lower-level system functions. These functions were then further refined into a set of atomic software requirements, each describing a single software behaviour that can be independently implemented, verified, and traced throughout the software lifecycle.

The decomposition process follows the principle of *requirement atomicity*. Whenever a system requirement specifies multiple independent behaviours, it is split into several software requirements, each addressing only one responsibility. This refinement improves requirement clarity, facilitates implementation, and enables independent verification of every software function.

For example, the system requirement

> *FUNES-AI-12.1-003: "The FUNES system shall support semantic retrieval of operational records, historical decisions, and mission outcomes for AI-assisted reasoning processes."*

contains three distinct retrieval capabilities. During the software specification phase, it is decomposed into three independent software requirements:

- **SRS-AI-001**: The software shall support semantic retrieval of operational records for AI-assisted reasoning.

- **SRS-AI-002**: The software shall support semantic retrieval of historical decisions for AI-assisted reasoning.
- **SRS-AI-003**: The software shall support semantic retrieval of mission outcomes for AI-assisted reasoning.

Although these software requirements originate from the same parent system requirement, each one can be independently allocated to software components, implemented, and verified.

Every software requirement maintains bidirectional traceability with its originating system requirement while being assigned to the Configuration Item responsible for its implementation. Furthermore, following the ECSS verification philosophy, each software requirement is associated with a verification method selected among Test (T), Analysis (A), Inspection (I), or Review of Design (RoD). This information establishes a complete traceability chain from the original operational requirement to the corresponding implementation and verification activities.

The adopted refinement process resulted in a Software Requirements Specification comprising **358 atomic software requirements**. Each requirement is uniquely identified, allocated to a specific Configuration Item, linked to its parent system requirement, and associated with the corresponding ECSS verification method.

The resulting traceability chain adopted throughout the development process is illustrated below:

$$\begin{aligned}\text{System Requirement} &\rightarrow \text{System Function} \rightarrow \text{Atomic Software Requirement} \\ &\rightarrow \text{Configuration Item} \rightarrow \text{Verification Method}\end{aligned}$$

This hierarchical organization guarantees complete traceability between operational needs, software implementation, and verification activities, while ensuring compliance with the model-based systems engineering process adopted for the development of the FUNES platform.

3.7.4 Software Design

The software design phase refines the architectural decomposition introduced in the previous sections by defining the implementation technologies, software frameworks, and deployment strategies adopted for the FUNES system.

The technology selection was driven by the software requirements identified in the Software Requirements Specification (SRS), with particular attention to modularity, scalability, maintainability, interoperability with existing ground segment infrastructures, and suitability for AI-based mission support applications.

The system is primarily implemented using Python as the common development language across all Configuration Items. Python was selected due to its mature ecosystem for artificial intelligence, machine learning, data processing, and scientific computing, as well as its widespread adoption in data-intensive software applications. The use of a common development environment simplifies integration between traditional software modules and AI-based components.

Where strict performance constraints may arise, particularly in high-throughput data processing tasks, lower-level implementations through C++ or Rust extensions can be introduced at module level without affecting the overall software architecture.

The implementation of the individual Configuration Items is described below.

CI-1 - Data Processing Manager

The Data Processing Manager is responsible for the acquisition, ingestion, normalization, validation, and correlation of heterogeneous mission data sources.

To support continuous data ingestion and asynchronous processing, CI-1 relies on Apache Kafka as the underlying event-streaming platform. Kafka provides fault-tolerant message handling, ordered data delivery, and scalable processing capabilities suitable for heterogeneous mission data streams.

The incoming data are validated and transformed into the Unified Internal Data Model before being distributed to the other system components. Data validation and schema enforcement are implemented using Pydantic, enabling structured validation rules and consistency checks before data propagation.

The design of CI-1 supports integration with multiple external mission systems, including telemetry providers, mission planning systems, flight dynamics systems, and ground infrastructure monitoring systems.

CI-2 - AI Analytics Manager

The AI Analytics Manager represents the core analytical component of FUNES and is responsible for anomaly detection, trend analysis, operational recommendation generation, and explainability artifact production.

The component is based on a self-hosted Large Language Model (LLM) inference architecture, selected to support deployment within controlled mission environments where data sovereignty, operational security, and predictable performance are required.

LLM inference is performed through vLLM, which provides optimized high-throughput serving of open-weight models compatible with the HuggingFace model ecosystem. Candidate model families considered for deployment include LLaMA, Qwen, and Mistral, selected due to their open-weight availability and suitability for on-premise execution.

The orchestration of complex analytical workflows, including retrieval-augmented generation (RAG), tool-assisted reasoning, and multi-step agent workflows, is implemented using frameworks such as LangGraph and LlamaIndex.

In addition to generative AI capabilities, CI-2 integrates classical machine learning components for anomaly detection and trend analysis. These models are implemented using established frameworks such as Scikit-learn and PyTorch.

The AI outputs generated by CI-2, including anomaly assessments, confidence scores, recommendations, and explanations, remain advisory and are provided to support, rather than replace, mission operator decision-making.

CI-3 - Decision Support Interface

The Decision Support Interface provides the interaction layer between the FUNES system and mission operators.

The interface is implemented as a web-based application using React for the frontend and FastAPI as the backend service layer. This architecture enables the development of modular user interface components while maintaining efficient communication with the internal FUNES services.

Real-time updates, including alerts, dashboard information, and analytical results, are delivered through WebSocket communication channels, enabling continuous monitoring of mission conditions.

Authentication and authorization mechanisms are implemented through Keycloak, providing role-based access control and supporting multiple authentication methods, including username/password authentication, multi-factor authentication, and certificate-based authentication.

The interface provides mission dashboards, conversational interaction with the AI system, visualization of anomalies and recommendations, and mechanisms for collecting operator feedback.

CI-4 - Configuration and Model Management

The Configuration and Model Management component provides centralized control of system parameters, AI model lifecycle management, and configuration versioning.

The component uses MLflow to manage AI model versions, experiment tracking, deployment workflows, and model artifact registration. Dataset versioning and metadata management are also supported to preserve information regarding data provenance, lifecycle, and usage.

CI-4 manages model deployment, rollback procedures, retraining workflows, and configuration changes while maintaining reproducibility and traceability of system states.

These capabilities support the requirements related to AI lifecycle management and operational governance defined in the SRS.

CI-5 - Storage Manager

The Storage Manager provides persistent storage capabilities and standardized data access services for the other FUNES components.

The storage architecture follows a multi-store approach, where different types of information are stored according to their operational characteristics.

Time-series operational data, including normalized telemetry and ingestion records, are stored using TimescaleDB, a PostgreSQL extension optimized for time-dependent data workloads.

AI-related artifacts, including model files, datasets, and explainability outputs, are stored within an S3-compatible object storage system. This approach enables flexible deployment both in on-premise environments through solutions such as MinIO and in cloud-based infrastructures through compatible object storage services.

Audit logs and immutable operational records are maintained through PostgreSQL storage with append-only policies to preserve historical traceability.

Semantic embeddings generated for retrieval-augmented generation workflows are stored using vector indexing capabilities provided by pgvector, enabling similarity-based retrieval of historical mission data, operational procedures, anomaly reports, and previously generated analytical artifacts.

All access to persistent data is mediated through standardized storage interfaces, with encrypted communication channels and access control mechanisms applied at database and object storage levels.

Deployment and Integration Environment

The FUNES system is designed for deployment within hybrid cloud environments using containerized software components.

Docker is adopted for software packaging and environment isolation, while Kubernetes is used for service orchestration, scalability management, health monitoring, and deployment automation.

Communication between components follows a hybrid approach. Synchronous interactions are implemented through gRPC-based interfaces, while asynchronous information exchange relies on Kafka messaging mechanisms.

This deployment strategy enables flexible allocation of computational resources, particularly for AI-intensive workloads, while maintaining the modularity and operational robustness required for mission support applications.

3.7.5 System Data Flows and Operational Use Cases

This section describes the dynamic data flows of the FUNES system through representative operational use cases. Each use case illustrates the path data follows across the Configuration Items (CI-1 to CI-5), the transformations applied, and the interaction points where human-in-the-loop validation is required. Four primary use cases are defined to cover nominal automatic operations, interactive investigations, model maintenance, and degraded operational conditions.

UC-1: Near Real-Time Anomaly Detection and Recommendation (Nominal, Fully Automatic) This represents the primary nominal end-to-end data processing pipeline. Data arrives continuously from external space and ground sources, gets processed, analyzed, and presented to the operator. No operator intervention is required during the ingestion and analysis phases; human interaction is strictly required only at the final recommendation review and execution stage, adhering to the system's advisory-only design philosophy. See Fig.3.4.

1. **Data Acquisition & Normalization (CI-1):** External data streams (telemetry, MPS schedules, network logs) are ingested by CI-1 via dedicated ICD adapters (F-01). CI-1 normalizes payloads into the Unified Internal Data Model (F-02), validates syntax/semantics (F-03), appends system timestamps, and correlates records (F-04).

2. **Persistence (CI-5):** Normalized streams are continuously written to CI-5 for time-series persistence (F-22).

3. **AI Inference & Detection (CI-2):** Simultaneously, CI-1 emits normalized event streams to CI-2. CI-2 executes real-time anomaly detection inference (F-06). Upon detecting an anomaly, CI-2 generates an operational recommendation (F-08), calculates confidence scores (F-09), and produces explainability artifacts (F-10).

4. **Visualization & Notification (CI-3):** CI-2 transmits structured anomaly payloads to CI-3, which triggers visual/auditory alerts on the operator dashboard (F-13) and presents the prioritized recommendation (F-14).

5. **Operator Feedback & Decision (CI-3):** The operator evaluates the recommendation and explainability details. The operator explicitly commits, modifies, or rejects the suggestion (F-16). The feedback action is stored in CI-5 for compliance and future model optimization (F-24).

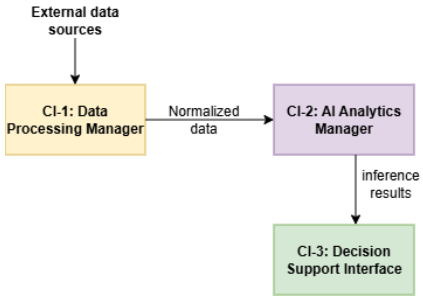


Figure 3.4. End-to-end data flow for near real-time anomaly detection and recommendation, from continuous data acquisition and normalization to AI-based analysis, operator notification, and final decision.

UC-2: Operator Conversational Investigation via AI-Assisted Chatbot
This use case covers interactive decision support. The operator uses a natural language interface embedded in the dashboard to perform ad-hoc diagnostic queries, explore root causes, or analyze complex operational scenarios. See Fig. 3.5

1. **Query Submission (CI-3):** The operator formulates a free-text prompt within the conversational interface of CI-3 (F-11). CI-3 attaches session tokens, operator identity (RBAC), and forwards the request to CI-2.

2. **Agentic Workflow & Tool Execution (CI-2):** CI-2 parses the prompt using an LLM-based agent. To construct an accurate answer, CI-2 dynamically queries CI-5 via historical retrieval APIs (F-23) to fetch historical telemetry, past anomalies, or configuration snapshots.

3. **Response Generation & Grounding (CI-2):** CI-2 executes Retrieval-Augmented Generation (RAG), synthesize retrieved telemetry data into a structured response, and appends confidence scores (F-09) and source citations (F-10).

4. **Rendering & Multi-turn Dialogue (CI-3):** CI-3 renders the response, supporting interactive tables or chart widgets (F-11, F-15). The conversation may continue across multiple dialogue turns.

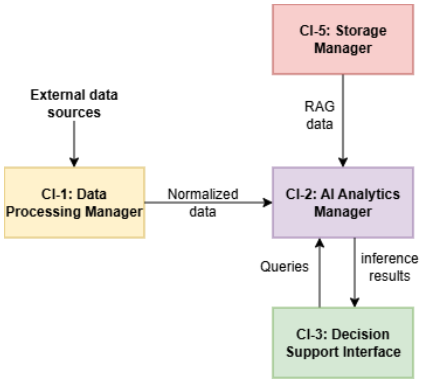


Figure 3.5. Conversational data flow for AI-assisted operator investigation, showing query submission, historical data retrieval, RAG-based analysis, and interactive response generation.

UC-3: Operator-Initiated Dashboard Investigation This use case describes a scenario where an operator observes a suspicious trend or receives an external notice (e.g., ground station maintenance) and manually triggers a deep-dive investigation across monitoring dashboards. See Fig. 3.6

1. **Manual Trigger & Filtering (CI-3):** The operator accesses the historical and real-time visualization views in CI-3 (F-12, F-15) and applies custom time-range, satellite, or ground station filters.
2. **Data Fetching (CI-5):** CI-3 issues high-performance query requests to CI-5, fetching raw/normalized historical parameters (F-23).
3. **On-Demand Analysis Trigger (CI-2):** The operator selects a telemetry window and requests an on-demand pattern/trend analysis (F-07). CI-2 processes the designated subset and returns comparative metrics or degradation curves to CI-3.
4. **Audit Logging (CI-5):** All query filters, manual inspection views, and operator actions are recorded in the audit log (F-24).

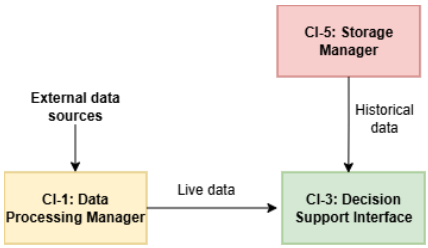


Figure 3.6. Operator-initiated investigation data flow, showing manual filtering, historical data retrieval, on-demand telemetry analysis, and audit logging of operator actions.

3.7.6 LLM Model Selection

The selection of the Large Language Model (LLM) represents a key design decision for the AI Analytics Manager (CI-2), as the model capabilities directly influence the reliability of the multi-agent reasoning workflows implemented within the FUNES framework.

The selection process was driven by the software requirements allocated to CI-2, with particular attention to instruction following capability, reasoning performance, structured output reliability, context management, and deployment feasibility in controlled operational environments.

Since the FUNES system is designed for deployment within mission-critical and potentially restricted environments, proprietary cloud-based models were excluded from the evaluation. The analysis therefore focused on open-weight Large Language Models that can be deployed locally while maintaining control over data processing, model execution, and system integration.

The candidate model families considered during the evaluation were:

- LLaMA;
- Qwen;
- DeepSeek.

The evaluation was performed using publicly available benchmark results taken from the Open LLM Leaderboard of huggin face, adopted as proxy indicators of model suitability due to the absence of standardized benchmarks specifically targeting agent-based decision support systems. The following benchmark categories were considered:

- **General knowledge and reasoning capability:** evaluated through MMLU-Pro;
- **Multi-step reasoning and long-context understanding:** evaluated through MUSR;
- **Instruction following and structured output reliability:** evaluated through IFEVAL.

These evaluation criteria were selected according to the main operational requirements of the AI Analytics Manager:

- reliable execution of multi-step analytical workflows;
- consistent adherence to tool-use constraints and structured output formats;
- capability to integrate heterogeneous information retrieved from external knowledge sources;
- robustness when operating inside stateful multi-agent orchestration frameworks.

Because the FUNES architecture relies on LLM-based agents executing predefined workflows, instruction-following capability represents a critical selection parameter. For this reason, a minimum threshold of IFEVAL ≥ 80 was introduced to exclude models with insufficient compliance with structured prompts and orchestration constraints.

Following this filtering phase, the evaluated candidates were:

- Qwen2.5-14B-Instruct-1M;
- Qwen2.5-32B-Instruct;
- Llama-3.3-70B-Instruct;
- Qwen2.5-72B-Instruct.

The final model selection was performed through a multi-objective trade-off analysis considering reasoning capability, instruction adherence, supported context length, and computational requirements.

Although larger models generally provide improved reasoning performance, their deployment introduces increased hardware requirements, latency, and operational complexity. Therefore, the selected model needed to provide sufficient reasoning capability while remaining compatible with an on-premise execution environment.

Based on this analysis, the Qwen2.5 family was selected, with the 14B and 32B variants identified as the most suitable configurations for the FUNES architecture.

The selected models provide an effective compromise between reasoning performance, instruction-following reliability, and computational efficiency. This configuration enables the execution of multi-agent decision-support workflows while maintaining control over deployment constraints, latency, and system complexity.

Chapter 4

Proof of Concept Development

4.1 PoC Overview and Architecture

The development of the Proof of Concept (PoC) followed the software architecture introduced in Section 3.7.2. The implementation was carried out following a bottom-up approach, progressively developing the core services required by the final decision-support system.

The proposed PoC is composed of three main software layers:

- the *Storage Manager*, responsible for providing unified access to heterogeneous information sources and managing the knowledge required by the system;
- the *AI Manager*, responsible for integrating Large Language Models, specialized agents, and the reasoning capabilities required to support decision-making activities;
- the *Decision Support Interface*, providing the interaction layer between users and the underlying AI-based services.

The communication and coordination between these layers are managed through the multi-agent orchestration framework described in the following sections.

Throughout the implementation, particular attention was devoted to the creation of a modular and extensible software architecture. Each component was developed following an abstraction-oriented design, where functionalities are exposed through well-defined interfaces and concrete implementations are decoupled from the higher-level logic.

This approach enables individual components to be replaced or extended with minimal modifications to the overall system. For example, different storage technologies, language models, or agent implementations can be integrated by providing alternative implementations of the corresponding interfaces.

The first component developed was the *Storage Manager*, as reliable access to contextual information represents a fundamental requirement for the subsequent reasoning and decision-support layers. Its implementation provides the data foundation of the system, including document retrieval capabilities through a custom Retrieval-Augmented Generation (RAG) pipeline, structured operational data access, and persistent conversation management.

4.2 Storage Manager

4.2.1 Architecture and Design Principles

The Storage Manager represents the data management layer of the proposed multi-agent framework. Its main responsibility is to provide a unified access interface to the heterogeneous information required by the AI components during the decision-support process.

Instead of allowing the reasoning layer to directly interact with individual data sources, the Storage Manager abstracts the underlying storage mechanisms and data formats. This design choice separates data management concerns from reasoning capabilities, reducing the dependency between the AI components and the specific implementation of the information sources.

The Storage Manager integrates multiple functionalities:

- a Retrieval-Augmented Generation (RAG) subsystem for managing technical documentation and enabling semantic knowledge retrieval;
- structured data interfaces for accessing operational information, including planning data stored in CSV, XML, and JSON formats;
- a persistent chat storage service for maintaining conversation histories between users and agents;
- additional interfaces for retrieving system status information and maintenance-related data.

Among these functionalities, the RAG subsystem represents the main knowledge management component of the Storage Manager. While structured data interfaces provide deterministic access to operational information, the RAG pipeline enables semantic exploration of unstructured technical documentation, supporting the reasoning capabilities of the multi-agent system.

For this reason, the following sections provide a detailed description of the RAG architecture, including document processing, embedding generation, vector storage, hybrid retrieval, and validation activities.

Following the same architectural principles adopted for the overall PoC, the Storage Manager has been implemented through an abstraction-oriented design. Core functionalities are defined through abstract interfaces and specialized through concrete implementations, allowing individual components to be replaced independently without affecting the remaining layers of the system.

This modular structure improves maintainability, supports experimentation with alternative technologies, and provides a scalable foundation for future extensions of the decision-support framework.

4.2.2 Rag System

Figure 4.2 illustrates the overall architecture of the implemented RAG in the Storage Manager.

The document ingestion process starts from the `Chunker`, responsible for loading heterogeneous documents and dividing them into semantically coherent chunks. The generated chunks are then converted into dense vector representations through the selected embedding model.

The resulting embeddings, together with their metadata, are stored inside the `Vector Store`, while a parallel lexical index based on `BM25` is maintained to support keyword-based retrieval.

Finally, the `Memory` component orchestrates the interaction among all the previous modules by managing document ingestion, preventing duplicate insertions, performing hybrid retrieval, and applying an additional reranking stage before returning the final context to the upper layers of the multi-agent framework.

Document Chunking

The first stage of the Storage Manager is responsible for transforming raw documents into semantically meaningful text fragments that can later be embedded and indexed. Since the quality of the retrieved information strongly depends on how documents are partitioned, particular attention was devoted to the design of the chunking process.

To ensure modularity, the chunking functionality is defined through the abstract class `BaseChunker`, which specifies the two fundamental operations required by the framework: loading a document and generating its corresponding chunks. This abstraction decouples the retrieval pipeline from any specific chunking strategy, allowing alternative implementations to be introduced without modifying the remaining components of the Storage Manager.

The concrete implementation developed for the Proof of Concept, named `LocalSemanticChunker`, performs semantic chunking locally by exploiting the *all-MiniLM-L6-v2* Sentence Transformer model. Instead of dividing documents into fixed-size windows, the adopted strategy identifies semantically coherent boundaries between consecutive text segments. This approach produces chunks that better preserve the original context, improving the quality of the subsequent embedding generation and retrieval phases. The selected Sentence Transformer model produces dense embeddings of 384 dimensions, providing a favorable trade-off between computational efficiency and semantic representation quality.

The semantic chunking process is implemented using the LangChain SemanticChunker with the `standard_deviation` breakpoint strategy. A breakpoint threshold of 1.5 is adopted, producing larger semantic segments by splitting only where the semantic distance between consecutive sentences significantly exceeds the average. Additionally, a buffer size of five sentences is used to avoid generating excessively small chunks. Moreover, chunks containing fewer than 50 characters or less than eight words are discarded, as they may provide insufficient contextual information and may negatively affect retrieval quality.

The implemented pipeline consists of several preprocessing steps before the actual semantic segmentation is performed. First, the input document is parsed according to its format (PDF, DOCX, or TXT). For PDF documents, metadata such as title, author, and last modification date are extracted and associated with every

generated chunk.

To minimize noise and prevent non-informative repetitive text (such as running headers, footers, page numbers, and recurrent table structures) from polluting the vector space, a dynamic multi-stage text cleaning algorithm was designed and integrated prior to semantic chunking.

Unlike traditional static rule-based cleaning methods (e.g., rigid regular expressions), the proposed approach dynamically identifies document-specific boilerplate text through fuzzy string matching across page boundaries. The cleaning pipeline consists of three core steps:

1. **Dynamic Pattern Discovery via Sampling:** To optimize computational efficiency, especially for large technical documents, the algorithm performs an initial sampling over the first N pages ($N = 5$ by default). Each sampled page is decomposed into discrete line blocks.

2. **Cross-Page Fuzzy Similarity Analysis:** For every line block b_i in page i, the algorithm compares its content against all line blocks b_j in other sampled pages $j \neq i$. To account for minor textual variations (e.g., page numbers changing within a running header), textual overlap is computed using the matching algorithm via Python's `SequenceMatcher`.

   A line block is identified as a repetitive candidate pattern P_{rep} if its similarity ratio equals or exceeds a tuned threshold $\theta_{sim} = 0.95$:

   $$\text{similar}(a, b) = \frac{2 \cdot |M|}{|a| + |b|} \geq \theta_{sim} \tag{4.1}$$

   where $|M|$ is the number of matching characters in the longest common substrings between a and b, and $|a|, |b|$ are the lengths of the respective string blocks. Short lines (fewer than 10 characters) are ignored during pattern discovery to avoid accidentally stripping valid titles or bullet points.

3. **Document Filtering and Sanitization:** The discovered set of repetitive patterns P_{rep} is then used to filter the entire document page-by-page. Lines exceeding the similarity threshold when compared against any pattern in P_{rep} are pruned. Finally, string normalizations, including whitespace collapsing and low-character noise removal (discarding chunks under 50 characters or 8 words), are applied to produce clean, semantically dense text blocks.

This adaptive cleaning process ensures that recurring administrative header/footer metadata is stripped out, preventing artificial semantic similarity spikes during embedding generation and improving the signal-to-noise ratio in the retrieved context.

To preserve document structures during ingestion, PDF documents are parsed into structured Markdown format using `pymupdf4llm`. Unlike plain-text extractions that flatten layout elements, converting pages into Markdown preserves section headings, bullet lists, and tabular structures. Preserving tables and hierarchical layout in Markdown enables downstream embedding models to capture structural relationships, while remaining natively compatible with Large Language Model context processing.

After the cleaning phase, each page is semantically partitioned into coherent text fragments using the Semantic Chunker algorithm. Finally, each generated chunk is enriched with a comprehensive set of metadata, including a unique chunk identifier, a content hash made with SHA-1 computed over its normalized textual content used for duplicate detection, the source document, page number, chunk position within the page, chunk size, and document metadata. This information is preserved throughout the entire retrieval pipeline and enables efficient indexing, traceability, and document management.

The chunking process is showed in Fig. 4.1.

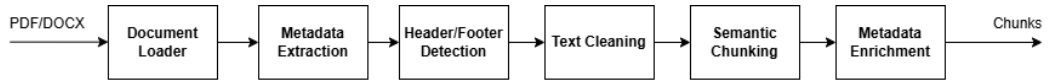


Figure 4.1. Document chunking workflow implemented in the Storage Manager. The document is preprocessed for metadata extraction like: "chunk_id", "content_hash", "source", "title", "author", "last_modified", "page_number", "chunk_index_in_page", "chunk_size" and "format". Then the repetitive parrterns like page headers and footers are removed. The text is then cleaned and transformed into markdown. Finally the chunking process will turn the text into vectors that are going to be associated with the metadata and the original text.

Embedding Generation

After the document chunking phase, each generated text fragment is transformed into a numerical representation suitable for semantic retrieval. This process is performed by the embedding generation module, whose objective is to map textual information into a continuous vector space where semantically similar contents are represented by nearby vectors.

Similarly to the other components of the Storage Manager, the embedding generation layer was designed following an abstraction-based approach. The abstract class `EmbeddingModel` defines the interface required by the system, exposing a single operation responsible for converting a text input into its corresponding embedding vector.

This abstraction decouples the retrieval pipeline from the specific embedding model used during execution. Therefore, different embedding techniques can be integrated by implementing the same interface, without requiring modifications to the memory management or storage layers.

For the Proof of Concept, the selected implementation is based on the Sentence Transformer architecture through the `all-MiniLM-L6-v2` model. The model generates dense vector representations that capture semantic relationships between textual fragments, enabling similarity-based retrieval even when the query and the stored document use different wording.

The generated embeddings are computed during the document ingestion phase and stored together with the corresponding text chunks and metadata inside the vector database. During the retrieval phase, the same embedding model is used to encode the user query, allowing the system to perform semantic similarity comparison between the query representation and the indexed knowledge base.

The separation between embedding generation and storage management allows the system to remain independent from a specific embedding technology, facilitating future experimentation with alternative models or domain-specific embedding approaches.

Vector Storage

The vector storage layer is responsible for persisting document embeddings and supporting efficient similarity search operations. To preserve the modular architecture adopted throughout the Storage Manager, this functionality is defined through the abstract class `VectorStore`, which specifies the common interface required by the retrieval pipeline.

The interface exposes the basic operations required by any vector database implementation, including document insertion, similarity search, retrieval of the complete collection, and access to the content hashes used for duplicate detection. As a consequence, replacing the underlying vector database only requires implementing the same interface, while the higher-level components remain unchanged.

The Proof of Concept adopts ChromaDB as the concrete implementation through the `ChromaStore` class. This wrapper encapsulates all interactions with the database, including collection creation, persistent storage, document insertion, similarity search, and collection management. ChromaDB was selected because it provides persistent storage, native similarity search, efficient metadata management, and seamless integration with Python-based LLM frameworks, making it particularly suitable for rapid prototyping.

During the ingestion phase, each chunk is stored together with its embedding vector and the corresponding metadata. The metadata are preserved inside the vector database and include information such as the document source, page number, chunk identifier, and content hash. This additional information is subsequently exploited during retrieval and traceability operations.

Before new chunks are inserted, the vector store exposes the set of already stored content hashes, allowing the Memory component to efficiently identify duplicate chunks and avoid redundant indexing.

During retrieval, the vector store performs similarity search by comparing the embedding of the user query with the embeddings stored in the collection, returning the most semantically relevant document fragments together with their associated metadata.

Knowledge Memory Management

The Memory component represents the central layer responsible for managing the lifecycle of the knowledge stored inside the Retrieval-Augmented Generation system. Its main purpose is to provide a unified interface for storing, updating, and retrieving information, while hiding the implementation details of the underlying storage and retrieval mechanisms.

Following the modular design adopted throughout the Storage Manager, the memory layer is defined through the abstract class `Memory`. This interface specifies

the fundamental operations required by any memory implementation: adding new knowledge items, searching for relevant information, and retrieving the complete stored knowledge base.

This abstraction allows the retrieval pipeline to remain independent from the specific memory implementation. Different types of memories can therefore be introduced in the future, such as domain-specific knowledge bases or agent-specific memories, without requiring changes to the components responsible for document processing or storage.

The concrete implementation developed for the Proof of Concept is represented by the `KBMemory` class, which manages the knowledge base used by the multi-agent system. The class acts as an intermediate layer between the document processing components and the storage mechanisms, coordinating chunk generation, embedding computation, indexing, retrieval, and result refinement.

During the ingestion phase, the memory receives a document path and uses the configured chunker to transform the document into a collection of semantically meaningful chunks. Before inserting new information, the system performs a duplicate detection phase based on the content hash associated with each chunk. This mechanism prevents the same information from being stored multiple times, reducing redundancy and improving the efficiency of the retrieval process.

Only previously unseen chunks are processed further. Their embeddings are generated through the configured embedding model and stored together with the associated metadata inside the vector database. At the same time, the textual representation of each chunk is inserted into a lexical index based on BM25, enabling the system to combine semantic and keyword-based retrieval strategies.

The retrieval process is designed as a multi-stage pipeline rather than a simple similarity search. The first stage performs semantic retrieval using vector similarity, exploiting the contextual information captured by embeddings. In parallel, a lexical retrieval strategy based on BM25 identifies documents containing relevant keywords from the user query.

For a requested retrieval size of k, both the vector store and the BM25 index independently retrieve $2k$ candidate chunks. The enlarged candidate set increases the probability of recovering relevant documents that may appear only in one of the two rankings before the fusion stage.

The results produced by the two retrieval mechanisms are then combined using the Reciprocal Rank Fusion (RRF) algorithm [13]. This approach allows the system to exploit the complementary strengths of both methods: vector retrieval improves semantic understanding, while lexical retrieval provides accurate matching for specific terms, identifiers, or technical expressions.

Given a document d, its RRF score is computed as

$$RRF(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \tag{4.2}$$

where R denotes the set of ranked retrieval methods, $\text{rank}_r(d)$ is the rank assigned to document d by retrieval method r, and k is a positive constant controlling the influence of lower-ranked documents. In the proposed implementation, the commonly adopted value $k = 60$ is used [13].

Unlike score-based fusion methods, RRF does not require the normalization of similarity scores, which may originate from heterogeneous retrieval models and therefore be expressed on different scales. Instead, the fusion process relies solely on the relative ranking of retrieved documents, making the resulting ranking more robust and less sensitive to differences between lexical and semantic retrieval functions.

The fused ranking is finally refined through a Cross-Encoder reranker: `BAAI/bge-reranker-base` While the initial retrieval stages evaluate query-document similarity independently, the Cross-Encoder jointly processes the query and each candidate chunk, allowing it to model deeper semantic interactions between the two texts.

This additional ranking stage is computationally more expensive than embedding-based retrieval and is therefore applied only to the limited set of candidate documents produced by the hybrid retrieval phase. The final ranked list returned by the reranker constitutes the contextual information provided to the upper layers of the multi-agent framework.

The resulting memory architecture therefore implements a complete retrieval pipeline composed of:

- semantic retrieval through vector embeddings;
- lexical retrieval through BM25 indexing;
- rank fusion through Reciprocal Rank Fusion;
- relevance refinement through Cross-Encoder reranking.

This multi-stage approach improves the robustness of the knowledge retrieval process, reducing the limitations of relying on a single retrieval strategy and providing higher-quality contextual information to the decision-support agents. The RAG system is summarized in figure Fig. 4.2

RAG Validation

To evaluate the effectiveness of the implemented Retrieval-Augmented Generation pipeline, an experimental validation was conducted on a heterogeneous knowledge base composed of twenty technical documents for a total of approximately 5000 chunks. The dataset included both publicly available ECSS (European Cooperation for Space Standardization) standards and few internal technical documentation provided by Telespazio. The selected corpus reflects the type of documentation that the proposed decision support system is expected to process in real operational scenarios.

A benchmark composed of one hundred queries was manually created for the evaluation. Each query was manually formulated to capture the main semantic content of a randomly selected page while avoiding verbatim copying of the original text. For every query, the expected relevant page was manually annotated and considered as the ground truth.

The objective of the evaluation was to measure the capability of the retrieval pipeline to return the correct document among the retrieved candidates. Three retrieval configurations were evaluated:

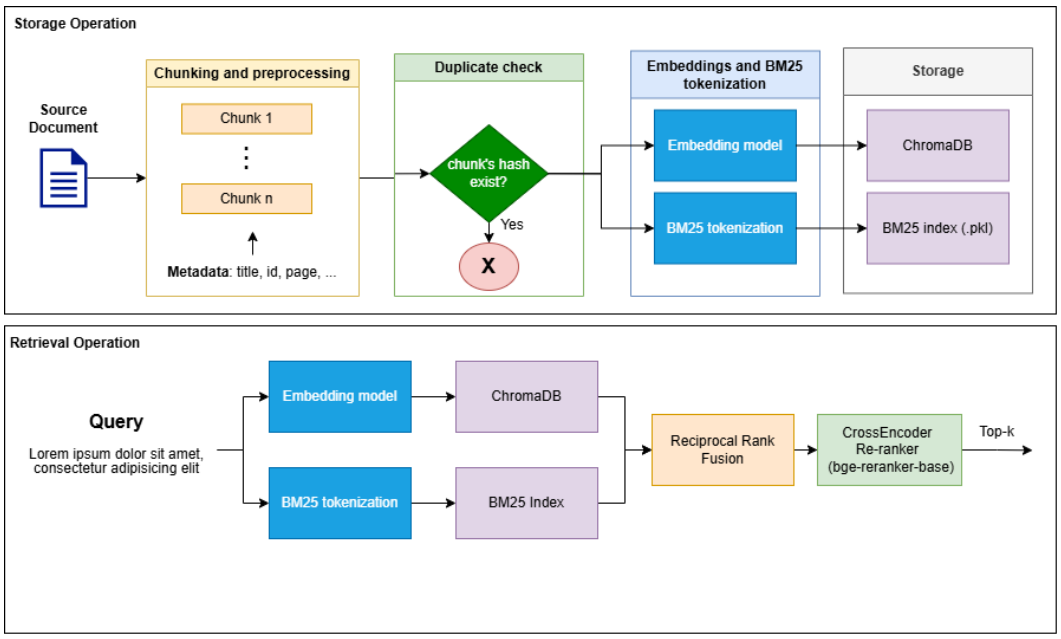


Figure 4.2. Architecture of the Hybrid RAG System. The system integrates a *Storage Pipeline* (top) that cleans, dynamically chunks, and deduplicates raw documents using SHA-1 hashing, indexing unique content simultaneously into a dense vector database (`ChromaDB`) and a sparse lexical in-memory index (`BM25Okapi`, serialized via `pickle`); and a *Retrieval Pipeline* (bottom) that executes parallel dense-semantic and sparse-keyword searches for a given query, merges the candidate results using Reciprocal Rank Fusion (RRF) based on a unified `chunk_id`, and re-ranks them using a Cross-Encoder model (`BAAI/bge-reranker-base`) to output the top-k most contextually relevant passages for the downstream LLM.

- semantic retrieval using only the vector database;
- hybrid retrieval using Reciprocal Rank Fusion (RRF);
- hybrid retrieval followed by Cross-Encoder reranking.

The performance was assessed using two standard Information Retrieval metrics:

- **Recall@5**, measuring the percentage of queries for which the correct document appeared within the first five retrieved results;
- **Mean Reciprocal Rank (MRR)**, evaluating how early the correct document appeared in the ranking.

Table 4.1. Retrieval performance obtained during the Storage Manager validation.

Configuration	Recall@5 (%)	MRR (%)
Vector Search	79	68.48
Hybrid Search (RRF)	86	71.78
Hybrid Search + Cross Encoder	**86**	**80.50**

Table 4.2. Position of the correct document within the retrieved Top-5 results.

Configuration	Rank1	Rank2	Rank3	Rank4	Rank5
Vector Search	61	11	4	1	2
Hybrid (RRF)	62	14	4	5	1
Hybrid + Reranker	**76**	7	3	0	0

The obtained results highlight the contribution of each stage of the retrieval pipeline.

The introduction of hybrid retrieval through Reciprocal Rank Fusion increased the Recall@5 from 79% to 86%, demonstrating that combining semantic and lexical retrieval allows the system to recover relevant documents that may be overlooked by either retrieval strategy individually.

Although the increase in Mean Reciprocal Rank is more moderate (from 68.48% to 71.78%), the improvement indicates that the correct document is generally positioned closer to the top of the retrieved ranking.

The largest improvement is observed after introducing the Cross-Encoder reranker. While the Recall@5 remains unchanged, the MRR increases significantly from 71.78% to 80.50%. This behaviour is expected since the reranker does not retrieve additional documents but rather reorders the candidates already returned by the hybrid retrieval stage.

The position distribution further confirms this behaviour. The number of queries for which the correct document is ranked first increases from 62 to 76, while no relevant document appears in the fourth or fifth position after reranking. This demonstrates that the Cross-Encoder is particularly effective at improving the ranking quality of already retrieved candidates.

Limitations
Although the proposed pipeline achieves encouraging retrieval performance, the evaluation has been conducted on a relatively limited corpus composed of twenty technical documents (around 5000 chunks). Future work should assess scalability on substantially larger document collections and investigate the impact of alternative embedding models and reranking strategies.

4.2.3 Additional Data Interfaces

Although the Retrieval-Augmented Generation pipeline represents the main knowledge management component of the Storage Manager, the module is not limited to document-based information retrieval. The decision-support system requires access to heterogeneous data sources, including operational data, planning information, and system status information.

Since for the PoC capabilities is not required to ingest data from the real data sources of Telespazio, we decided to use standard local files for the data sources to be ingested in the system. The Storage Manager was designed as a unified data access layer, exposing high-level interfaces for retrieving different types of information while hiding the specific format and parsing logic of each source.

The implemented Proof of Concept supports the ingestion and retrieval of structured and semi-structured data, including:

- planning informations stored in CSV files;
- network data config of the ground segment
- open tickets of the ground segment

Each data source is handled through dedicated processing functions responsible for parsing the original format and transforming the information into a representation suitable for consumption by the upper layers of the architecture.

For example, planning data can be filtered according to operational parameters such as time interval, satellite identifier, or ground station, allowing agents to retrieve only the information relevant to the current decision-support task.

Unlike the knowledge base, these data sources are not indexed through semantic retrieval mechanisms, since they represent structured operational information where explicit filtering and deterministic retrieval provide a more appropriate access strategy.

The Storage Manager therefore acts as an abstraction layer over both unstructured knowledge sources and structured operational data, providing a single entry point for the AI components. This design prevents agents from directly interacting with heterogeneous data formats, improving maintainability and allowing additional data sources to be integrated without modifying the reasoning layer.

4.2.4 Chat Storage

Besides managing external knowledge and operational data, the Storage Manager provides a persistent storage layer for the conversational information generated during the interaction between users and the multi-agent system.

The Chat Storage component has been designed to maintain the history of interactions, enabling users to retrieve previous conversations and allowing the AI components to access contextual information related to ongoing tasks. Unlike the Knowledge Base memory, which contains stable domain information retrieved through the RAG pipeline, chat storage manages dynamic information generated during system execution.

The implemented solution relies on `MongoDB`, a document-oriented NoSQL database selected due to its flexibility in managing conversational data, where messages may contain heterogeneous metadata depending on the involved agent, language model, or execution context.

The architecture separates conversation-level information from individual message data through two different collections:

- `chats`, storing high-level information related to each conversation;
- `messages`, storing the individual messages exchanged during the interaction.

Each conversation is uniquely identified by a `chat_id` and contains information such as the associated user, the agent involved in the interaction, creation and update timestamps, and aggregate statistics including the number of exchanged messages and the total number of processed tokens.

Individual messages are stored as independent documents containing the conversation identifier, user identifier, message role, textual content, and additional execution metadata. The stored information includes the language model used to generate the response, the number of input and output tokens, the response latency, and optional metadata generated during the execution.

The inclusion of execution-related information enables the system to monitor the interaction history and provides the basis for future extensions such as performance analysis, cost estimation, and agent behaviour evaluation.

To improve retrieval efficiency, the database defines dedicated indexes on the main query fields. In particular, indexes are created on the conversation identifier and on the combination of conversation identifier and creation timestamp, allowing chronological reconstruction of conversations with limited query overhead. Additionally, user-based indexing enables efficient retrieval of all conversations associated with a specific user.

The main operations exposed by the Chat Storage component are:

- creation of new conversations associated with a user and a selected agent;
- persistent storage of user and assistant messages;
- chronological retrieval of messages belonging to a conversation;
- retrieval of the conversation history associated with a user.

The separation between conversational storage and knowledge retrieval follows the principle of information specialization. While the RAG memory is optimized for semantic search over external documents, the Chat Storage component is optimized for temporal reconstruction of previous interactions.

This distinction is particularly important in a multi-agent decision-support system, where conversational context, user requests, and generated responses represent operational information rather than static domain knowledge. The proposed design allows both components to evolve independently, enabling future extensions such as conversation summarization, user-specific preferences, or persistent agent memory mechanisms.

4.2.5 Storage Manager Limitations

Despite the modular architecture adopted for the Storage Manager, the current implementation presents some limitations related to the scope of the Proof of Concept and the characteristics of the available data sources.

First, the current document ingestion pipeline mainly focuses on PDF, DOCX, and TXT formats. Although the chunking architecture allows the integration of additional loaders, more complex formats such as HTML pages, engineering databases, or domain-specific document structures are not yet directly supported.

Second, the implemented Retrieval-Augmented Generation pipeline relies on a general-purpose embedding model, namely *all-MiniLM-L6-v2*. While this model provides a suitable balance between computational efficiency and semantic retrieval capability, it is not specifically optimized for aerospace engineering terminology. Domain-specific embedding models could potentially improve retrieval performance on highly technical documentation.

Furthermore, the current validation campaign was performed on a limited knowledge base composed of twenty technical documents and one hundred manually generated queries. Although the obtained results demonstrate the effectiveness of the proposed retrieval strategy, a larger benchmark containing a higher variety of documents and automatically generated evaluation scenarios would provide a more comprehensive assessment.

Finally, the current Storage Manager implementation focuses primarily on retrieval and data accessibility. Advanced knowledge management capabilities, such as automatic document updating, conflict detection between different sources, and temporal versioning of knowledge, represent possible future extensions required for deployment in a fully operational environment.

These limitations do not affect the objectives of the Proof of Concept, whose main goal is to demonstrate the feasibility of a modular decision-support architecture. However, they highlight the main aspects requiring further development before industrial adoption.

4.3 AI Manager Module

4.3.1 Overview

The AI Manager Module is the core component responsible for providing intelligent decision-support capabilities within the multi-agent Decision Support System (DSS) developed for space mission operators. The module is designed to assist Mission Managers during operational activities by automatically investigating mission-related queries, collecting evidence from heterogeneous subsystems, and generating

evidence-based operational assessments.

The implemented architecture follows a multi-agent Large Language Model (LLM)-based approach, where different agents are assigned specialized responsibilities instead of relying on a single monolithic reasoning component. This design enables the decomposition of complex operational questions into smaller investigation tasks, improving modularity, scalability, and the traceability of the decision-making process.

The AI Manager workflow is organized as an iterative investigation loop composed of four main phases: planning, evidence retrieval, synthesis, and validation. Given a Mission Manager query, the Planner Agent analyzes the objective and generates an execution plan identifying the required tools and subsystem information needed to answer the request. The selected tools are then executed to retrieve operational evidence from the available mission data sources, including planning information, knowledge bases, ticketing systems, and ground segment network monitoring.

The collected evidence is provided to the Synthesizer Agent, which integrates information from different subsystems and produces a coherent operational assessment with reccomendations. The generated response is subsequently evaluated by a Critic Agent, whose role is to determine whether the available evidence is sufficient to answer the original query. If the investigation is considered incomplete, the Critic identifies the missing information and requests additional tool executions, triggering a new planning and retrieval cycle.

The resulting architecture, summarized in Fig. 4.3, implements a controlled reasoning process where LLM-based agents collaborate through explicit roles and structured information exchange. This approach aims to reduce unsupported assumptions, hallucinations and huge context windows on a single LLM to provide Mission Managers with reliable decision-support information.

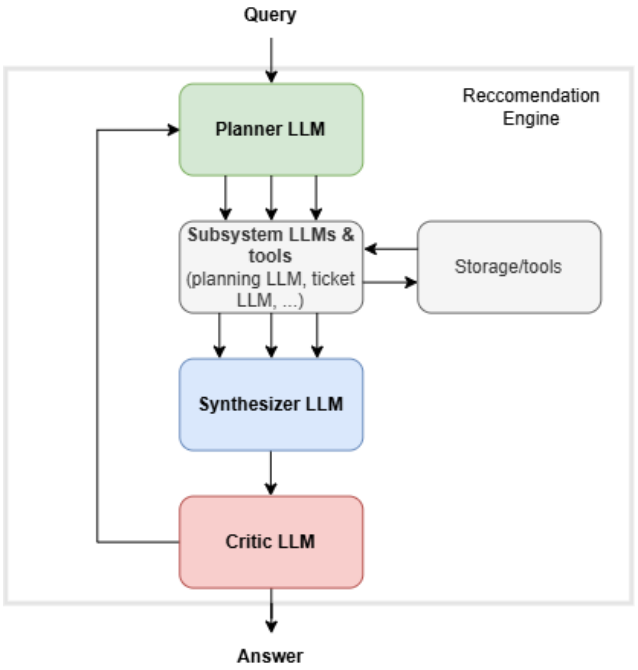


Figure 4.3. Multi-agent reasoning workflow of the AI Manager, showing the iterative interaction between the Planner, tool-based evidence retrieval, Synthesizer, and Critic agents.

4.3.2 Software Architecture and Implementation

The AI Manager Module has been designed following a modular software architecture, separating configuration management, agent lifecycle management, LLM interaction, workflow orchestration, and external tool integration. This organization aims to improve maintainability, extensibility, and the possibility of replacing individual components without affecting the overall system behavior.

The implementation is organized into independent layers, each responsible for a specific aspect of the AI module execution. The main components include the configuration layer, responsible for managing runtime parameters and LLM settings, the agent management layer, responsible for the creation and registration of LLM-based agents, and the workflow and tool layers, which implement the execution logic and interaction with external mission resources.

Configuration Layer

The configuration layer provides a centralized mechanism for managing the parameters required by the AI Manager Module. Configuration data are separated from the application logic, allowing modifications of model parameters and provider settings without requiring changes to the source code.

The layer is composed of the `config_loader.py` module and the `llm_config.yaml` configuration file. The YAML file contains the definition of the LLM agents, for each agent we defined the selected model, provider, system prompt and user prompt. The

configuration loader is responsible for parsing these parameters and making them available during agent initialization.

This approach improves flexibility by enabling the replacement of LLM providers or modification of agent behavior through configuration updates rather than code modifications.

Agent and LLM Management Layer

The agent management layer is responsible for the creation, registration, and retrieval of LLM-based agents within the AI Manager Module. This layer introduces an abstraction between the workflow execution logic and the concrete implementation of individual agents.

The `agent_registry.py` component maintains the association between agent identifiers and their corresponding implementations, enabling the dynamic retrieval of available agents. The registration process is handled by `register_agents.py`, where the supported agents are declared and added to the registry dictionary.

The `AgentFactory` class implements the creation mechanism for agent instances. Instead of directly instantiating specific agent classes throughout the system, components request agents through the factory interface, improving modularity and reducing dependencies between software components. The factory dynamically loads the configuration associated with the requested agent and returns an initialized instance through the method:

```
factory.create_agent(agent_name:  str)
```

The LLM provider management is implemented through an abstraction layer based on the `BaseProvider` abstract class. The `GroqProvider` class extends this interface by implementing the `call()` method required to interact with the selected LLM service. The method receives the model configuration, including model name, system prompt, user prompt, and generation parameters such as temperature, and returns the generated response.

During the orchestration workflow, the generated responses are dynamically inserted into subsequent user prompts through string formatting, allowing the information produced by different agents to be propagated across the multi-agent execution flow.

4.3.3 Multi-Agent Architecture

The AI Manager Module adopts a multi-agent architecture in which multiple Large Language Model (LLM)-based components cooperate to support the decision-making process of space mission operators. Instead of relying on a single general-purpose LLM responsible for the entire workflow, the system decomposes the investigation process into specialized components, each assigned to a specific operational task.

The architecture follows an evidence-driven approach, where the objective of the agents is not to generate autonomous conclusions, but to collect, process, and correlate mission-related information retrieved from different sources. The collaboration among agents is coordinated by a graph-based orchestration layer, which manages the execution flow and maintains the shared state of the investigation.

Agent Specialization

Each LLM-based agent is designed with a specific responsibility within the operational assessment workflow. The specialization of the agents allows the system to reduce the complexity of individual reasoning tasks and improves the traceability of the decision-support process.

The main agents composing the AI Manager Module are:

- **Planner Agent:** responsible for analyzing the Mission Manager query and generating an execution plan. The agent determines which information sources are required and defines the input parameters that must be provided to the available tools. The Planner Agent does not directly answer the user query, but coordinates the information acquisition phase.
- **Master Agent (Synthesizer):** responsible for integrating the evidence collected from the different sources and producing the final operational assessment. The Master Agent correlates information from multiple subsystems, identifies operational impacts, and provides recommendations only when supported by the retrieved evidence.
- **Critic Agent:** responsible for evaluating whether the available evidence is sufficient to answer the Mission Manager query. The Critic Agent does not perform additional information retrieval directly, but determines whether further investigation is required. If relevant information is missing, the agent generates feedback that is provided to the Planner Agent to trigger an additional investigation cycle.
- **Query Agent:** responsible for enhancing and reformulating user queries before they are provided to retrieval-based components. This agent improves the effectiveness of the Retrieval-Augmented Generation (RAG) process by transforming natural language requests into more suitable retrieval queries.
- **Planning Agent:** responsible for translating high-level planning requests into structured filtering parameters for the planning data source. For example, when a Mission Manager requests the activities associated with a specific satellite, the Planning Agent identifies the relevant constraints, such as satellite identifier, time interval, or ground station, before querying the planning tool. This prevents unnecessary retrieval of unrelated mission data.
- **Ticket Agent:** responsible for analyzing available anomaly and ticket information and selecting only the items relevant to the current investigation. The agent receives the investigation requirements generated by the Planner Agent and filters the available ticket information to reduce irrelevant data.
- **Knowledge Base Agent:** responsible for filtering and prioritizing the documents retrieved by the Retrieval-Augmented Generation (RAG) component. The agent identifies the documents most relevant to the current investigation and forwards only the selected evidence to the Synthesizer Agent, reducing irrelevant information and unnecessary context.

The architecture therefore separates information acquisition, evidence synthesis, and validation activities into independent components. This separation enables the extension of the system with additional specialized agents or data sources without requiring modifications to the overall decision-support workflow.

Agent Responsibilities and Collaboration Mechanism

The interaction among agents is not implemented through direct agent-to-agent communication. Instead, the agents cooperate through a centralized orchestration mechanism based on a shared execution state. The orchestration layer is responsible for coordinating the execution of the different agents, maintaining the investigation context, and propagating intermediate results throughout the workflow.

The shared state contains the information generated during the investigation lifecycle, including the original Mission Manager request, conversation history, generated execution plans, retrieved tool results, synthesizer responses, critic feedback, and information about previously executed tools. This representation allows each agent to access the results produced by previous steps without requiring direct communication with other agents.

The investigation workflow starts when the Planner Agent receives the Mission Manager query and generates an execution plan identifying the tools and information required to address the request. The selected tools can then be executed in parallel to retrieve evidence from the available mission data sources. The retrieved information is stored in the shared state and provided to the Synthesizer Agent, which correlates the available evidence and generates an operational assessment.

The generated assessment is subsequently analyzed by the Critic Agent. Its role is to determine whether the available evidence is sufficient to provide a complete and reliable answer to the original query. If the investigation is considered incomplete, the Critic Agent identifies the missing information and produces feedback containing additional tool requests required to complete the investigation. This feedback is stored in the shared state and provided to the Planner Agent, which generates a new execution plan based on the identified information gaps.

This iterative mechanism is particularly important because relevant information gaps may only become apparent after the execution of an initial tool. For example, if the Mission Manager asks the system to “check the network state”, the Planner Agent may initially invoke the network monitoring tool. If the tool reports a degraded network condition, however, the initial result does not explain the underlying cause. The Critic Agent can identify this information gap and request additional evidence, such as open tickets or other relevant operational data, allowing the Planner Agent to extend the investigation accordingly.

The resulting workflow therefore implements an iterative investigation loop, as illustrated in Fig. 4.3. Rather than operating as a set of independent autonomous agents, the components form a coordinated workflow in which each agent performs a specialized reasoning function and the orchestration layer controls the sequencing and exchange of information.

The use of a shared execution state provides two main advantages. First, it improves the traceability of the decision-support process by preserving the intermediate information generated during the investigation. Second, it enables iterative

reasoning cycles while allowing the system to retain previously retrieved evidence and avoid unnecessary or redundant tool executions. This mechanism supports a controlled evidence acquisition process in which additional information is collected only when required to answer the original query.

4.3.4 Graph-Based Orchestration with LangGraph

The execution workflow of the AI Manager Module is implemented using LangGraph, a graph-based orchestration framework designed to represent complex LLM workflows as stateful directed graphs. Instead of implementing the interaction between agents through direct function calls, the proposed architecture models the investigation process as a sequence of computational nodes that operate on a shared state.

This approach provides a centralized mechanism for controlling the interaction between the Planner, specialized tools, Synthesizer, and Critic agents. Each component modifies specific portions of the shared state, while the graph structure defines the execution order and the possible transitions between investigation steps.

State Management

The communication between the different components of the workflow is handled through a shared state object defined by the `AgentState` TypedDict. This state acts as the central memory of the investigation process, storing both intermediate reasoning artifacts and the information required by subsequent nodes.

The state contains the original user query and conversation messages, the execution plan generated by the Planner Agent, the retrieved evidence produced by the tools, the synthesized answer generated by the Synthesizer Agent, and the feedback produced by the Critic Agent.

The main state variables are:

- `messages`: stores the conversation history exchanged with the Mission Manager. The `add_messages` reducer allows new messages to be appended during graph execution while preserving previous interactions.
- `execution_plan`: contains the list of tool calls generated by the Planner Agent, including the required inputs for each tool.
- `tool_results`: stores the evidence retrieved from the different mission subsystems and makes it available to the Synthesizer Agent.
- `synthesized_response`: contains the operational assessment generated by the Synthesizer Agent.
- `critic_feedback`: stores the decision of the Critic Agent and, when required, the additional tool calls needed for a new investigation cycle.
- `remaining_iterations`: limits the number of refinement cycles to prevent uncontrolled execution loops.

The adoption of a shared state mechanism allows the individual agents to remain independent and stateless, while the LangGraph workflow manages the information exchange between them. This design improves modularity, since new agents or tools can be introduced without requiring modifications to the internal logic of existing components.

Graph Engine

The `GraphEngine` class represents the main orchestration component of the AI Manager Module. Its responsibility is to initialize the required LLM agents, register the available tools, create the graph nodes, and compile the final workflow.

During initialization, the Graph Engine retrieves the required agents through the previously described Agent Factory mechanism. The instantiated agents are then provided to the graph nodes and to the corresponding tools.

The workflow is represented as a directed graph using LangGraph's `StateGraph`. The implemented execution sequence is composed of four main nodes:

- **Dispatcher node**: invokes the Planner Agent to generate the investigation plan.
- **Tool node**: executes the tools selected by the Planner and stores the retrieved evidence.
- **Master node**: invokes the Synthesizer Agent to generate the operational assessment.
- **Critic node**: evaluates whether the available evidence is sufficient and determines whether another investigation cycle is required.

The normal execution flow follows the sequence:

$$Dispatcher \rightarrow Tools \rightarrow Master \rightarrow Critic$$

After the Critic execution, the graph performs a conditional transition. If the investigation is considered complete, the workflow terminates. Otherwise, the execution returns to the Dispatcher node, where the Planner Agent receives the critic feedback and generates a refined execution plan.

This iterative loop allows the system to progressively acquire additional evidence only when required, while avoiding unnecessary tool executions.

A checkpoint mechanism based on LangGraph's `MemorySaver` is also integrated into the workflow. The checkpointing system enables the persistence of the graph state associated with a specific conversation identifier, allowing the continuation of previous Mission Manager interactions.

Agent Nodes

Within LangGraph, each agent interaction is implemented as a graph node. The node abstraction separates the workflow orchestration logic from the internal implementation of the individual LLM agents.

The `GraphNodes` class encapsulates the functions executed by the graph nodes and provides each node with access to the required LLM agents and tools.

The **Dispatcher node** is responsible for preparing the input data required by the Planner Agent. It extracts the latest user query from the conversation state, includes previous critic feedback if available, and invokes the Planner Agent. The generated execution plan is then stored in the shared state.

The **Tool node** receives the execution plan generated by the Dispatcher and forwards each requested operation to the Tool Executor. The returned information is stored in the `tool_results` field, becoming the evidence base for the Synthesizer Agent.

The **Master node** combines the user request with the retrieved evidence and invokes the Synthesizer Agent. The resulting operational assessment is stored in the graph state.

Finally, the **Critic node** evaluates the generated answer together with the retrieved evidence. Its output determines whether the current investigation cycle is sufficient or whether additional information must be collected.

This node-based organization allows each reasoning step to be independently modified or extended while maintaining a clear execution structure.

Tool Integration

The interaction with external data sources and subsystem-specific processing logic is implemented through a modular tool architecture. Every tool inherits from the abstract `BaseTool` class, which defines a common interface composed of a unique tool identifier and an execution method.

The `ToolExecutor` component acts as an intermediary between the graph nodes and the available tools. Tools are dynamically registered during the initialization of the Graph Engine and are retrieved at runtime according to the execution plan generated by the Planner Agent.

The implemented tools provide access to the main mission information sources:

- **Network Tool**: retrieves the current ground segment network status from the storage layer.
- **Planning Tool**: combines LLM-based query interpretation with database filtering. The Planning Agent converts the natural language request into structured parameters, such as satellite identifier, ground station, and time interval, which are then used to retrieve only relevant planning information.
- **Knowledge Base Tool**: provides access to the RAG subsystem by combining query enhancement and document retrieval capabilities.
- **Ticket Tool**: uses the Ticket Agent to filter operational anomalies and return only tickets relevant to the current investigation.

This abstraction enables the orchestration layer to interact with heterogeneous information sources through a unified interface. Consequently, new tools can be added to the system by implementing the `BaseTool` interface and registering the new component in the Tool Executor, without modifying the graph execution logic.

4.3.5 LLM Agent Implementation

This subsections will firther expose the implementation details of the different LLMs agents used in the PoC.

Base LLM Class

The `BaseLLM` abstract class serves as a standardized interface for interacting with Large Language Models, decoupling prompt construction logic from the underlying model provider execution.

This architecture was chosen to support key software design principles:

- **Template Method Pattern:** The `speak` method defines the core execution workflow, while delegating dynamic prompt creation to the abstract `build_prompt` method. Subclasses only need to define how input data is formatted into a prompt without duplicating execution logic.
- **Provider Decoupling (Strategy Pattern):** By delegating the actual API invocation to an injected `BaseProvider` instance, the LLM logic remains completely independent of specific vendor SDKs or API client implementations (e.g., OpenAI, Anthropic, or Ollama).
- **State Encapsulation:** The class encapsulates model configurations, such as the model name, system instructions, and prompt templates, ensuring consistent, reproducible executions across the application.

As a consequence, implementing a new agent only requires defining its configuration in the YAML file (model, prompts, and provider) and implementing the corresponding `build_prompt` method responsible for formatting the runtime inputs.

For example for the ticket LLM which has the following user prompt:

```
### Context
I need you to filter and summarize open ground segment tickets that
are relevant to the following incoming user query.

### User Query
"{user_query}"

### Open Tickets Data
Below is the list of currently open ground segment tickets.  Evaluate
each one carefully.

{tickets}

### Instructions
Analyze the query and the tickets.  Generate the JSON payload following
the strict schema defined in your system prompt.  Ensure your summaries
isolate technical facts (error codes, specific antennas, frequency
bands, software versions) that will help the next LLM diagnose the
user's issue.
```

We will just have to implement this method in its class definition:

```
def build_prompt(self, data):

return self.user_prompt.format(
    user_query=data['user_query'],
    tickets = data.get("tickets", [])

)
```

Specialized Agents Details

The AI Manager Module is composed of multiple specialized LLM agents, each designed to perform a well-defined task within the investigation workflow. Instead of assigning all reasoning capabilities to a single model, the proposed architecture distributes the decision-support process among agents with complementary responsibilities.

Although the agents implement different reasoning tasks, they all follow a common design philosophy. Each agent is instructed through a specialized system prompt that clearly defines its responsibilities, constraints, and expected output format. Whenever the agent output is consumed by another component of the system, the prompt enforces a structured JSON response. This design simplifies the orchestration logic, enables deterministic parsing of the generated content, and constrains the reasoning process by requiring the model to produce information in a predefined schema rather than unrestricted natural language.

The implemented agents can be divided into two categories. The first category includes the *core orchestration agents*, which coordinate the investigation process and produce the final operational assessment. The second category consists of *tool-support agents*, whose purpose is to improve the interaction with specific data sources by preprocessing queries or filtering retrieved information before it is provided to the orchestration agents.

Planner Agent The Planner Agent represents the entry point of the investigation workflow. Given a Mission Manager query, it analyzes the user's objective and generates a structured execution plan describing which tools must be executed and the information that should be retrieved from each subsystem. Rather than directly answering the user's request, the Planner Agent coordinates the information acquisition process by producing a JSON execution plan containing the selected tools together with the corresponding input parameters. When additional evidence is requested by the Critic Agent, the Planner Agent generates a revised execution plan based on the received feedback. The Planner Agent is configured to operate in two distinct modes. During the initial investigation, it autonomously selects the minimum set of tools required to answer the Mission Manager query while avoiding redundant tool executions. When the Critic Agent requests additional evidence, the Planner switches to a repair mode in which the critic feedback becomes authoritative and is incorporated into a refined execution plan. This iterative planning strategy enables progressive evidence acquisition without repeating unnecessary investigations. The Planner is also tasked to provide the input for the tools when required. For example the ticket tool can receive a query to be forwarded to the

Ticket LLM that asks to check for tickets that are related to planning informations or network problems. To facilitate deterministic orchestration, the Planner Agent is instructed to return only a structured JSON document containing the investigation goal, the planning rationale, and the ordered execution plan. This structured output can be directly parsed by the orchestration layer, eliminating the need for natural language interpretation and reducing ambiguity during workflow execution.

Synthesizer Agent The Synthesizer Agent, also referred to as the Master Agent, is responsible for producing the final operational assessment. After the execution of the selected tools, it receives all the collected evidence and correlates the information originating from the different mission subsystems. The generated response is exclusively based on the retrieved evidence by the evidence-grounded prompt strategy and includes operational impacts, supporting evidence, possible mitigation actions, and any remaining knowledge gaps. The Synthesizer Agent is explicitly instructed not to generate unsupported conclusions or information that is not grounded in the available evidence. The system prompt explicitly constrains the agent to correlate information only when the relationships are supported by the collected evidence. Rather than behaving as a general-purpose conversational assistant, the Synthesizer acts as an evidence integration component, combining information originating from planning, network monitoring, ticketing, and knowledge base retrieval into a coherent operational assessment. To improve the readability of the generated assessment, the prompt enforces a predefined response structure including a situation summary, supporting evidence, operational impact, recommended actions, identified knowledge gaps, and the final answer to the Mission Manager. This organization simplifies the interpretation of the assessment while maintaining traceability between conclusions and supporting evidence.

Critic Agent The Critic Agent represents the final stage of the investigation loop. Its responsibility is not to validate the correctness of the generated text, but to determine whether the available evidence is sufficient to answer the Mission Manager's request. Instead, the Critic implements the investigation termination criterion of the proposed multi-agent architecture. Its objective is to determine whether the available evidence is sufficient to answer the Mission Manager query, rather than maximizing the amount of collected information. If the retrieved information is considered adequate, the investigation terminates. Otherwise, the Critic Agent identifies the missing evidence, specifies which additional tool executions are required, and provides structured feedback to the Planner Agent, triggering a new investigation cycle. The prompt explicitly follows a minimum sufficient evidence principle, encouraging the agent to terminate the investigation whenever the available evidence is adequate. Additional tool executions are recommended only when the missing information could materially affect the final operational assessment. Similar to the Planner Agent, the Critic returns its decision as a structured JSON object containing the investigation status, the additional tool calls required, and the rationale supporting the decision. This structured feedback can be directly consumed by the Planner Agent to generate a refined investigation plan during the next execution cycle.

Query Agent The Query Agent is employed by the Knowledge Base Tool to improve the retrieval process of the Retrieval-Augmented Generation (RAG) subsystem. Its purpose is to reformulate the original user request into a more effective retrieval query, increasing the probability of retrieving relevant documentation while preserving the original semantic meaning.

Planning Agent The Planning Agent is integrated within the Planning Tool and acts as an intelligent query interpreter for planning data. Instead of querying the planning database using the original user request, the agent extracts structured filtering parameters, such as satellite identifiers, time intervals, or ground stations, allowing the Planning Tool to retrieve only the relevant operational data and reducing unnecessary data processing.

Ticket Agent The Ticket Agent is used by the Ticket Tool to analyze the set of available anomaly reports and operational tickets. Based on the investigation requirements generated by the Planner Agent, it filters the available tickets and returns only those relevant to the current investigation. This approach significantly reduces the amount of irrelevant information propagated to the Master Agent while preserving the evidence required for the final operational assessment.

Knowledge Base Agent The Knowledge Base Agent is integrated within the Knowledge Base Tool and is responsible for filtering and prioritizing the documents retrieved by the Retrieval-Augmented Generation (RAG) subsystem. After the initial retrieval phase, the agent receives the set of candidate documents identified as potentially relevant to the Mission Manager's query. It analyzes their content and relevance with respect to the investigation objective and selects the most informative documents to be provided to the Synthesizer Agent.

The agent acts as a relevance filtering layer between the retrieval and synthesis stages. Its purpose is to reduce the amount of irrelevant or redundant information propagated to the Synthesizer while preserving the documents containing the evidence most relevant to the current investigation. This approach limits the context that must be processed by the Synthesizer Agent and improves the focus and consistency of the final operational assessment.

The selected documents, together with their associated metadata and source information, are then returned to the orchestration workflow as evidence for the Synthesizer Agent. In this way, the Knowledge Base Agent does not generate the final answer, but supports the evidence-grounding process by ensuring that the synthesis stage receives a more focused and relevant set of retrieved information.

The LLM model assigned to each specialized agent is summarized in Table 4.3, while the workflow of the agents is summarized in Fig. 4.4.

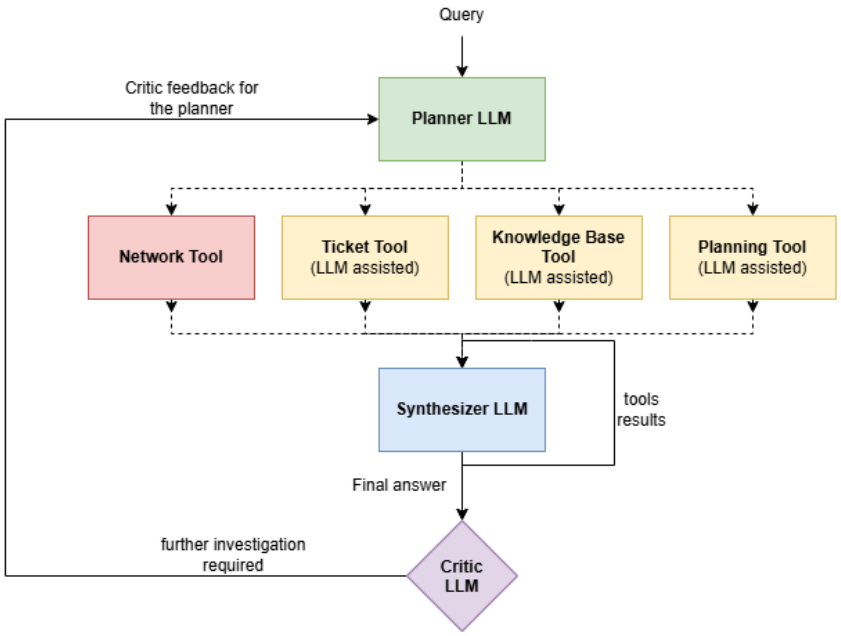


Figure 4.4. This figure summarizes the workflow of the different agents of the system. The mission manager query is passed to the planner agent that decides what tools and agents have to be called to answer. The results are then passed to the Synthesizer that will provide a final answer. The synthesizer's response and the tools results are then passed to the critic that will determine if other tools are necessary for a more complete response. If so the critic will generate a feedback that is given to the planner for a plan refinement and another tool calling, only the new tools are going to be called if the inputs for the others are unchanged.

Table 4.3. LLM models employed by the specialized agents of the AI Manager Module. The model allocation follows the functional complexity of the agents. The core orchestration agents use the larger Llama 3.3 70B model, as they are responsible for planning, evidence synthesis, and investigation validation. In contrast, the tool-support agents use the smaller Llama 3.1 8B model, since their tasks are more constrained and domain-specific, such as query reformulation, parameter extraction, and relevance filtering.

Agent	Category	LLM Model
Planner Agent	Core orchestration	llama 3.3 70B Versatile
Synthesizer Agent	Core orchestration	llama 3.3 70B Versatile
Critic Agent	Core orchestration	llama 3.3 70B Versatile
Query Agent	Tool-support	llama 3.1 8B Instant
Planning Agent	Tool-support	llama 3.1 8B Instant
Knowledge Base Agent	Tool-support	llama 3.1 8B Instant
Ticket Agent	Tool-support	llama 3.1 8B Instant

Overall, the proposed prompting strategy transforms the individual LLMs into specialized modules with clearly defined responsibilities rather than general conversational agents. Combined with structured outputs and graph-based orchestration, this design reduces ambiguity in the interaction among components and improves

the robustness and reproducibility of the decision-support workflow.

Chapter 5

Proof of Concept Evaluation

5.1 Evaluation Objectives

The objective of the Proof of Concept (PoC) evaluation is to assess the ability of FUNES to provide context-aware decision support in a representative Earth Observation ground segment scenario. The evaluation focuses on the core capabilities introduced by the proposed architecture, with particular emphasis on the retrieval and integration of heterogeneous operational information, the identification of anomalies and their operational impact, and the generation of appropriate decision-support outputs.

The evaluation is structured to progressively assess the capabilities required by FUNES to support mission operators. At a first level, the evaluation considers the ability of the system to retrieve relevant information from individual operational sources. At a second level, it assesses the ability to correlate information originating from different mission domains, such as network monitoring, incident management, and mission planning. Finally, more complex tasks are considered, requiring the system to combine information from multiple sources, identify the resulting operational risks, and provide recommendations consistent with the available evidence and with the role of a Decision Support System.

A further objective is to assess the contribution of the proposed retrieval and multi-agent orchestration approach with respect to a general-purpose Large Language Model operating directly on the complete operational context. For this purpose, the responses generated by FUNES are compared with those produced by a baseline LLM (llama 3.3 70b instruct) under the same operational scenario. The comparison is intended to evaluate whether the system architecture can improve the ability to identify relevant information, correlate operational evidence, and provide decision-support outputs while limiting the amount of contextual information that must be supplied to the underlying language model.

The evaluation therefore addresses the following main objectives:

- assess the capability of FUNES to retrieve relevant information from heterogeneous operational sources;
- evaluate the ability to correlate information across different mission domains;

- assess the identification of operational anomalies and their impact on planned satellite activities;
- evaluate the accuracy of planning-related information and the identification of affected operations;
- assess the ability of the system to filter irrelevant information and focus on the elements relevant to the operator's query;
- evaluate whether the generated outputs are consistent with the role of a Decision Support System, supporting rather than replacing human decision-making;
- assess the quality and operational consistency of the recommendations generated for complex scenarios;
- compare the resulting decision-support performance with that of a general-purpose LLM provided with the complete operational context.

The evaluation is intended as a validation of the PoC capabilities rather than as a comprehensive validation of a production-ready operational system. Consequently, the results are interpreted in terms of the feasibility and effectiveness of the proposed approach within the considered representative scenario, rather than as a general measure of system performance across all possible mission operational conditions.

5.2 Evaluation Scenario

A representative Earth Observation ground segment scenario was defined to evaluate the Proof of Concept under operational conditions requiring the analysis and correlation of heterogeneous information sources. The scenario, depicted in Fig. 5.1, was designed to reproduce, at a simplified level, the type of information fragmentation that characterizes a real mission operations environment, where the assessment of an operational situation may require information from mission planning, network monitoring, incident management, and mission knowledge sources.

The scenario includes three ground stations, namely GS-01, GS-02, and GS-03, and four satellites, namely SAT-01, SAT-02, SAT-03, and SAT-04. SAT-01 is nominally served by GS-01, SAT-02 and SAT-03 by GS-02, and SAT-04 by GS-03. GS-01 is additionally connected to GS-02 and GS-03 through inter-station network links, introducing dependencies between the operational status of different ground stations.

The operational context was constructed from four main information domains: mission planning, network monitoring, incident and change management, and mission knowledge. The resulting scenario provides both nominal operational information and intentionally introduced anomalies, allowing the evaluation to consider not only direct information retrieval but also cross-domain correlation and operational impact analysis.

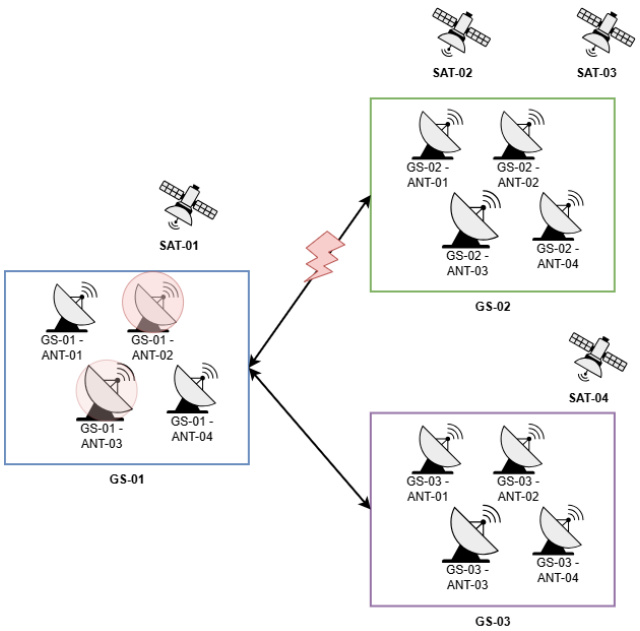


Figure 5.1. Overview of the evaluation scenario: the are 3 ground segment stations GS-01, GS-02, and GS-03, each with 4 antennas, and 4 satellites SAT-01, SAT-02, SAT-03, and SAT-04. The GS-01 is connected to both GS-02 and GS-03 through inter-station network links. The link between GS-01 and GS-02 is degraded, while the link between GS-01 and GS-03 is operational but exhibits increased latency. The GS-01/ANT-02 is unavailable, while GS-01/ANT-03 is degraded. The remaining antennas of GS-01 are operational, while GS-02 and GS-03 are nominal at station level. The ANT positioning is close to the respective ground stations.

5.2.1 Mission Planning

For each satellite, approximately 36 hours of mission planning data were generated around the evaluation period. The planning information is structured as a sequence of satellite passages, with each passage associated with a ground station and a set of scheduled activities. Each activity contains its type, temporal interval, and the antenna on which it is scheduled.

The activities are not identical across passages and may include different operational types, such as uplink (UPL) and downlink (DWL) operations. This variability was introduced to provide a realistic planning context in which the impact of an unavailable resource cannot be determined solely from the satellite or ground station identifier, but requires inspection of the actual scheduled activities and their temporal characteristics.

The planning dataset therefore enables questions involving different levels of temporal and operational reasoning, including the retrieval of the next satellite pass, identification of scheduled activities, identification of affected acquisitions, and assessment of potential planning conflicts.

5.2.2 Ground Segment and Network Status

The scenario contains network monitoring information for all three ground stations and their associated antennas. For each station, the network data include the overall station status, backhaul capacity and utilization, average latency, and packet loss. At antenna level, the status, throughput, latency, packet loss, and utilization are provided.

The network scenario intentionally includes different operational conditions. GS-01 is in a degraded overall state, with antenna ANT-02 offline and ANT-03 exhibits increased latency and packet loss. The remaining antennas of GS-01 are operational. GS-02 and GS-03 are nominal at station level, although their individual resource utilization differs.

Inter-station connectivity is also included in the scenario. The connection between GS-01 and GS-02 is degraded, with reduced available bandwidth, increased latency, and packet loss. The connection between GS-01 and GS-03 is reported as operational in terms of connectivity status, but exhibits a significantly higher latency. These conditions introduce additional information that must be interpreted and filtered in relation to the actual operational activities and user questions rather than treated independently.

This configuration was deliberately designed to avoid a scenario in which the presence of an anomaly directly determines its operational consequence. For example, detecting that an antenna is unavailable is not sufficient to determine whether satellite operations are affected. The corresponding planning information must also be examined to identify the activities scheduled on the affected resource.

5.2.3 Operational Events and Tickets

The scenario includes a set of operational events representing anomalies, maintenance activities, and external operational requests. These events generate different

types of operational records, including incident tickets, change tickets, and customer requests.

The main events include the unavailability of GS-01/ANT-02, degradation of the connectivity between GS-01 and GS-02, planned preventive maintenance on GS-02/ANT-02, and an additional customer acquisition request for the busy satellite SAT-01 of GS-01. These events have been associated with potential impacts on scheduled satellite activities, allowing the system to assess the relationship between an operational event and its consequences on mission planning.

To reproduce the presence of unrelated information typically encountered in operational environments, the scenario contains 32 tickets in total. Only six of these tickets are directly relevant to the operational events introduced in the scenario, while the remaining tickets describe plausible but unrelated operational issues and ground segments events. These additional records act as contextual noise and require the system to distinguish information relevant to the current operational situation from unrelated historical or concurrent events.

The inclusion of irrelevant but plausible tickets is particularly important for the evaluation of a decision-support system. In a realistic operational environment, the presence of a large amount of available information does not imply that all information is relevant to a given operator query. The ability to identify and filter the relevant evidence is therefore considered an integral part of the evaluation scenario.

5.2.4 Mission Knowledge Base

The scenario is complemented by a knowledge base containing 20 documents covering system informations and ECSS standards. These documents provide contextual knowledge that is not directly represented in the operational planning, network, or ticket data.

The knowledge base is used to support questions requiring background information about the mission and its systems, as well as questions in which operational observations need to be interpreted using previously defined system knowledge. Its inclusion allows the evaluation to distinguish between direct retrieval of operational data and the use of persistent domain knowledge during the reasoning process.

5.2.5 Scenario Design Rationale

The scenario was deliberately constructed to reproduce the main characteristics that motivate the FUNES architecture: heterogeneous information sources, temporal dependencies, operational anomalies, cross-domain relationships, and the presence of irrelevant information.

A scenario based exclusively on isolated retrieval questions would not adequately test the capabilities targeted by the PoC. For example, identifying that an antenna is offline is a relatively simple retrieval task. Determining whether this condition affects a satellite operation, however, requires the system to correlate the network status with the mission planning and to identify the activities assigned to the affected resource. Similarly, determining whether an operational issue requires immediate attention may require the combination of anomaly information, incident records, mission planning, and domain knowledge.

For this reason, the scenario was designed with increasing levels of information dependency. Some questions can be answered using a single information source, while others require the correlation of two or more sources. The most complex questions require the integration of multiple operational domains to identify current risks and formulate appropriate decision-support recommendations.

The presence of both relevant and irrelevant information was also intentional. The objective was not to create an artificially clean dataset in which every available record contributes to the answer, but to approximate the information overload encountered in operational environments. This allows the evaluation to consider not only whether FUNES can retrieve information, but also whether it can identify the information that is relevant to the specific operational question.

5.3 Evaluation Tasks and Experimental Setup

The evaluation was designed around a set of operational questions derived from a representative ground segment scenario. The questions were defined to progressively increase in complexity and to exercise the main capabilities provided by the FUNES architecture. In particular, three levels of task complexity were considered: single-source retrieval, multi-source correlation, and decision-support tasks.

5.3.1 Evaluation Tasks

The first category consists of *easy tasks*, which require the retrieval of information from a single operational source. These tasks are intended to verify the basic ability of the system to identify the appropriate information source and retrieve the relevant information. The considered sources include the knowledge base, mission planning, network monitoring, and incident management systems. Representative questions include queries regarding the architecture and operational interfaces of the Flight Operations System, the current network status, the status of a ground-station antenna, currently open incidents, and the next scheduled satellite passes.

The second category consists of *medium tasks*, which require the retrieval and correlation of information from multiple operational sources. These tasks are designed to evaluate whether the system can combine heterogeneous information to derive operationally relevant conclusions. For example, determining why a ground station is unavailable requires the correlation of network information with incident records, while identifying the satellite acquisitions affected by an incident requires information from both the incident management and mission planning domains. Other tasks require the system to determine whether a ground station is available for a planned acquisition or whether an acquisition can be rescheduled based on the available planning information.

The third category consists of *hard tasks*, which extend the previous category by requiring situational assessment and decision support. These tasks require the system to integrate information from multiple operational sources, identify relevant anomalies, determine their impact on mission activities, and, where appropriate, propose possible operational actions. Examples include identifying the current operational issues affecting the ground segment, determining which issues require immediate attention, identifying risks to planned activities for a given operational

day, and summarizing current anomalies together with their operational impact and possible mitigation actions.

This progressive organization allows the evaluation to distinguish between the ability to retrieve individual pieces of information and the higher-level capabilities required for operational decision support. The complete set of evaluation questions is reported in Table 5.1.

5.3.2 Experimental Setup

The evaluation was conducted by comparing the responses generated by the FUNES PoC with those produced by a general-purpose Large Language Model used as a baseline. FUNES was evaluated using its complete processing pipeline, including the selection of the appropriate information sources, retrieval of the relevant operational context, correlation of the retrieved information, and generation of the final response.

As a baseline, *Llama 3.3 70B Instruct* was provided with the complete operational context associated with the evaluation scenario. In contrast to FUNES, the baseline did not perform explicit source selection or selective retrieval. Instead, the entire available context, of around 24000 tokens, was provided directly to the model together with the user query. This configuration was selected to provide a reference for assessing the contribution of the retrieval and orchestration mechanisms implemented by FUNES. The model has the same system prompt and user prompt of the synthesizer agent of FUNES and the same `temperature = 0`.

The two systems were evaluated using the same set of operational questions and the same underlying scenario. For each question, the generated response and, for FUNES, the information sources selected during the reasoning process were recorded. This allowed the evaluation to consider both the quality of the final answers and the behaviour of the underlying information-selection mechanism.

5.4 Evaluation Methodology

For each question, both systems generated an answer independently. The resulting answers were subsequently evaluated using an LLM-as-a-Judge approach [70]. Llama 3.3 70b instruct with `temperature = 0` was employed as an independent evaluator and was provided with the operational context, the question, and the two generated answers. The identities of the systems were not considered part of the evaluation criteria, and the assessment focused exclusively on the content and operational validity of the generated responses. For each question, the individual criterion scores were recorded and subsequently aggregated to enable comparison between FUNES and the baseline across the different task complexity levels and evaluation dimensions.

The use of an LLM-as-a-Judge was selected to provide a consistent evaluation procedure for responses where correctness cannot always be expressed through exact matching. In particular, complex decision-support questions may admit multiple valid formulations or operationally reasonable recommendations. The evaluator therefore assesses the consistency of each response with the evidence and operational constraints defined by the scenario rather than requiring an exact reference answer.

To reduce potential evaluator bias related to the identity or position of the compared systems, the association between the two responses and the system identities was randomized during the evaluation.

The evaluation criteria were defined to reflect the main capabilities expected from an operational Decision Support System. Each criterion was evaluated using a three-level ordinal scale, where:

- Score 0 - Incorrect: A score of 0 was assigned when the answer contained a material factual or operational error that directly affected the criterion being evaluated. This included, for example, false statements, contradictions with the operational context, incorrect correlations, materially inaccurate planning information, unsupported causal claims that affected the validity of the answer, or operational recommendations that were incompatible with the available evidence. Minor imprecisions or isolated omissions were not, by themselves, considered sufficient to warrant a score of 0.

- Score 1 - Partially correct: A score of 1 was assigned when the answer was substantially correct but exhibited one or more significant limitations preventing it from fully satisfying the criterion. Such limitations included the omission of relevant information, the establishment of only part of a required correlation, failure to identify an important operational impact, a limited error that did not invalidate the overall answer, or a recommendation that was relevant but overly generic or incomplete. Thus, a score of 1 indicated that the answer was generally useful and substantively correct, but did not completely meet the requirements of the criterion.

- Score 2 - Fully correct: A score of 2 was assigned when the answer fully satisfied the criterion for the specific question. To receive this score, the answer was required to provide the information requested, correctly incorporate the relevant operational evidence, establish supported correlations, accurately report planning information when required, and provide appropriate and evidence-consistent recommendations when requested.

The criteria were evaluated independently for each answer against the operational context provided to the evaluator. Non-applicable criteria were excluded from the total score rather than being assigned a zero value.

The following criteria were considered in the evaluation:

- **Answer Correctness (AC):** evaluates whether the answer correctly addresses the question using the available operational evidence. This includes the correct reporting of requested information, values, identifiers, operational states, and conclusions, while penalizing factual errors and unsupported claims that materially affect the answer.

- **Anomaly Identification (AI):** evaluates the ability to identify and correctly characterize the operational anomalies relevant to the question. The criterion considers whether relevant anomalies, their status, and associated technical information are correctly reported without introducing unrelated or unsupported anomalies.

- **Evidence Correlation (EC):** evaluates the ability to establish correct relationships between evidence originating from different operational sources, such as network monitoring, incident reports, planning information, and the knowledge base. Unsupported causal or operational relationships are not considered valid correlations.
- **Noise Filtering (NF):** evaluates the ability to distinguish information relevant to the question from unrelated information contained in the operational context. A high score indicates that the answer uses relevant evidence without introducing unrelated information or unsupported relationships.
- **DSS Role Adherence (DSS):** evaluates whether the answer behaves consistently with the role of an operational Decision Support System. The criterion considers whether the system provides evidence-based situational awareness, appropriately identifies operational implications and uncertainties, supports human decision-making, and avoids presenting unsupported conclusions or assuming autonomous operational authority.
- **Operational Recommendation Quality (ORQ):** evaluates the quality of operational recommendations when the question explicitly requires mitigation actions or a proposed course of action. Recommendations are assessed according to their relevance, consistency with the available evidence, operational usefulness, and respect for known constraints and uncertainties.

Not all evaluation criteria are applicable to every question. The applicable criteria depend on the information and reasoning capabilities required by the specific task. For example, a single-source network status query primarily evaluates information retrieval and anomaly identification, whereas a question concerning the impact of an incident on scheduled acquisitions requires evidence correlation and planning accuracy. Similarly, recommendation-oriented questions require the additional assessment of Decision Support System adherence and operational action quality. This criterion-dependent evaluation avoids penalising a response for not providing information or reasoning that was not required by the question, Table 5.3 shows what criteria are applicable to each question.

The methodology is intended to provide a comparative evaluation of the PoC rather than a statistically exhaustive validation of an operational system. The results should therefore be interpreted within the scope of the considered scenario, question set, and evaluation procedure. In particular, the use of an external LLM as an evaluator introduces an additional source of potential evaluation bias, while the proprietary nature of the evaluator prevents complete inspection of its internal decision process. These aspects are considered when discussing the results and limitations of the evaluation.

The set of questions is reported in Table 5.1, together with their corresponding identifiers. The questions are designed to progressively increase in complexity, with the first 10 questions classified as easy tasks (E), the next 6 (Q11-Q16) as medium tasks (M), and the final 4 (Q17-Q20) as hard tasks (H). Table 5.2 reports the information sources and task type associated with each question, while Table 5.3 indicates the evaluation criteria applicable to each question.

Table 5.1. Evaluation question set. The evaluation is composed of 10 easy questions, 6 medium questions, and 4 hard questions.

ID	Question
Q1	What are the architecture and operational interfaces of FOS for HR-MS satellites in the IRIDE ground segment?
Q2	How does FOS manage and interact with the HR-MS satellites in the IRIDE system?
Q3	What are the requirements and procedures for developing software validation specifications according to ECSS-E-ST-40?
Q4	When is satellite SAT-01 next pass on GS-01?
Q5	How many passes does satellite SAT-02 have on 2026-05-26 on GS-02?
Q6	What is the current network status?
Q7	Is the connection to antenna ANT-02 of GS-01 online?
Q8	Which incidents are currently open?
Q9	Is there an open ticket for GS-02?
Q10	Is there an open ticket regarding power supplies?
Q11	Why is GS-01 degraded?
Q12	Which acquisitions are affected by incident INC-101?
Q13	Is the ground station assigned to SAT-02 available?
Q14	How many acquisitions are we going to lose if antenna ANT-01 is unavailable from 2026-05-26 10:00 to 2026-05-26 12:00?
Q15	When can SAT-01-PASS-032 be rescheduled according to its usual time schedule of past days? Give the next possible datetime.
Q16	Can the urgent acquisition request be accommodated?
Q17	Provide an overview of the current operational issues and their impact on scheduled satellite operations.
Q18	What operational issues require immediate attention and what mitigation actions should be applied?
Q19	Identify risks for planned activities on 2026-05-26 and report the affected activities with their corresponding datetime.
Q20	Summarize the current anomalies, explain their operational impact and recommend the next possible operational actions.

Table 5.2. Information sources and task type associated with each evaluation question. **Legenda:** KB = Knowledge Base, MP = Mission Planning, NM = Network Monitoring, TM = Ticket Management.

ID	KB	MP	NM	TM	Task type
Q1	X				Single-source retrieval
Q2	X				Single-source retrieval
Q3	X				Single-source retrieval
Q4		X			Single-source retrieval
Q5		X			Single-source retrieval
Q6			X		Single-source retrieval
Q7			X		Single-source retrieval
Q8				X	Single-source retrieval
Q9				X	Single-source retrieval
Q10				X	Single-source retrieval
Q11			X	X	Multi-source correlation
Q12		X		X	Multi-source correlation
Q13		X	X		Multi-source correlation
Q14		X			Planning impact analysis
Q15		X			Planning analysis
Q16		X		X	Decision support
Q17		X	X	X	Decision support
Q18	X		X	X	Decision support
Q19		X	X	X	Risk analysis
Q20	X	X	X	X	Decision support

Table 5.3. Applicability of evaluation criteria to the evaluation questions.

ID	AC	AI	EC	NF	DSS	ORQ
Q1	X			X		
Q2	X			X		
Q3	X			X		
Q4	X			X		
Q5	X			X		
Q6	X	X		X		
Q7	X	X		X		
Q8	X	X		X		
Q9	X	X		X		
Q10	X	X		X		
Q11	X	X	X	X		
Q12	X		X	X		
Q13	X		X	X		
Q14	X	X	X	X		
Q15	X			X		X
Q16	X	X	X	X	X	X
Q17	X	X	X	X	X	
Q18	X	X		X	X	X
Q19	X	X	X	X	X	
Q20	X	X	X	X	X	X

Legend: *AC*: Answer Correctness; *AI*: Anomaly Identification; *EC*: Evidence Correlation; *NF*: Noise Filtering; *DSS*: DSS Role Adherence; *ORQ*: Operational Recommendation Quality.

5.5 Results

5.5.1 Overall Results

The overall performance obtained by FUNES and the baseline across the three task-complexity levels is reported in Table 5.4. The scores are computed by aggregating the points obtained across all evaluation criteria applicable to the questions belonging to each difficulty category. The overall score therefore provides a quantitative comparison of the two systems across the complete evaluation set, while the subsequent analysis considers the results at the individual question and evaluation-criterion levels.

Table 5.4. Benchmark overall performance results for Llama 3.3 vs FUNES grouped by task difficulty. The scores are computed by aggregating the points obtained across all evaluation criteria applicable to the questions belonging to each difficulty category, their details are in Tables 5.5, 5.6, and 5.7.

Difficulty	**Llama 3.3**	**FUNES**
Easy	23/50 = 46.0%	48/50 = 96.0%
Medium	33/46 = 71.7%	40/46 = 87.0%
Hard	31/42 = 73.8%	36/42 = 85.7%
Overall	**87/138 = 63.0%**	**124/138 = 89.9%**

FUNES achieves a higher aggregate score than the baseline across all three task-complexity categories. The largest performance difference is observed for easy tasks, where FUNES substantially outperforms the baseline. The performance gap is smaller for medium and hard tasks, although FUNES maintains a higher aggregate score in both categories. Considering the complete evaluation set, FUNES obtains a higher overall score than the baseline, with the difference being observed across the full set of evaluated criteria. The question-level results and the contribution of the individual evaluation criteria are analysed in the following subsections.

5.5.2 Results by question complexity

The results were further analysed at the individual-question level to identify differences in system performance across tasks with different levels of complexity. Tables 5.5, 5.6, and 5.7 report the scores obtained by FUNES and the baseline for easy, medium, and hard questions, respectively. The tables also provide the scores obtained for each applicable evaluation criterion, allowing the differences between the two systems to be examined at a more detailed level.

For the easy tasks (Table 5.5), FUNES achieves a higher or equal score compared with the baseline for all questions. FUNES obtains the maximum applicable score for eight of the ten questions, while the baseline reaches the maximum score for Q7 only. Equal performance is observed for Q7, where both systems obtain the same overall score. At the aggregate level, the easy-task results therefore show a consistent difference in favour of FUNES across the evaluated questions.

The medium-task results reported in Table 5.6 show a smaller difference between

Table 5.5. Benchmark scores for easy questions, each metric has a maximum score of 2 points. The total score is computed by aggregating the points obtained across all evaluation criteria applicable to the question. **Legenda**: AC = Answer Correctness, AI = Anomaly Identification, EC = Evidence Correlation, NF = Noise Filtering, DSS = DSS Role Adherence, ORQ = Operational Recommendation Quality.

Question	LLM	AC	AI	EC	NF	DSS	ORQ	Total
Q1	llama	1	-	-	1	-	-	2/4
	FUNES	**2**	-	-	**2**	-	-	**4/4**
Q2	llama	1	-	-	0	-	-	1/4
	FUNES	**2**	-	-	**2**	-	-	**4/4**
Q3	llama	1	-	-	0	-	-	1/4
	FUNES	**2**	-	-	**2**	-	-	**4/4**
Q4	llama	2	-	-	1	-	-	3/4
	FUNES	2	-	-	**2**	-	-	**4/4**
Q5	llama	0	-	-	0	-	-	0/4
	FUNES	**2**	-	-	**2**	-	-	**4/4**
Q6	llama	1	2	-	0	-	-	3/6
	FUNES	**2**	2	-	**2**	-	-	**6/6**
Q7	llama	2	**2**	-	1	-	-	5/6
	FUNES	2	1	-	**2**	-	-	5/6
Q8	llama	1	1	-	1	-	-	3/6
	FUNES	**2**	**2**	-	**2**	-	-	**6/6**
Q9	llama	1	2	-	1	-	-	4/6
	FUNES	**2**	2	-	**2**	-	-	**6/6**
Q10	llama	0	1	-	0	-	-	1/6
	FUNES	**2**	1	-	**2**	-	-	**5/6**
TOTAL	llama	10/20	8/10	-/-	5/20	-/-	-/-	23/50
	FUNES	**20/20**	8/10	-/-	**20/20**	-/-	-/-	**48/50**

Table 5.6. Benchmark scores for medium questions, each metric has a maximum score of 2 points. The total score is computed by aggregating the points obtained across all evaluation criteria applicable to the question. **Legenda**: AC = Answer Correctness, AI = Anomaly Identification, EC = Evidence Correlation, NF = Noise Filtering, DSS = DSS Role Adherence, ORQ = Operational Recommendation Quality.

Question	LLM	AC	AI	EC	NF	DSS	ORQ	Total
Q11	llama	1	1	1	0	-	-	3/8
	FUNES	**2**	**2**	**2**	**2**	-	-	**8/8**
Q12	llama	1	-	2	2	-	-	5/6
	FUNES	1	-	2	2	-	-	5/6
Q13	llama	2	-	2	2	-	-	6/6
	FUNES	2	-	2	2	-	-	6/6
Q14	llama	0	1	0	2	-	-	3/8
	FUNES	**2**	1	**2**	2	-	-	**7/8**
Q15	llama	1	-	-	2	-	1	4/6
	FUNES	**2**	-	-	2	-	**2**	**6/6**
Q16	llama	**2**	2	**2**	**2**	2	**2**	**12/12**
	FUNES	1	2	1	1	2	1	8/12
TOTAL	llama	7/12	4/6	7/10	10/12	2/2	3/4	33/46
	FUNES	**10/12**	**5/6**	**9/10**	**11/12**	**2/2**	**3/4**	**40/46**

the two systems compared with the easy-task category. FUNES obtains a higher score than the baseline for Q11, Q14, and Q15, while the two systems obtain the same score for Q12 and Q13. For Q16, the baseline obtains a higher score than FUNES. The results therefore indicate comparable performance across most of the medium-complexity questions, with FUNES achieving a higher score in three of the six questions.

Considering the aggregate scores across all applicable evaluation criteria, FUNES obtains 40/46 points compared with 33/46 for the baseline. The largest differences are observed in Answer Correctness and Evidence Correlation. These results indicate that, within the evaluated medium-task benchmark, FUNES achieved a higher aggregate score than the baseline, although the limited number of evaluated questions does not support generalizing this result to medium-complexity tasks in general.

For the hard tasks, reported in Table 5.7, the relative performance varies across the individual questions. FUNES obtains a higher score than the baseline for Q17 and Q20, while both systems obtain the same score for Q18. The baseline obtains a higher score for Q19. Despite these differences at the individual-question level, FUNES achieves the higher aggregate score for the hard-task category.

Overall, the results show a progressively smaller performance difference between FUNES and the baseline as task complexity increases, in fact we can observe a difference of 25 raw points for easy task, 7 for the medium tasks, and 5 for the hard tasks. This trend should, however, be interpreted with caution, as the number of applicable criterion evaluations and the number of questions decreases across the

Table 5.7. Benchmark scores for hard questions, each metric has a maximum score of 2 points. The total score is computed by aggregating the points obtained across all evaluation criteria applicable to the question. **Legenda**: AC = Answer Correctness, AI = Anomaly Identification, EC = Evidence Correlation, NF = Noise Filtering, DSS = DSS Role Adherence, ORQ = Operational Recommendation Quality.

Question	LLM	AC	AI	EC	NF	DSS	ORQ	Total
Q17	llama	1	1	1	2	1	-	6/10
	FUNES	**2**	**2**	**2**	**2**	**2**	-	**10/10**
Q18	llama	1	2	-	2	1	1	7/10
	FUNES	1	2	-	2	1	1	7/10
Q19	llama	**2**	2	**2**	**2**	2	-	**10/10**
	FUNES	1	2	1	1	2	-	7/10
Q20	llama	1	1	1	2	2	1	8/12
	FUNES	**2**	**2**	**2**	**2**	**2**	**2**	**12/12**
TOTAL	llama	5/8	6/8	4/6	**8/8**	6/8	2/4	31/42
	FUNES	**6/8**	**8/8**	**5/6**	7/8	**7/8**	**3/4**	**36/42**

three task categories, we have respectively: 50-46-42 total raw score points for the three categories and 10-6-4 questions for each category. Moreover, the evaluation is based on a limited number of questions within a single controlled scenario and a single baseline model. Therefore, the observed differences provide an indication of the relative behaviour of the two systems within the considered benchmark, but should not be interpreted as evidence of a general performance trend with increasing task complexity. Further interpretation of these results, including the factors that may contribute to the observed differences, is provided in the Discussion section 5.6.

5.6 Discussion

5.6.1 Overall Comparison

The overall results indicate that FUNES provides a substantial improvement in the evaluation score of the generated responses compared with the baseline within the considered evaluation scenario. FUNES achieves an overall score of 89.9%, compared with 63.0% obtained by Llama 3.3 70B Instruct, corresponding to a difference of 26.9 percentage points. The performance advantage is consistently observed across all three task-complexity categories, with FUNES obtaining higher aggregate scores for easy, medium, and hard tasks.

The observed improvement is particularly relevant considering the configuration of the baseline. Llama 3.3 70B Instruct was provided with the complete operational context of approximately 24K tokens, including the information available across the different operational sources considered in the scenario. Therefore, the performance difference cannot be attributed simply to the baseline lacking access to the information required to answer the questions. Rather, the results indicate that providing the

complete context to a general-purpose LLM does not necessarily result in the most effective use of the available operational information. In contrast, FUNES explicitly selects and retrieves information according to the user query before generating the final response, this allows the system to focus on the most relevant information for each specific query and avoid to get lost in irrelevant details [39]. The observed improvement is consistent with the hypothesis that selective information retrieval and filtering contribute to the higher performance of FUNES, particularly for easy tasks where the questions are highly specific and the relevant information can be retrieved from a single source.

This result is also consistent with one of the main objectives of the proposed architecture, namely to support the underlying language model by controlling how heterogeneous operational information is selected and integrated into the reasoning process. However, the aggregate comparison alone does not establish that each individual architectural component is responsible for the observed improvement, as no ablation study was performed. The detailed results are therefore analysed in the following sections to identify the task types and evaluation criteria for which FUNES provides the largest benefits, as well as the cases in which the baseline achieves comparable or better performance.

5.6.2 Effect of task complexity

Easy task

The results for easy tasks show the largest performance difference between FUNES and the baseline. As reported in Table 5.4, FUNES achieves 48/50 points (96.0%), compared with 23/50 points (46.0%) obtained by the baseline. The difference is particularly pronounced for questions requiring the retrieval of information from a single source, where the task complexity is primarily associated with identifying the relevant information rather than performing complex operational reasoning.

A qualitative analysis of the individual questions provides an indication of the type of behaviour underlying this difference. For questions Q2 and Q3, which require the retrieval of document-based mission knowledge, the baseline included additional information concerning the current ground-segment operational situation, despite this information not being required to answer the questions. The responses therefore combined the requested document-based information with unrelated operational context. In contrast, FUNES selected the Knowledge Base as the relevant information source and restricted the retrieved context to the information required for answering the query. This resulted in higher scores for both answer correctness and noise filtering.

A more pronounced example is provided by Q5, which asks for the number of passages scheduled for SAT-03 on a specific date. The baseline produced an incorrect count of 32 passages and additionally included information about the network status, operational tickets, potential impacts, and recommended actions. The relevant planning information was therefore not only incorrectly interpreted, but was also accompanied by substantial unrelated context. FUNES, instead, retrieved the relevant planning information and identified the 16 passages corresponding to PASS-017 through PASS-032, providing a concise answer focused on the requested

information. The resulting score was 4/4 for FUNES compared with 0/4 for the baseline across the applicable evaluation criteria.

These examples suggest that, for simple retrieval-oriented tasks, the availability of the complete operational context does not necessarily lead to effective use of the relevant information. The baseline had access to the information required by the query, but its response included unrelated operational information and, in Q5, produced an incorrect result. FUNES instead used its source-selection and retrieval mechanisms to constrain the context available to the subsequent reasoning process. The observed behaviour is therefore consistent with the intended role of the FUNES architecture in reducing irrelevant contextual information before answer generation. However, these examples alone do not establish a causal relationship between the individual architectural components and the observed performance improvement, as the evaluation does not include an ablation study.

Medium Tasks

The results for medium tasks show a smaller but still consistent performance advantage for FUNES compared with the baseline. FUNES achieves 40/46 points (87.0%), while Llama 3.3 70B Instruct achieves 33/46 points (71.7%). Compared with the easy tasks, the difference between the two systems is therefore substantially reduced. This result suggests that the advantage associated with selective information retrieval becomes less pronounced when the task requires a higher degree of cross-source reasoning and operational interpretation. At the same time, FUNES maintains a higher aggregate performance, indicating that the proposed architecture continues to provide benefits for tasks involving multiple information sources.

A representative example is Q11, which asks why GS-01 is offline and requires the correlation of network monitoring information with incident and problem management records. FUNES obtained the maximum score of 8/8, while the baseline obtained 3/8. The baseline correctly identified the connectivity problem affecting GS-01/ANT-02, but extended the explanation to the degraded status of GS-01/ANT-03 without sufficient evidence that this condition was responsible for the station-level status. It also included information about SAT-01 scheduled activities and operational recommendations that were not required to answer the question. FUNES, instead, correlated the relevant network and ticket information while maintaining a more focused response. This example illustrates again the potential benefit of explicit source selection and information correlation when the answer depends on evidence distributed across multiple operational domains.

However, the results also show that the use of multi-source orchestration does not guarantee an improvement for every medium-complexity task. This is particularly evident in Q16, concerning the urgent acquisition request for SAT-01. In this case, the baseline achieved the maximum score of 12/12, whereas FUNES obtained 8/12. Both systems correctly identified the degraded condition of GS-01 and the relevant antenna anomalies, but the baseline provided a more direct assessment of the specific acquisition request and proposed alternative options for addressing it. FUNES instead focused primarily on the general resolution of the connectivity and antenna issues and did not sufficiently address the specific feasibility of the urgent acquisition

request. The judge therefore assigned lower scores to FUNES for answer correctness, evidence correlation, noise filtering, and operational recommendation quality.

The contrast between Q11 and Q16 is relevant for interpreting the role of the FUNES architecture. In Q11, the main challenge is to identify and correlate evidence distributed across different sources, which corresponds closely to the capabilities targeted by the retrieval and orchestration mechanisms. In Q16, however, the retrieved information must subsequently be transformed into a specific operational assessment and recommendation. The results therefore suggest that effective information retrieval and correlation are important but not sufficient by themselves to guarantee high-quality decision support. The final reasoning and synthesis stage remains critical, particularly when the question requires the system to evaluate the feasibility of a specific operational action rather than simply establish the relationship between available evidence.

Overall, the medium-task results indicate that FUNES retains an advantage when compared with a general-purpose LLM operating on the complete operational context, but that this advantage is less pronounced than for single-source retrieval tasks. The individual results also show that the benefit of the proposed architecture is task-dependent: explicit retrieval and orchestration appear particularly useful when relevant evidence must be identified and correlated across heterogeneous sources, while tasks requiring a more specific operational judgement may still be handled more effectively by the baseline.

Hard Task

The results for hard tasks show a smaller performance difference between FUNES and the baseline than that observed for easy and medium tasks. FUNES achieves 36/42 points (85.7%), while the baseline achieves 31/42 points (73.8%). The reduced performance gap is particularly relevant for these tasks, as they require the integration of heterogeneous information, the identification of operational consequences, and, in several cases, the formulation of operational recommendations. In contrast to the easy tasks, where the main challenge is often the identification of the relevant information, the hard tasks place a greater emphasis on the subsequent interpretation and integration of the retrieved evidence.

The results also show that the advantage of FUNES is not uniform across complex decision-support tasks. A representative counterexample is Q19, which asks the systems to identify risks affecting activities planned for a specific day and to report the corresponding affected activities and datetimes. In this case, the baseline achieved the maximum score of 10/10, while FUNES obtained 7/10. The judge particularly penalized FUNES for failing to provide the specific affected activities and corresponding datetimes requested by the question. Although FUNES correctly identified the relevant anomalies and their general operational implications, it did not complete the reasoning chain required by the query, namely linking these anomalies to the specific affected activities and their corresponding datetimes. This indicates that retrieving the relevant operational information does not by itself guarantee that the final synthesis will fully exploit the available evidence.

Q20 provides a contrasting example and represents the most comprehensive task in the benchmark, requiring the identification of current anomalies, the assessment

of their operational impact, and the formulation of possible operational actions. FUNES achieved the maximum score of 12/12, while the baseline obtained 8/12. The judge identified differences in anomaly identification, evidence correlation, and recommendation quality as the main sources of the performance gap. In particular, the FUNES response successfully incorporated a wider set of relevant anomalies, including the degraded resources at GS-01 and the network latency issue affecting GS-03, and connected these observations to their potential operational consequences. The resulting recommendations were also considered more specific and consistent with the available evidence.

The contrasting results of Q19 and Q20 are particularly relevant to the interpretation of the overall hard-task performance. They show that FUNES can obtain high scores on complex operational reasoning when the relevant information is correctly retrieved and integrated, but that the orchestration and retrieval process does not eliminate the possibility of errors during the final synthesis. Conversely, the baseline can occasionally exploit its access to the complete operational context to construct a more complete answer, as observed in Q19. Therefore, the results do not support the interpretation that selective retrieval in hard tasks is inherently superior to providing the complete context. Rather, they suggest that the benefit of selective retrieval depends on the ability of the architecture to preserve the relationships between the retrieved information and the specific requirements of the query.

This behaviour also provides a possible explanation for the smaller performance gap observed for hard tasks. As task complexity increases, the contribution of simple information filtering becomes less dominant, while the quality of cross-source reasoning and final answer synthesis becomes increasingly important. Both systems rely on the underlying language model for these capabilities, which may reduce the relative advantage provided by FUNES's retrieval and orchestration mechanisms. At the same time, the limited number of hard questions and applicable criterion evaluations prevents drawing strong conclusions about a systematic relationship between task complexity and relative performance. The observed results should therefore be interpreted as evidence of different system behaviours within the considered benchmark rather than as a general performance law.

5.6.3 Where FUNES Helps and Where It Does Not

The results indicate that the benefits of FUNES depend on the type of reasoning required by the task. The largest performance advantage is observed for easy tasks, where selecting the appropriate information source and limiting irrelevant context are central to producing a correct answer. The advantage decreases for medium and hard tasks, which place progressively greater emphasis on cross-source reasoning, operational impact assessment, and the generation of task-specific recommendations. This trend suggests that selective retrieval is particularly beneficial when information selection is a major source of difficulty, while its contribution becomes less decisive when the main challenge lies in the interpretation and synthesis of the retrieved evidence.

The individual task results also highlight a limitation of the current architecture. Although FUNES can effectively retrieve and correlate relevant information, it does

not always preserve the specific objective of the query throughout the final synthesis stage. In particular, identifying an anomaly does not necessarily ensure that the system correctly identifies all affected activities, reports the requested datetimes, or evaluates the feasibility of a specific operational action. Conversely, the results show that FUNES can successfully address complex decision-support tasks when the retrieved evidence is adequately connected to the requested operational assessment. These findings indicate that information retrieval and orchestration should be complemented by mechanisms that ensure the completeness and task-specific relevance of the final response.

These observations should be interpreted within the limits of the considered benchmark. The number of questions, particularly for hard tasks, is limited, and the absence of an ablation study prevents the individual contribution of retrieval, orchestration, and answer synthesis from being isolated. The results therefore demonstrate task-dependent performance differences between FUNES and the baseline, but do not establish a general relationship between task complexity and the effectiveness of selective retrieval.

5.6.4 Architectural Implications

The evaluation results provide several indications regarding the effectiveness of the architectural choices adopted in FUNES. Although the evaluation does not include an ablation study and therefore cannot isolate the contribution of individual components, the observed behaviour is consistent with the design objectives of the proposed architecture. In particular, the results highlight the importance of controlled information acquisition, agent specialization, and iterative evidence refinement in operational decision-support scenarios.

One of the most relevant implications concerns the separation between information acquisition and response generation. In the baseline configuration, the LLM receives the complete operational context and is responsible for determining which information is relevant to the query, this configuration is comparable to a setting in which the Synthesizer Agent receives the complete output of all available sources without prior filtering. In the evaluated tasks, this configuration was associated with the inclusion of irrelevant information and, in some cases, incorrect conclusions. FUNES instead introduces an explicit planning stage through the Planner Agent, which determines which information sources should be queried before the final response is generated. The strong performance observed for retrieval-oriented tasks is therefore consistent with the architectural decision to control the information flow before the reasoning stage rather than providing the complete operational context to a single LLM.

The results are also compatible with the intended role of specialized agents and tools for heterogeneous operational sources. The Knowledge Base, Planning, Network, and Ticket tools expose different types of information through a common interface, while specialized agents are responsible for adapting queries and filtering source-specific information. This separation is particularly relevant for medium-complexity tasks, where the information required to answer a question may be distributed across multiple operational domains. The performance observed in Q11, for example, is consistent with the intended role of the Planner and specialized

retrieval components, since the answer requires the correlation of information originating from different sources. Rather than requiring a single LLM to interpret the complete operational context, the architecture decomposes information acquisition into source-specific operations and subsequently provides their results to the Synthesizer Agent.

At the same time, the results suggest that the current critic-driven refinement mechanism could be extended beyond missing information retrieval. In Q19, the main limitation was not necessarily the absence of the relevant operational information, but the incomplete synthesis of the available evidence with respect to the exact requirements of the query. A refinement mechanism that evaluates not only whether sufficient evidence has been retrieved, but also whether each requested element has been explicitly addressed, could therefore improve the robustness of the final response.

The results also highlight the importance of the Synthesizer Agent as a distinct architectural component. The comparison between Q16, Q19, and Q20 suggests that the main challenge in complex tasks is not always information retrieval. In Q16 and Q19, the available evidence was not fully reflected in the final response with respect to the specific requirements of the query. Conversely, in Q20, the system successfully integrated anomalies, operational impacts, and recommendations and obtained the maximum score. This variability indicates that the final synthesis stage remains a critical component of the architecture.

The architecture therefore suggests a division of responsibilities between information selection and operational reasoning. The Planner and specialized agents primarily control which information enters the reasoning process, while the Synthesizer Agent is responsible for transforming that information into an operational assessment. The evaluation indicates that this separation is beneficial, particularly when irrelevant context is a major source of error, but that the boundary between retrieval and reasoning must remain sufficiently flexible for complex decision-support tasks. In particular, aggressive information filtering could potentially remove contextual relationships that are necessary for a complete operational assessment. The objective should therefore not be to minimize the amount of retrieved information, but to maximize its relevance to the specific requirements of the query.

Another architectural implication concerns the use of a shared state and graph-based orchestration. The evaluation tasks increasingly require information obtained at different stages of the investigation to be combined into a single response. The shared `AgentState` provides a persistent representation of the query, execution plan, retrieved evidence, synthesized response, and critic feedback, allowing subsequent investigation cycles to build upon previous results. This mechanism is particularly suitable for the iterative nature of operational investigations, where an initially observed anomaly may require additional queries to determine its cause and operational impact. The Q20 task illustrates a use case for which maintaining evidence collected from multiple operational sources is important. The shared state and graph-based orchestration mechanisms described in Section 4.3.4 are designed to support this type of iterative evidence integration

Finally, the results suggest that the main architectural advantage of FUNES should not be interpreted as the replacement of the underlying LLM reasoning capability. Instead, the architecture acts as a control layer around the LLM, deter-

mining which information is retrieved, how heterogeneous evidence is progressively collected, and when the generated assessment should be reconsidered. The observed 26.9 percentage-point improvement therefore provides empirical support for the overall architectural strategy, but does not establish that any single component is responsible for the improvement.

5.7 Limitations

The results presented in this evaluation should be interpreted in light of several limitations concerning the benchmark design, the comparison methodology, and the evaluation procedure.

First, the evaluation is based on a limited number of questions within a single controlled operational scenario. Although the questions cover different task-complexity levels and involve heterogeneous operational information sources, they represent only a subset of the situations that may occur in a real satellite ground-segment environment. In particular, the benchmark includes 20 questions in total, with 10 easy, 6 medium, and 4 hard questions. The relatively small number of questions, especially for the medium and hard-complexity categories, limits the statistical robustness of comparisons within individual task-complexity levels. In particular, the results for hard tasks are based on only four questions and may therefore be sensitive to the specific characteristics of the selected test cases. The observed performance differences should therefore be interpreted as evidence of the behaviour of the two evaluated systems within the considered benchmark, rather than as evidence of general performance across operational scenarios.

Second, the comparison considers a single baseline model, Llama 3.3 70B Instruct. The results therefore establish a comparison between FUNES and this specific baseline configuration, but do not determine whether similar improvements would be obtained with other LLMs or alternative prompting and context-management strategies. In particular, the observed performance difference should not be interpreted as evidence that FUNES universally outperforms general-purpose LLMs.

Third, the experiments were conducted through the Groq API, which imposes limits on the number of requests and the number of tokens that can be processed within a given time period. These constraints limited the extent to which the evaluation could be repeated under identical experimental conditions and may therefore affect the reproducibility of large-scale repeated evaluations. The benchmark was consequently evaluated under the available API constraints rather than through an unrestricted experimental setup. Future evaluations could mitigate this limitation by using an inference environment with fewer restrictions, allowing a larger number of repeated runs and a more robust estimation of performance variability.

Fourth, the evaluation does not include an ablation study of the FUNES architecture. The system combines several mechanisms, including source selection, specialized retrieval components, orchestration, iterative evidence refinement, and final response synthesis. Since these components are evaluated as part of the complete system, the benchmark cannot isolate the contribution of any individual architectural mechanism to the observed performance. The results therefore support conclusions about the behaviour of FUNES as a complete architecture, but not

causal conclusions about the effectiveness of individual components.

Fifth, the evaluation relies on an LLM-based judge to assign scores according to the defined evaluation criteria. Although the criteria and scoring scale were explicitly specified and the judge was instructed to evaluate the answers against the operational context, LLM-based evaluation may still introduce evaluator-dependent variability. The obtained scores should therefore be interpreted as the outcome of the adopted evaluation procedure rather than as an absolute ground truth of response quality. A complementary evaluation involving human domain experts, multiple independent judges, or inter-rater agreement analysis would provide stronger evidence for the reliability of the evaluation.

Sixth, the benchmark uses a controlled operational context rather than live operational data. The evaluation therefore does not assess the behaviour of FUNES under conditions such as continuously changing system states, concurrent operational events, incomplete or inconsistent information, real-time timing constraints, or unexpected failures. These factors may affect both retrieval and decision-support performance in an operational deployment and should be considered in future validation activities.

Finally, the aggregate scoring procedure assigns the same maximum weight to each applicable evaluation criterion. This provides a simple and consistent comparison between the two systems, but it does not necessarily reflect the relative operational importance of different error types. For example, an incorrect operational recommendation may have substantially different consequences from the inclusion of a limited amount of irrelevant information. A future evaluation could therefore incorporate criterion-specific weighting or risk-oriented metrics reflecting the operational criticality of different types of errors.

These limitations constrain the interpretation and generalizability of the observed differences between FUNES and the baseline. The results provide evidence of the relative behaviour of the two complete systems within the considered benchmark, but broader claims regarding generalization across task types and operational scenarios, component-level effectiveness, evaluation robustness, and real-world deployment require further validation.

Chapter 6

Future Work

The results obtained in this evaluation provide several directions for future research. These directions concern both the evolution of the FUNES architecture and the broader evaluation methodology, including the underlying language models, benchmark design, and validation in operational settings. The main objective is to determine whether the improvements observed in the current benchmark generalize beyond the considered scenario and to identify which architectural mechanisms are responsible for the observed behaviour.

6.1 Architectural Improvements

A first direction concerns the extension of the retrieval and orchestration mechanisms towards more explicit requirement-aware reasoning. The current architecture focuses primarily on identifying and retrieving the information required to answer a query, while the evaluation results show that some failures occur even when the relevant evidence has already been retrieved. In particular, the results for Q19 indicate that the system may identify the relevant anomalies without fully mapping them to all the entities, attributes, and temporal information explicitly requested by the user. Future work could therefore introduce a requirement-tracking mechanism that decomposes the query into explicit answer requirements and verifies, during the synthesis and validation stages, whether each requirement has been addressed. Such a mechanism could verify, for example, that all requested activities, datetimes, affected resources, and operational relationships are explicitly represented in the final response.

The representation of evidence could also be extended through explicit provenance and relationship tracking. Rather than providing the Synthesizer Agent with a collection of retrieved facts, future implementations could maintain an evidence graph representing relationships between anomalies, assets, operational activities, tickets, planning events, and observed impacts. Each element could retain its source and temporal information, allowing the final reasoning stage to distinguish directly observed facts from inferred relationships. This could improve both answer traceability and the ability of the Critic Agent to verify whether the conclusions generated by the system are supported by the available evidence.

The Critic Agent could consequently be extended beyond the current focus on

evidence sufficiency and retrieval refinement. Future implementations could introduce requirement-level validation, factual consistency checks, temporal consistency checks, and provenance verification. For example, the Critic could explicitly verify that every activity identified as affected by an anomaly is supported by the available planning evidence and that the corresponding datetime is correctly reported. This would shift part of the validation process from checking whether sufficient information has been retrieved to checking whether the retrieved information has been correctly transformed into the requested operational conclusion.

Another possible extension concerns uncertainty and confidence representation. Operational decision-support systems must distinguish between confirmed observations, supported inferences, and hypotheses that require further investigation. FUNES could therefore associate confidence or evidence-status information with retrieved facts and generated conclusions. This would allow the Synthesizer Agent to explicitly communicate when a causal relationship is not established by the available evidence, reducing the risk of presenting plausible operational interpretations as confirmed facts.

Finally, future work could investigate adaptive and cost-aware orchestration. The current architecture prioritizes information relevance and answer quality, but operational deployment also requires controlling latency, computational cost, number of tool calls, and token consumption. The orchestration strategy could therefore be optimized jointly for answer quality and resource consumption. This would allow the system to determine when an additional retrieval or refinement cycle provides sufficient expected benefit to justify its computational cost.

6.2 LLM and Reasoning Strategies

A second direction concerns the interaction between the FUNES architecture and the underlying language model. The current evaluation compares FUNES with a single baseline based on Llama 3.3 70B Instruct. Although this provides a controlled comparison, it does not establish whether the observed benefits are independent of the characteristics of the underlying model. Future work should therefore evaluate FUNES with multiple LLM families and model sizes, including both smaller and larger models.

In particular, an interesting research direction would be to investigate whether explicit information orchestration can compensate, at least partially, for the lower reasoning capacity of smaller language models. Experiments could compare a large LLM operating on the complete operational context with a smaller LLM operating within the FUNES architecture. Such a comparison would provide evidence on whether the architectural mechanisms can reduce the dependence on increasingly large models for operational decision-support tasks.

Future evaluations could also investigate different reasoning and prompting strategies within the same FUNES architecture. This could include structured intermediate representations, explicit reasoning plans, constrained output formats, and different approaches to final answer synthesis. The objective would not simply be to identify the highest-scoring configuration, but to determine whether the architectural separation between information acquisition and reasoning remains beneficial

across different LLM configurations.

Another important direction concerns the evaluation of the LLM-based judge itself. The current benchmark relies on an evaluator model to assign scores according to the defined criteria. Future work should validate the reliability of this evaluation methodology through comparison with human domain experts and, where possible, multiple independent evaluator models. Agreement between different evaluators could be analysed to identify criteria for which automated evaluation is stable and criteria for which human judgement remains particularly important.

The evaluation could also be extended from absolute criterion scoring to pairwise comparison. In addition to assigning independent scores to each system, evaluators could determine which of two responses is more correct or operationally useful with respect to the same ground truth. Combining absolute scoring and pairwise preference could provide a more robust evaluation of relative system performance and reduce the sensitivity of the results to the boundaries of the discrete scoring scale.

6.3 Benchmark and Evaluation Extensions

The current benchmark is intentionally limited to a controlled operational scenario and a relatively small number of questions. A primary direction for future work is therefore the construction of a larger and more diverse benchmark. Increasing the number of questions would reduce the influence of individual cases such as Q16 and Q19 and would allow performance differences to be analysed with greater statistical confidence.

The benchmark should also include a broader range of operational scenarios, ground stations, satellites, anomalies, planning constraints, and combinations of operational events. This would allow the evaluation to determine whether the observed behaviour is specific to the current scenario or persists across different configurations of the ground segment.

A more systematic definition of task complexity would also be beneficial. Instead of assigning questions to broad easy, medium, and hard categories, future benchmarks could define complexity along multiple dimensions, such as number of information sources, number of reasoning steps, number of entities involved, temporal dependencies, degree of ambiguity, and requirement for operational recommendations. This would allow performance to be analysed as a function of specific task characteristics rather than a single categorical notion of difficulty.

Future benchmark versions should also include a larger proportion of multi-source and multi-hop questions. These tasks would provide a more direct evaluation of the capabilities that motivate the FUNES architecture, particularly the ability to identify relevant sources, correlate heterogeneous evidence, and preserve relationships across multiple retrieval steps. The benchmark could explicitly control the number of sources required to answer each question, enabling the relationship between source heterogeneity and system performance to be investigated.

Another important extension would be the introduction of noisy and adversarial contexts. Irrelevant information, redundant records, temporally similar events, and operational anomalies that are correlated but not causally related could be delib-

erately introduced into the context. This would allow the robustness of FUNES and the baseline to be evaluated under controlled levels of information overload and ambiguity. Such experiments would provide a stronger test of the hypothesis that explicit information selection can reduce errors caused by irrelevant contextual information.

The benchmark should also include questions for which the available evidence is intentionally insufficient to determine a unique answer. These cases are particularly relevant for operational decision support, since a reliable system should be able to distinguish between a supported conclusion and a situation requiring additional investigation. Evaluating the ability to explicitly identify insufficient evidence would therefore complement the current focus on answer correctness.

Temporal reasoning represents another important benchmark dimension. Satellite ground-segment operations are inherently time-dependent, and future benchmark scenarios should explicitly test relationships between anomaly occurrence, planned activities, maintenance windows, acquisition opportunities, and subsequent operational impacts. Questions could require systems to identify temporal conflicts, determine whether a planned activity is affected by an anomaly, or reconstruct the sequence of events leading to an operational condition.

Finally, controlled ablation experiments should be introduced to isolate the contribution of individual FUNES components. Variants removing or modifying the Planner, specialized retrieval agents, Critic, iterative refinement, or Synthesizer could be compared under the same benchmark conditions. Such experiments would allow the architectural contribution of each component to be quantified independently and would provide stronger evidence for the architectural claims than the current end-to-end comparison alone.

6.4 Operational Validation

A final direction concerns validation in more realistic operational settings. The current evaluation measures the correctness of generated responses against a predefined operational ground truth. While this is necessary for controlled benchmarking, it does not directly measure whether the generated information improves the work of human operators.

Future studies should therefore involve domain experts in the evaluation process. Expert operators could assess not only factual correctness, but also operational usefulness, relevance, trustworthiness, clarity of uncertainty, and suitability of proposed actions. Human evaluation would provide a complementary perspective to automated scoring and could identify failure modes that are difficult to capture through predefined evaluation criteria.

The effect of FUNES on the decision-making process could also be evaluated directly. Metrics such as time-to-decision, number of required information queries, number of operator interventions, and rate of correct operational decisions could provide evidence on whether the system produces a measurable benefit beyond response-level accuracy. This would shift the evaluation from assessing the quality of individual answers towards assessing the effectiveness of the overall human-AI decision-support process.

Another relevant research direction is the evaluation of explainability and evidence provenance. A future version of FUNES could explicitly expose the evidence supporting each major conclusion, allowing operators to trace an operational assessment back to the corresponding network measurements, tickets, planning records, or knowledge-base entries. Experiments could then investigate whether this form of evidence-grounded explanation improves operator trust, verification speed, and resistance to unsupported model conclusions.

Finally, FUNES should be evaluated under more dynamic operating conditions. The current benchmark is based on a controlled snapshot of operational information, whereas real ground-segment environments involve continuously changing network states, planning updates, newly generated tickets, and evolving investigations. Future experiments could therefore evaluate long-running investigations in which the system maintains state across multiple queries and updates its assessment as new evidence becomes available. This would provide a more realistic validation of the shared state and graph-based orchestration mechanisms and would assess whether FUNES can maintain consistency across successive investigation cycles.

Overall, these research directions would extend the current evaluation from a controlled comparison of two response-generation configurations towards a broader investigation of information orchestration, LLM reasoning, evidence-grounded decision support, and human-AI interaction. Such extensions would make it possible to determine not only whether FUNES improves response quality in the considered benchmark, but also which architectural mechanisms produce the improvement, under which task conditions they remain effective, and whether the resulting benefits translate into measurable improvements in operational decision support.

Chapter 7

Conclusion

This thesis investigated the use of a multi-agent, tool-based architecture for LLM-driven decision support in a satellite ground-segment operational context. The main objective was to address a limitation of conventional LLM-based approaches, namely the difficulty of effectively exploiting large and heterogeneous operational contexts when relevant information is distributed across different sources. To this end, FUNES was designed as an orchestration layer around an underlying language model, introducing explicit mechanisms for information planning, source-specific retrieval, evidence refinement, and response synthesis.

The proposed architecture was evaluated through a controlled benchmark consisting of twenty operational questions divided into three task-complexity categories. The evaluation compared FUNES with Llama 3.3 70B Instruct operating directly on the complete operational context. The results show that FUNES achieved an overall score of 124/138 (89.9%), compared with 87/138 (63.0%) for the baseline, corresponding to a difference of 26.9 percentage points. FUNES obtained a higher aggregate score in all three task-complexity categories, with the largest difference observed for easy tasks and progressively smaller differences for medium and hard tasks. The question-level analysis further showed that the advantage of FUNES was particularly pronounced for retrieval-oriented tasks and for cases where relevant evidence had to be selected from heterogeneous operational sources. At the same time, the results included cases in which the baseline achieved equal or higher performance, particularly when the task required detailed operational synthesis or a precise mapping between evidence and requested operational consequences.

These results provide empirical support for the overall design principle underlying FUNES: explicitly controlling the acquisition and organization of operational information can improve the quality of LLM-generated decision-support responses compared with providing a general-purpose LLM with the complete operational context. In particular, the observed improvements are consistent with the hypothesis that source selection and contextual filtering can reduce the impact of irrelevant information and facilitate the subsequent reasoning process. However, the evaluation does not establish that any individual architectural component is responsible for the observed improvement. The absence of an ablation study, the limited number of questions, the use of a single operational scenario and baseline model, and the reliance on automated evaluation limit the extent to which the results can be

generalized beyond the considered benchmark.

An important outcome of the evaluation is therefore that information retrieval and orchestration should not be considered a substitute for the reasoning and synthesis capabilities of the underlying LLM. The results for the more complex tasks show that retrieving the correct evidence is not sufficient when the system must transform that evidence into a precise operational judgement, preserve detailed temporal and causal relationships, or formulate a specific operational recommendation. This suggests that future development of FUNES should focus not only on improving information acquisition, but also on requirement-aware synthesis, evidence provenance, uncertainty handling, and iterative validation of the final response.

Overall, FUNES demonstrates the potential of an explicit information-orchestration architecture for integrating LLMs into satellite ground-segment decision-support workflows. Within the scope of the evaluated scenario, the architecture achieved a substantial improvement over the considered full-context baseline while also revealing clear conditions under which this advantage is reduced or absent. Future work should therefore extend the evaluation to larger and more diverse benchmarks, perform controlled ablation studies, investigate different underlying LLMs and reasoning strategies, and validate the system with domain experts in realistic operational environments. These extensions would allow the contribution of the individual architectural mechanisms to be isolated and would determine whether the improvements observed in the current benchmark translate into measurable benefits for real-world operational decision support.

Bibliography

[1] Alansari, A. and Luqman, H. Large language models hallucination: A comprehensive survey. *arXiv preprint arXiv:2510.06265*, (2025).

[2] Alkayyal, M., Malberg, S., and Groh, G. An llm-based decision support system for strategic decision-making. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pp. 460–464. Springer (2025).

[3] Anh-Hoang, D., Tran, V., and Nguyen, L.-M. Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior. *Frontiers in Artificial Intelligence*, **Volume 8 - 2025** (2025). Available from: `https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1622292`, `doi:10.3389/frai.2025.1622292`.

[4] Bengio, Y., Ducharme, R., Vincent, P., and Jauvin, C. A neural probabilistic language model. *Journal of machine learning research*, **3** (2003), 1137.

[5] Besta, M., et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, pp. 17682–17690 (2024).

[6] Bianchi, O., et al. Hidden in the haystack: Smaller needles are more difficult for llms to find. *arXiv preprint arXiv:2505.18148*, (2025).

[7] Boiko, D. A., MacKnight, R., Kline, B., and Gomes, G. Autonomous chemical research with large language models. *Nature*, **624** (2023), 570.

[8] Bojanowski, P., Grave, E., Joulin, A., and Mikolov, T. Enriching word vectors with subword information. *CoRR*, **abs/1607.04606** (2016). Available from: `http://arxiv.org/abs/1607.04606`, `arXiv:1607.04606`.

[9] Bommasani, R., et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, (2021).

[10] Brown, T., et al. Language models are few-shot learners. *Advances in neural information processing systems*, **33** (2020), 1877.

[11] Chen, S., Wong, S., Chen, L., and Tian, Y. Extending context window of large language models via positional interpolation (2023). Available from: https://arxiv.org/abs/2306.15595, arXiv:2306.15595.

[12] Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, **30** (2017).

[13] Cormack, G. V., Clarke, C. L., and Buettcher, S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 758–759 (2009).

[14] Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems* (edited by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh), vol. 35, pp. 16344–16359. Curran Associates, Inc. (2022). Available from: https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.

[15] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186 (2019).

[16] Eigner, E. and Händler, T. Determinants of llm-assisted decision-making. *arXiv preprint arXiv:2402.17385*, (2024).

[17] Elman, J. L. Finding structure in time. *Cognitive science*, **14** (1990), 179.

[18] European Cooperation for Space Standardization. Space Engineering: System Engineering General Requirements. ECSS Standard ECSS-E-ST-10C Rev.1, ESA Requirements and Standards Division, ESTEC, Noordwijk, The Netherlands (2017). Available from: https://ecss.nl/standard/ecss-e-st-10c-rev-1-system-engineering-general-requirements-15-february-2017/.

[19] European Cooperation for Space Standardization. Space Engineering: Software. ECSS Standard ECSS-E-ST-40C Rev.1, ESA Requirements and Standards Division, ESTEC, Noordwijk, The Netherlands (2025). Available from: https://ecss.nl/standard/ecss-e-st-40c-rev-1-software-30-april-2025/.

[20] European Cooperation for Space Standardization. Space Product Assurance: Software Product Assurance. ECSS Standard ECSS-Q-ST-80C Rev.2, ESA Requirements and Standards Division, ESTEC, Noordwijk, The Netherlands (2025). Available from: https://ecss.nl/standard/ecss-q-st-80c-rev-2-software-product-assurance-30-april-2025/.

[21] Fink, G. A. n-gram models. In *Markov Models for Pattern Recognition: From Theory to Applications*, pp. 107–127. Springer (2014).

[22] Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, (2022).

[23] Gordon, M., Duh, K., and Andrews, N. Compressing bert: Studying the effects of weight pruning on transfer learning. In *Proceedings of the 5th Workshop on Representation Learning for NLP*, pp. 143–155 (2020).

[24] Grosse, R. Lecture 15: Exploding and vanishing gradients. *University of Toronto Computer Science*, (2017).

[25] Han, Z., Gao, C., Liu, J., Zhang, J., and Zhang, S. Q. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*, (2024).

[26] Harris, Z. S. Distributional structure. *Word*, **10** (1954), 146.

[27] Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, (2015).

[28] Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural computation*, **9** (1997), 1735.

[29] Hoffmann, J., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, **10** (2022).

[30] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *Iclr*, **1** (2022), 3.

[31] Ji, Z., et al. Survey of hallucination in natural language generation. *ACM computing surveys*, **55** (2023), 1.

[32] Jiang, Z., Araki, J., Ding, H., and Neubig, G. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, **9** (2021), 962. Available from: `https://doi.org/10.1162/tacl_a_00407`, `arXiv:https://direct.mit.edu/tacl/article-pdf/doi/10.1162/tacl_a_00407/1962628/tacl_a_00407.pdf`, `doi:10.1162/tacl_a_00407`.

[33] Kalyuzhnaya, A., et al. Llm agents for smart city management: Enhancing decision support through multi-agent ai systems. *Smart Cities*, **8** (2025). Available from: `https://www.mdpi.com/2624-6511/8/1/19`, `doi:10.3390/smartcities8010019`.

[34] Kaplan, J., et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, (2020).

[35] Lewis, P., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, **33** (2020), 9459.

[36] Li, J., et al. Large language model inference acceleration: A comprehensive hardware perspective. *arXiv preprint arXiv:2410.04466*, (2024).

[37] Li, K. and Whittle, R. S. Toward an Earth-Independent System for EVA Mission Planning: Integrating Physical Models, Domain Knowledge, and Agentic RAG to Provide Explainable LLM-Based Decision Support. In *Advancing Human-Computer Interaction for Space Exploration (SpaceCHI 2025)* (edited by L. Bensch, T. Nilsson, M. Nisser, P. Pataranutaporn, A. Schmidt, and V. Sumini), vol. 130 of *Open Access Series in Informatics (OASIcs)*, pp. 6:1–6:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2025). ISBN 978-3-95977-384-3. Available from: `https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.SpaceCHI.2025.6`, `doi:10.4230/OASIcs.SpaceCHI.2025.6`.

[38] Lin, J., et al. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In *Proceedings of Machine Learning and Systems* (edited by P. Gibbons, G. Pekhimenko, and C. D. Sa), vol. 6, pp. 87–100 (2024). Available from: `https://proceedings.mlsys.org/paper_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf`.

[39] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, **12** (2024), 157. Available from: `https://aclanthology.org/2024.tacl-1.9/`, `doi:10.1162/tacl_a_00638`.

[40] Manning, C. D. *Introduction to information retrieval*. Syngress Publishing, (2008).

[41] Maranto, D. Llmsat: A large language model-based goal-oriented agent for autonomous space exploration. *arXiv preprint arXiv:2405.01392*, (2024).

[42] Mikolov, T., Chen, K., Corrado, G., and Dean, J. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, (2013).

[43] Öğdü, Ç. U., Arslanoğlu, K., and Karaköse, M. An adaptive multi-agent llm-based clinical decision support system integrating biomedical rag and web intelligence. *IEEE Access*, (2025).

[44] Ouyang, L., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, **35** (2022), 27730.

[45] Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface*

Software and Technology, UIST '23. Association for Computing Machinery, New York, NY, USA (2023). ISBN 9798400701320. Available from: https://doi.org/10.1145/3586183.3606763, doi:10.1145/3586183.3606763.

[46] Peng, B., Quesnelle, J., Fan, H., and Shippole, E. Yarn: Efficient context window extension of large language models (2023). arXiv:2309.00071.

[47] Pennington, J., Socher, R., and Manning, C. D. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1532–1543 (2014).

[48] Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)* (edited by M. Walker, H. Ji, and A. Stent), pp. 2227–2237. Association for Computational Linguistics, New Orleans, Louisiana (2018). Available from: https://aclanthology.org/N18-1202/, doi:10.18653/v1/N18-1202.

[49] Qader, W. A., Ameen, M. M., and Ahmed, B. I. An overview of bag of words; importance, implementation, applications, and challenges. In *2019 international engineering conference (IEC)*, pp. 200–204. IEEE (2019).

[50] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, **1** (2019), 9.

[51] Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, **36** (2023), 53728.

[52] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. Learning representations by back-propagating errors. *nature*, **323** (1986), 533.

[53] Sanh, V., Debut, L., Chaumond, J., and Wolf, T. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, (2019).

[54] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, **36** (2023), 68539.

[55] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, (2017).

[56] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, **36** (2023), 8634.

[57] Sreenivas, S. T., et al. Llm pruning and distillation in practice: The minitron approach. *arXiv preprint arXiv:2408.11796*, (2024).

[58] Stiennon, N., Ouyang, L., Wu, J., Ziegler, D., Lowe, R., Voss, C., Radford, A., Amodei, D., and Christiano, P. F. Learning to summarize with human feedback. *Advances in neural information processing systems*, **33** (2020), 3008.

[59] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, **568** (2024), 127063.

[60] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, **30** (2017).

[61] Wang, L., et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, **18** (2024), 186345.

[62] Wang, S., Li, B. Z., Khabsa, M., Fang, H., and Ma, H. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, (2020).

[63] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, (2022).

[64] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, **35** (2022), 24824.

[65] Wei, J., et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, (2022).

[66] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pp. 38087–38099. PMLR (2023).

[67] Xu, Y., Xie, L., Gu, X., Chen, X., Chang, H., Zhang, H., Chen, Z., Zhang, X., and Tian, Q. Qa-lora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*, (2023).

[68] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, **36** (2023), 11809.

[69] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations* (2022).

[70] Zheng, L., et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, **36** (2023), 46595.